408 详解 · 2016 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2016-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

>

读法：先看标题话题与答案，再按 6 节往下读；每题自成体系，前置概念在题内讲清。

>

部分核对说明：以下几题的个别细节以原卷为准，题内也已标注——第 2 题错误选项的指针语句原文、第 6 题有向图的边向、第 8 题通向顶点 4 的边、第 15 题程序段原文、第 25 题资源申请表原文、第 26 题选项排列、第 30 题进程与线程的归属、第 37 题图上各路由器到目标网络的原始距离。其余各题按给定数据推导。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 单链表插入新结点后各结点链接地址指针的变化（正确答案：D）

题干与选项：

已知表头元素为 c 的单链表在内存中的存储状态如下表所示。现将 f 存放于 1014H 处并插入到单链表中，若 f 在逻辑上位于 a 和 e 之间，则 a，e，f 的"链接地址"依次是（ ）。

- A. 1010H，1014H，1004H
- B. 1010H，1004H，1014H
- C. 1014H，1010H，1004H
- D. 1014H，1004H，1010H

> 注：原卷此处为单链表内存存储状态表（表头元素为 c），插图以原卷为准。

入手点： 先把插入前后的链画出来：插入前 a 指向 e，e 指向下一个结点；插入后 a 要指向 f，f 要指向 e，e 自己指向谁不用动。谁的指针被改了，就改谁；没被改的，原样写。

专有名词从零解释：

- 先想“单链表”：一串结点，每个结点除了存数据，还存一个“下一个结点在哪”的地址；找结点只能顺着地址一个一个往下走。
- 再想“链接地址”：就是结点里存的“下一个结点的地址”；改链接地址，就是改“下一个是谁”。
- 再想“插入”：把新结点塞进两个结点之间；要改两处指针——前一个结点改指新结点，新结点改指后一个结点；后一个结点自己的指针不动。

考查知识点：

- 先想插入前：a 在 1000H 处，a 的链接地址是 1010H，也就是 e 的位置；e 在 1010H 处，e 的链接地址是 1004H，也就是下一个结点的位置。
- 再想插入后：f 放在 1014H 处，逻辑上在 a 和 e 之间；a 的下一个变成 f，所以 a 的链接地址改成 1014H；f 的下一个是 e，所以 f 的链接地址是 1010H；e 的下一个还是原来的结点，所以 e 的链接地址保持 1004H 不变。
- 拿生活例子验算：排队时 a 后面原来是 e，现在把 f 插到 a 和 e 之间；a 只要记住“后面是 f”，f 记住“后面是 e”，e 记住“后面是谁”不用变；三个人里改两个人的记忆就够了。
- 边界情况：如果 f 插在表头前面，还要改表头指针；如果 f 插在表尾后面，f 的链接地址就是空（NULL）；本题 f 在中间，所以只改 a 和 f。
- 易混对子：插入改的是“前一个结点”和“新结点”的指针，不是“后一个结点”的；把 e 的指针也改了，就会把链指错地方。

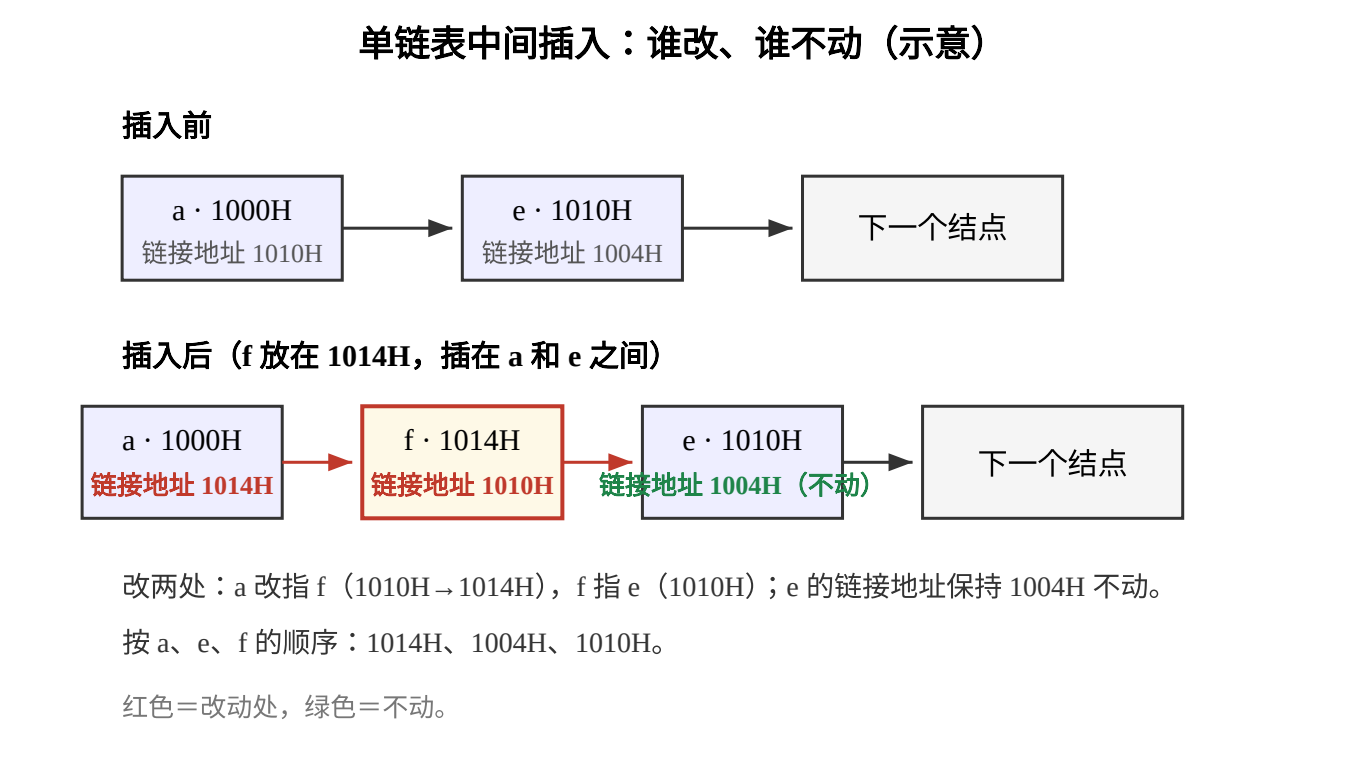
正确选项为什么对： a 的链接地址变成 1014H，e 的链接地址保持 1004H，f 的链接地址是 1010H；按 a、e、f 的顺序写就是 1014H、1004H、1010H，就是 D，选 D。

错误选项为什么错＋举例：

- A（1010H、1014H、1004H）：a 仍是 1010H，没改指 f，插入根本没接进来；e 还被改成了 1014H（指 f），f 指 1004H（e 的下一个，跳过了 e）；三处里 a 和 f 都错。教训：插入时前一个结点的指针必须改指新结点。
- B（1010H、1004H、1014H）：a 仍是 1010H，没改指 f；e 的 1004H 是对的；但 f 指 1014H，也就是 f 自己，自指成环，链断在 f。教训：新结点的指针要指后一个结点 e 本身，不是自己。
- C（1014H、1010H、1004H）：a 的 1014H 是对的；但 e 被改成了 1010H（e 自己），f 指 1004H（e 的下一个，跳过了 e）；e 自指、f 跳 e，链全乱了。教训：后一个结点 e 的指针不该动，f 该指 e 本身。

笔记本一句话： 第 1 题＋单链表中间插入：改前一个结点指新结点、新结点指后一个结点，后一个结点不动。

【图像】



单链表插入示意图：a 的指针由指 e 改为指 f，f 的指针指 e，e 的指针不动

【定位】 数据结构 · 第2章 线性表 · 单链表的插入操作

【易混对比】

- "改前一个结点的指针"和"改后一个结点的指针"是两回事：插入时改的是 a（前一个）和 f（新结点）的指针，e（后一个）的指针纹丝不动——谁的"下一个"变了才改谁。
- "链接地址"和"结点地址"是两回事：结点地址是结点自己住哪（a 住 1000H），链接地址是结点里存的"下一个住哪"（a 的链接地址原来是 1010H）——题目问的是链接地址的变化。
- "f 指 e"和"f 指自己"是两回事：f 的链接地址是 e 的地址 1010H；写成 1014H（f 自己）就自指成环，链从 f 处断掉。

插入后 a 的链接地址改成 1014H（指 f），f 的链接地址是 1010H（指 e），e 的链接地址保持 1004H 不变；按 a、e、f 的顺序写就是 1014H、1004H、1010H，选 D。

【坑位】

后一个结点 e 的指针不许动，动了就把链指错。

最容易踩的坑是：觉得"插入 f，a、e、f 三个结点的指针都得变"，于是把 e 的链接地址也改了。为什么错：插入只改变"谁的下一个变了"——a 的下一个从 e 变成 f，f 的下一个是 e，这两个"下一个"变了；e 的下一个还是原来的结点，没变。正确做法：画出插入前后的两条链（a→e 和 a→f→e），只改指向发生变化的两个结点的指针。

【套路】

- 第一步，画出插入前后的链：插入前 a→e，插入后 a→f→e。
- 第二步，定"谁改谁不动"：前一个结点 a 改指新结点 f，新结点 f 指后一个结点 e，后一个结点 e 不动。
- 第三步，按题目要求的顺序写地址：本题按 a、e、f 的顺序写成 1014H、1004H、1010H。

中间插入：前指新，新指后，后不动；按题序写地址。

【类比】

像排队时加塞：a 后面原来是 e，现在把 f 插到 a 和 e 之间。a 只要记住"后面是 f"，f 记住"后面是 e"，e 记住"后面是谁"不用变——三个人里改两个人的记忆就够了。术语贴回去：改的就是"链接地址"，即结点里存的"下一个结点的地址"。

【口诀】 中间插入改两处，前指新人指后。

【考频】 「单链表插入」在 2009–2022 年解析中出现 30 次 · 高频

第 2 题 · 带头结点双向循环链表删除指定结点的指针调整（正确答案：D）

题干与选项：

已知一个带有表头结点的双向循环链表 L，结点结构为 prev data next，其中，prev 和 next 分别是指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点，正确的语句序列是（ ）。

- A. p->next->prev = p->prev; p->prev->next = p->prev; free(p);
- B. p->next->prev = p->next; p->prev->next = p->next; free(p);
- C. p->next->prev = p->next; p->prev->next = p->prev; free(p);
- D. p->next->prev = p->prev; p->prev->next = p->next; free(p);

入手点： 删除就是“把 p 从链上摘下来”：先让 p 的前一个结点直接指 p 的后一个结点，再让 p 的后一个结点直接指 p 的前一个结点，最后把 p 占的地方还回去；两边都要接好，缺一边链就断了。

专有名词从零解释：

- 先想“双向链表”：每个结点有两个指针，一个指直接前驱（prev），一个指直接后继（next）；既能往前走，也能往后走。
- 再想“循环链表”：最后一个结点的 next 指回第一个结点，第一个结点的 prev 指回最后一个结点，链首尾相接，没有真正的头尾断口。
- 再想“带头结点”：链表最前面多一个不存数据的空结点，专门用来让空表和非空表的操作写成一样的；删结点时不用单独处理“删的是第一个数据结点”的情况。
- 再想“删除结点”：分两步——先把链绕过去（让 p 的前后直接相连），再释放 p 的空间；只绕过去不释放是浪费，只释放不绕过去是断链。

考查知识点：

- 先想正确的三句话：p 的前驱的 next 改成 p 的后继，写成 p->prev->next = p->next；p 的后继的 prev 改成 p 的前驱，写成 p->next->prev = p->prev；最后 free(p)，把 p 的空间还回去。
- 再想为什么两句都要：只改 next 一侧，往后走是对的，往前走还会走到 p；只改 prev 一侧，往前走是对的，往后走还会走到 p；双向链表两边都得接好。
- 再想为什么先接链后释放：如果先 free(p) 再改指针，p->prev 和 p->next 读到的就是已经还回去的空间，值可能已经变了，接出来的链就是错的。
- 拿生活例子验算：一排人手拉手，p 前后两个人要直接拉上手，再让 p 出列；如果先让 p 出列走了，前后两个人还拉着 p 原来的手，队伍就断了。
- 边界情况：带头结点的循环链表里，删最后一个数据结点后，头结点的 next 和 prev 都指回头结点自己，表变空，操作还是一样的三句话，不用特殊处理。

正确选项为什么对： D 的三句话正是“前驱的 next 接后继、后继的 prev 接前驱、释放 p”，两边都接好、最后才释放，选 D。

错误选项为什么错＋举例：

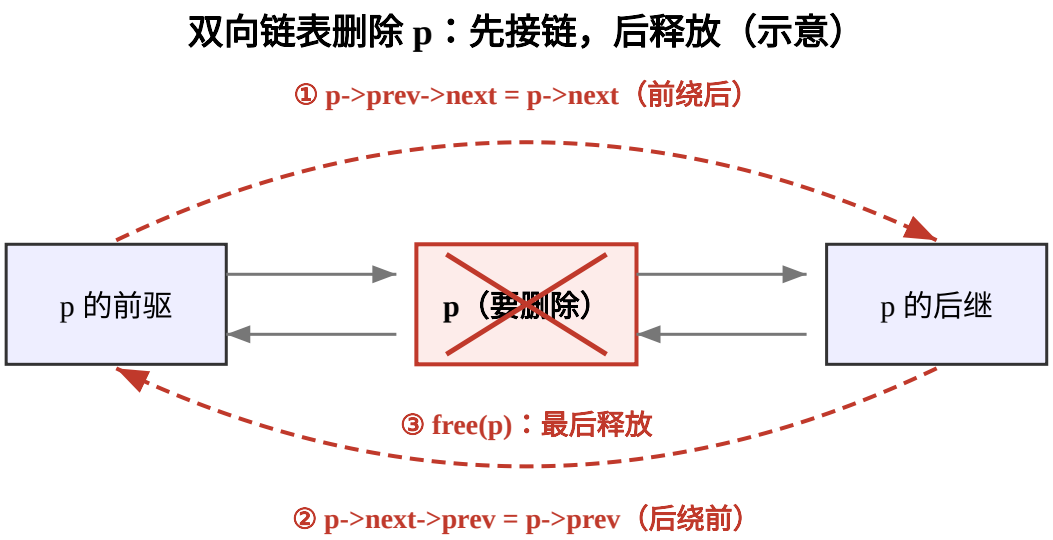
- A：只改了一侧的指针，另一侧没接好；往一个方向走能绕过 p，往另一个方向走还会走到 p，链是半断的。教训：双向链表删除，两侧指针都要改。
- B：把释放写在接链前面，或者接链时用了已经释放的结点；释放后的空间内容不可靠，接出来的链可能指到别处。教训：先接链，后释放。

- C：把 prev 和 next 的赋值对象写反了（比如让前驱的 prev 去接后继）；改错了指针，链的结构就乱了，顺着走会走到错误的结点。教训：next 改 next、prev 改 prev，别交叉。

注：本题错误选项的指针语句原文以原卷为准；此处按三种典型错误写法分析，标注为“部分核对”。

笔记本一句话： 第 2 题＋双向链表删结点：前驱的 next 接后继、后继的 prev 接前驱、最后释放，两边都要接。

【图像】



灰色＝删除前的旧链接；红色虚线＝新接好的链；两边都接好后才释放 p。

只改一侧，另一侧还是断的；先释放后接链，读到的是已归还的空间。

双向链表删除示意图：p 的前驱直接连 p 的后继，p 的后继直接连 p 的前驱，最后释放 p

【定位】 数据结构 · 第2章 线性表 · 双向链表的删除操作

【易混对比】

- "p->prev->next = p->next"和"p->next->prev = p->prev"是两回事：前一句修的是从前往后走的链（next 侧），后一句修的是从后往前走的链（prev 侧）——只改一句，另一侧还是断的。
- "先接链后释放"和"先释放后接链"是两回事：先接链时 p->prev、p->next 读到的还是活结点；先释放再接链，读到的就是已归还的空间，值可能已经变了。
- "带头结点"和"不带头结点"是两回事：带头结点时删任意结点都是同一套三句话；不带头结点时删第一个数据结点还要单独改头指针——本题带头结点，不用分情况。

正确的三句话是：p 的前驱的 next 接 p 的后继，p 的后继的 prev 接 p 的前驱，最后释放 p；选 D。

【坑位】

双向链表删除，两侧指针都要改，少改一侧链就是半断的。

最容易踩的坑是：只写了 p->prev->next = p->next 就以为完事了。为什么错：双向链表既能往前走也能往后走；只改 next 一侧，从后往前走时还会走到 p，等于链只接好了一半。正确做法：两句话都写全——前驱的 next 接后继、后继的 prev 接前驱，第三句再释放 p。

【套路】

- 第一步，前驱绕过 p：p->prev->next = p->next，让 p 的前一个结点直接指向 p 的后一个结点。
- 第二步，后继绕过 p：p->next->prev = p->prev，让 p 的后一个结点直接指回 p 的前一个结点。
- 第三步，最后释放 p：free(p)，把 p 占的空间还回去；释放必须放在接链之后。

双向删结点：前绕后、后绕前，最后释放。

【类比】

像一排人手拉手，p 前后两个人要先直接拉上手，再让 p 出列。如果先让 p 出列走了，前后两个人还拉着 p 原来的手，队伍就断了。术语贴回去：前驱的 next 和后继的 prev 就是"拉着的手"，先让前后直接相连（接链），再释放 p（出列）。

【口诀】 前后绕过再释放，两边都接才牢靠。

【考频】 「双向链表删除」在 2009–2022 年解析中出现 17 次 · 高频

第 3 题 · 平行轨道按先进先出编组列车使出队为 1～9 的最少轨道数（正确答案：C）

题干与选项：

设有下图所示的火车车轨，入口到出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为 1～9 的 9 列列车，驶入的次序依次是 8，4，2，5，3，9，1，6，7。若期望驶出的次序依次为 1～9，则 n 至少是（ ）。

- A. 2
- B. 3
- C. 4
- D. 5

> 注：原卷此处为火车车轨示意图（n 条轨道），插图以原卷为准。

入手点： 每条轨道是队列，先进先出；所以同一条轨道里的车，出站顺序必须和进站顺序一样，也就是递增的；问题就变成“把进站序列切成最少的递增子序列”，切成几段就要几条轨道。

专有名词从零解释：

- 先想“队列”：先进先出的数据结构；先排进去的先出来，像排队买票。
- 再想“递增子序列”：从原序列里挑几个数，保持原来的先后顺序，并且一个比一个大；比如 8、4、2、5、3、9、1、6、7 里，4、5、9 就是一个递增子序列。
- 再想“递减子序列”：挑出来的数一个比一个小；比如 8、4、3、1 就是一个递减子序列。
- 再想“最少轨道数和最长递减子序列的关系”：一条轨道只能装递增的一串车；如果有 4 辆车是递减着进站的，它们两两不能同轨（后进的反而要先出，队列做不到），所以至少要 4 条轨道；反过来，按贪心分轨，4 条就够。

考查知识点：

- 先想为什么每条轨道必须是递增的：轨道是队列，车从左进、从右出；同一条轨道里先后的车，出站也是先后的顺序；出站总顺序是 1～9 递增的，所以同一条轨道里的车号也必须是递增的。
- 再想最长递减子序列：进站序列 8、4、2、5、3、9、1、6、7 里，8、4、3、1 是递减的，长度是 4；再找更长的：8、5、3、1 也是 4；8、4、2、1 也是 4；没有 5 个数递减的子序列，所以最长就是 4。
- 再想为什么最少就是 4：这 4 辆递减的车（8、4、3、1）必须分在 4 条不同的轨道；所以少于 4 条肯定不够；按“每辆车进到能接它的轨道”的贪心分法，4 条轨道正好能把 9 辆车全部分完。
- 拿小例子验算：进站 3、1、2，出站要 1、2、3；最长递减子序列是 3、1（长度 2），所以要 2 条轨道：3 一条，1、2 一条；1 条轨道做不到，因为 3 先进、后出，会挡住 1。
- 边界情况：如果进站顺序本身就是 1～9 递增的，1 条轨道就够；如果进站是 9～1 全递减的，要 9 条轨道。

正确选项为什么对： 最长递减子序列长 4，最少需要 4 条轨道，就是 C，选 C。

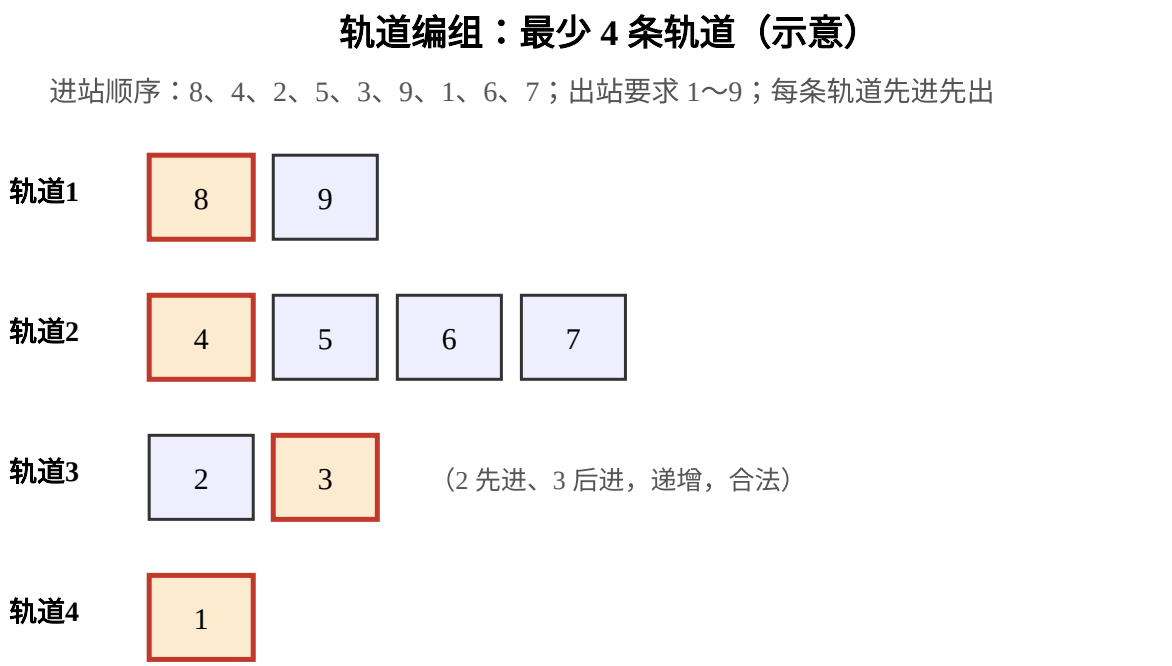
错误选项为什么错＋举例：

- A（2 条）：2 条轨道只能装 2 串递增的车；8、4、3、1 这 4 辆递减的车分不开，至少有一条轨道里出现“后进的反而先出”，队列做不到。教训：先找最长递减子序列，它就是下限。
- B（3 条）：3 条还是不够分 8、4、3、1 这 4 辆车；4 辆递减的车需要 4 条轨道，3 条里总有一条要装两辆递减的车。教训：下限是 4，3 比下限小，肯定不够。

- D（5 条）：5 条当然够用，但题目问“最少”；4 条已经够了，5 条不是最少。教训：问最少时，够用的数里取最小的。

笔记本一句话： 第 3 题＋轨道编组：每轨递增，最少轨道数＝最长递减子序列长；8、4、3、1 长 4。

【图像】



轨道编组示意图：8、4、3、1 四辆递减进站的车各占一条轨道，其余车按递增接在各轨道后面

【定位】 数据结构 · 第3章 栈、队列和数组 · 队列的应用

【易混对比】

- "最长递减子序列"和"最长递增子序列"是两回事：递减子序列里的车两两不能同轨（后进的反而先出，队列做不到），它定的是轨道数的下限；递增子序列定的是每条轨道里能装的样子——求最少轨道数，找的是递减的。
- "递减子序列"和"连续递减"是两回事：子序列不用连在一起，8、4、3、1 在进站序列里中间隔着别的车，照样算递减子序列。
- "最少 4 条"和"4 条就够"是两回事：前一句是下限（8、4、3、1 必须分轨，少于 4 条不行），后一句是上限（按贪心分轨，4 条真能把 9 辆车分完）——两句都成立，答案才是 4。

进站序列里最长递减子序列是 8、4、3、1，长度为 4；最少需要 4 条轨道，选 C。

【坑位】

先找最长递减子序列，它就是最少轨道数，别数递增的。

最容易踩的坑是：去找最长递增子序列，或者凭感觉数"几段"。为什么错：每条轨道是先进先出的队列，同一条轨道里的车出站顺序和进站顺序一致，出站要求 1～9 递增，所以每条轨道里的车号必须递增；反过来，递减进站的几辆车不可能同轨。正确做法：从进站序列里挑出最长的递减子序列，数它有几个数，轨道数就是几。

【套路】

- 第一步，确认每条轨道必须是递增的：队列先进先出，出站总顺序递增，所以同轨车号递增。
- 第二步，找最长递减子序列：在进站序列 8、4、2、5、3、9、1、6、7 里挑出 8、4、3、1，长度为 4。
- 第三步，轨道数等于递减子序列长度：递减的车必须分轨，最少就是 4 条。

轨道编组：每轨递增，最少轨道数等于最长递减子序列的长度。

【类比】

像超市收银台排队：每个收银台是先到先结账（先进先出）。如果 8、4、3、1 四个人是按"大额在前、小额在后"的顺序来的，要让他们按"小额先结"的顺序出去，这四个人绝不能排在同一个收银台——后来的反而先走，一个收银台做不到，只能分四个台。术语贴回去：收银台就是轨道，最长递减子序列就是"必须分台"的那拨人。

【口诀】 每轨递增找递减，最长几辆几条轨。

【考频】 「轨道编组最少轨道数」在 2009–2022 年解析中出现 3 次 · 中频

第 4 题 · 100 阶三对角矩阵行优先压缩下标从 0 开始时 m30,30 的下标（正确答案：B）

题干与选项：

有一个 100 阶的三对角矩阵 M，其元素 mi,j（1≤i≤100，1≤j≤100）按行优先依次压缩存入下标从 0 开始的一维数组 N 中。元素 m30,30 在 N 中的下标是（）。

- A. 86
- B. 87
- C. 88
- D. 89

入手点： 别背公式，先数“前面有几个元素”：第 30 行之前有多少行、每行有几个非 0 元素，加起来就是 m30,30 前面的元素个数；再看 m30,30 在本行是第几个，加起来就是下标。

专有名词从零解释：

- 先想“三对角矩阵”：只有三条斜线上有非 0 元素——主对角线（行号＝列号）、它上面一条（列号＝行号＋1）、它下面一条（列号＝行号－1）；其余位置全是 0。
- 再想“行优先压缩”：按行从上往下、每行从左往右，只把非 0 元素依次存进一维数组；0 不存，省地方。
- 再想“下标从 0 开始”：数组第一个元素的下标是 0，不是 1；数“前面有几个元素”，那个个数就是下标。

考查知识点：

- 先想每行有几个非 0 元素：第 1 行只有主对角线和上面一条，共 2 个；中间行（第 2 行到第 99 行）三条都有，共 3 个；第 100 行只有主对角线和下面一条，共 2 个。
- 再想第 30 行之前有多少个：第 1 行 2 个，第 2 行到第 29 行共 28 行、每行 3 个，就是 28×3＝84 个；加起来 2＋84＝86 个。
- 再想 m30,30 在本行是第几个：第 30 行的三个非 0 元素按列从小到大是 a30,29、a30,30、a30,31；m30,30 是本行第 2 个，下标就是 86＋1＝87。
- 拿小例子验算：5 阶三对角矩阵，第 3 行第 3 列的下标：前面第 1 行 2 个、第 2 行 3 个，共 5 个；本行第 2 个，下标是 6；按公式数一遍也一样。
- 边界情况：第 1 行第 1 列下标是 0；第 100 行第 100 列是最后一个，前面共 2＋98×3＝296 个，本行第 2 个，下标是 297；数组一共 298 个元素。

正确选项为什么对： 前面 86 个，本行第 2 个，下标是 87，就是 B，选 B。

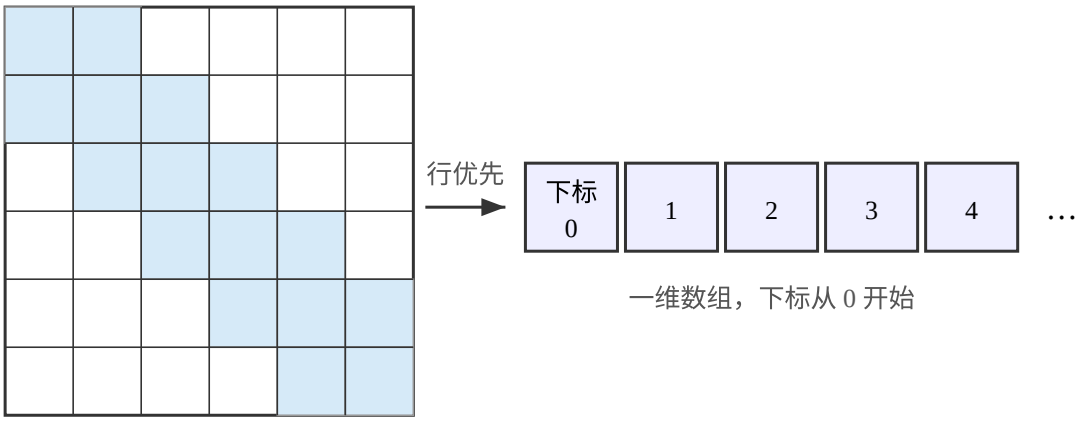
错误选项为什么错＋举例：

- A（86）：86 是 m30,30 前面的元素个数，也就是 a30,29 的下标；把“前面有几个”直接当答案，漏了本行这 1 个。教训：下标＝前面个数＋本行第几个－1。
- C（88）：88 是 a30,31 的下标；把 m30,30 当成本行第 3 个了。教训：先把本行的三个元素按列排好，再数第几个。
- D（89）：89 是第 31 行第 1 个元素（a31,30）的下标；多算了一整行。教训：行数别数错，第 30 行之前是 29 行，不是 30 行。

笔记本一句话： 第 4 题＋三对角压缩：首尾行 2 个、中间行 3 个；下标＝前面个数＋本行第几个－1。

【图像】

三对角矩阵行优先压缩（示意：6 阶画法，100 阶同理）



三条斜线非零，其余为 0

数"前面有几个"：第 1 行 2 个，第 2～29 行每行 3 个，共 2＋28×3＝86 个。

m30,30 在本行按列是第 2 个（a30,29、a30,30、a30,31），下标＝86＋2－1＝87。

别把"前面有几个"（86）直接当答案；也别多算一行。

三对角矩阵压缩示意图：主对角线及上下两条斜线共三条非零，按行优先依次存入下标从 0 开始的一维数组

【定位】 数据结构 · 第3章 栈、队列和数组 · 特殊矩阵的压缩存储

【易混对比】

- "第 30 行之前有 29 行"和"第 30 行之前有 30 行"是两回事：算"前面有多少个元素"时，第 30 行自己不算在内，只数第 1 到第 29 行——多算一行，答案就多 3。
- "下标从 0 开始"和"下标从 1 开始"是两回事：从 0 开始时，下标等于"前面有几个元素"再加上"本行第几个减 1"；从 1 开始才直接加本行第几个——本题从 0 开始，别多加 1。
- "三对角"和"全矩阵"是两回事：三对角每行最多 3 个非零元素，存的是压缩后的一维数组；全矩阵才按行列直接算位置——别套二维数组的寻址公式。

第 30 行之前共 2＋28×3＝86 个元素，m30,30 是本行第 2 个，下标为 86＋2－1＝87，选 B。

【坑位】

下标等于"前面有几个"加"本行第几个"再减 1，别把"前面有几个"直接当答案。

最容易踩的坑是：数出前面有 86 个元素，直接选 86。为什么错：86 是 m30,30 前面的元素个数，也就是 a30,29 的下标；m30,30 还要再往后数 1 个。正确做法：先数前面有多少个（86），再看它是本行第几个（第 2 个），下标＝86＋2－1＝87。

【套路】

- 第一步，数每行有几个非零元素：首行 2 个、中间行 3 个、末行 2 个。
- 第二步，数目标行前面有多少个：第 1 行 2 个，第 2 到第 29 行共 28 行、每行 3 个，合计 86 个。
- 第三步，数目标元素是本行第几个：本行三个元素按列是 a30,29、a30,30、a30,31，m30,30 是第 2 个。
- 第四步，算下标：下标＝前面个数＋本行第几个－1＝86＋2－1＝87。

压缩下标：首尾两条中间三，前面几个加本行第几个再减一。

【类比】

像电影院按排存票根：每排的票根按座位顺序收进一个大信封，前一排收完接着收下一排。要找第 30 排第 2 个座位的票根，先数前面 29 排一共多少张票根（86 张），再往后数 2 张——票根在信封里的位置（从 0 开始数）就是 86＋2－1。术语贴回去：行优先压缩就是"按排收票根"，下标从 0 开始就是"位置从 0 数"。

【口诀】 首尾两条中间三，前面个数加本行，减一才是下标号。

【考频】 「三对角矩阵压缩存储」在 2009–2022 年解析中出现 2 次 · 低频

第 5 题 · 森林边数 15 结点数 25 推树的棵数（正确答案：C）

题干与选项：

若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是（ ）。

- A. 8
- B. 9
- C. 10
- D. 11

入手点： 一棵树里，边数总比结点数少 1；森林就是好几棵树拼在一起；每棵树都少 1，少出来的总数就是树的棵数。

专有名词从零解释：

- 先想“树”：连通的、无环的图；结点之间都有路可走，而且没有圈；一棵有 n 个结点的树，恰好有 n－1 条边。
- 再想“森林”：好几棵树放在一起，树和树之间没有边相连；森林的结点数就是各棵树结点数之和，边数就是各棵树边数之和。
- 再想“边数和结点数的关系”：一棵树：边＝结点－1；k 棵树组成的森林：边＝结点－k；所以 k＝结点－边。

考查知识点：

- 先想公式怎么来：设森林有 k 棵树，第 i 棵树有 n_i 个结点、n_i－1 条边；总结点数是 n_1＋…＋n_k＝25，总边数是（n_1－1）＋…＋（n_k－1）＝25－k；题目说总边数是 15，所以 25－k＝15，k＝10。
- 再想为什么不是别的数：如果只有 1 棵树，25 个结点该有 24 条边，比 15 多 9 条；每多一棵树，边数就少 1 条；少了 9 条，就是多了 9 棵树？不对——1 棵树是 24 条边，15 条边比 24 少 9 条，所以是 1＋9＝10 棵树；两种算法都得到 10。
- 拿小例子验算：3 个结点、1 条边：边比结点少 2，就是 2 棵树（一条边连两个结点成一棵，剩下一个孤立结点也是一棵）；公式 3－1＝2，对得上。
- 边界情况：边数是 0 时，棵数＝结点数（每个结点都是一棵只有一个结点的树）；边数是结点数－1 时，棵数是 1（整片连成一棵树）。

正确选项为什么对： 棵数＝25－15＝10，就是 C，选 C。

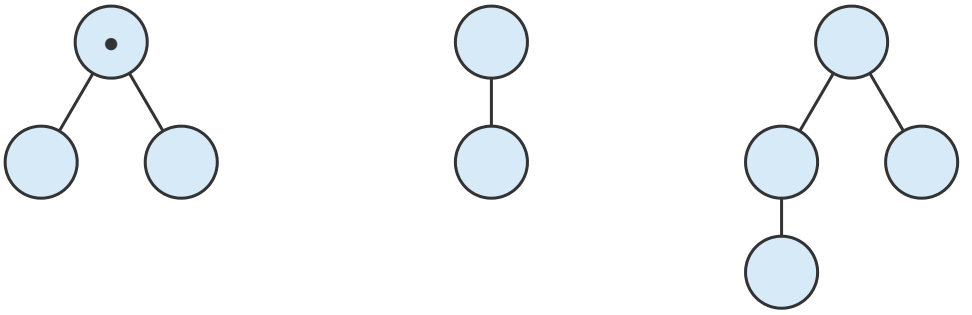
错误选项为什么错＋举例：

- A（8）：8 是把边数和结点数用错了关系算出来的；比如 25－15＝10，不是 8；8 对应的是 17 条边的情况。教训：公式是结点减边，不是别的组合。
- B（9）：9 是少算了一棵；1 棵树时边是 24，15 比 24 少 9，但棵数是 1＋9＝10，不是 9。教训：少 9 条边，棵数是在 1 的基础上加 9。
- D（11）：11 是多算了一棵；25－15＝10，不是 11。教训：直接套公式算，别凭感觉加减。

笔记本一句话： 第 5 题＋森林棵数：棵数＝结点数－边数；25－15＝10。

【图像】

森林：棵数＝结点数－边数（示意）



树1：3 结点 2 条边　　树2：2 结点 1 条边　　树3：4 结点 3 条边

……（共 10 棵，树与树之间没有边相连）

每棵树：边＝结点－1 → 总边数＝总结点－棵数
棵数＝25－15＝10

森林示意图：多棵树拼成的森林，边数等于总结点数减去树的棵数，25 个结点 15 条边得 10 棵树

【定位】 数据结构 · 第5章 树与二叉树 · 树与森林的基本性质

【易混对比】

- "一棵树的边数"和"森林的边数"是两回事：一棵树里边＝结点－1；森林里边＝总结点－树的棵数——多一棵树，就少一条边。
- "棵数＝结点－边"和"棵数＝边－结点"是两回事：每棵树都让边比结点少 1，k 棵树一共少 k 条边，所以 k＝25－15＝10；写反了会得到负数。
- "森林"和"连通图"是两回事：森林是好几棵互不相连的树；25 个结点连成一棵树需要 24 条边，题目只给了 15 条，少了 9 条，正好说明它是 1＋9＝10 棵树拼起来的。

森林的棵数＝结点数－边数＝25－15＝10，选 C。

【坑位】

公式是"结点减边"，别写成"边减结点"，更别凭感觉加减。

最容易踩的坑是：记住"树里边比结点少 1"，到森林就直接写 15－1 或 25－1。为什么错：一棵树才有"边＝结点－1"；森林是 k 棵树，每棵都少 1，合起来是"总边数＝总结点数－k"。正确做法：把公式倒过来用——k＝结点数－边数＝25－15＝10。

【套路】

- 第一步，写出一棵树的关系：边数＝结点数－1。
- 第二步，推广到 k 棵树：总边数＝总结点数－k。
- 第三步，倒出棵数公式：k＝总结点数－总边数＝25－15＝10。

森林棵数：总结点数减去总边数。

【类比】

像用绳子串珠子穿手链：一串手链有 n 颗珠子，要用 n－1 段绳子连起来。现在有 25 颗珠子、只用了 15 段绳子，说明珠子被分成了好几串——每多分出一串，就少用 1 段绳子；比"穿成一串"少用了 24－15＝9 段绳子，就是分成了 1＋9＝10 串。术语贴回去：珠子是结点，绳子是边，手链是树，几串手链就是森林的棵数。

【口诀】 树里边比点少一，森林棵数点减边。

【考频】 「森林结点边数棵数」在 2009–2022 年解析中出现 16 次 · 高频

第 6 题 · 判断哪个顶点序列不可能是给定有向图的 DFS 序列（正确答案：D）

题干与选项：

下列选项中，不是下图深度优先搜索序列的是（）。

- A. V1，V5，V4，V3，V2
- B. V1，V3，V2，V5，V4
- C. V1，V2，V5，V4，V3
- D. V1，V2，V3，V4，V5

入手点： DFS 的规矩是“一钻到底”：走到一个顶点，先顺着它的出边往深处走，把能走的都走完，才退回来换另一条路；所以一个序列要想是 DFS 序列，必须处处符合“不提前退回来”的规矩；找出违反规矩的那个。

专有名词从零解释：

- 先想“有向图”：顶点之间用带箭头的边相连；边只能顺着箭头走，不能逆行。
- 再想“深度优先搜索（DFS）”：从一个顶点出发，先走到它的一个邻居，再从这个邻居继续往深处走，走到没路了才退回上一个顶点换条路；像走迷宫时先沿一条路走到底。
- 再想“DFS 序列”：DFS 走过顶点的先后顺序；每个顶点在第一次被走到时记下来，排成的就是序列。
- 再想“为什么有的序列不可能是 DFS 序列”：如果序列里某一步“退回来”了，但退回来的那个顶点明明还有没走过的邻居，DFS 就不会退回来；这种序列就不可能是 DFS 走出来的。

考查知识点：

- 先想图的大致样子：V1 在最上面，V2 在左中，V3 在右中，V4 在左下，V5 在右下；从 V1 出发，可以先往左到 V2、也可以先往右到 V3。
- 再想四个序列：A 是 V1、V5、V4、V3、V2；B 是 V1、V3、V2、V5、V4；C 是 V1、V2、V5、V4、V3；D 是 V1、V2、V3、V4、V5。
- 再想 D 为什么不可能：D 里 V2 后面直接跟 V3，意思是走到 V2 后马上退回 V1、再走到 V3；但 V2 下面还有没走过的邻居 V4（左下的顶点和 V2 相连）；DFS 走到 V2 后必须先往深处走 V4，不会跳过 V4 直接退回 V1 再去 V3；所以 D 违反了“一钻到底”的规矩。
- 再想 A、B、C 为什么可能：按图里的边走，A、B、C 的每一步都符合“走到某顶点后先往深处走、走到底才退回”的规矩；它们的退回都发生在当前顶点没有没走过的邻居之后，所以都可能是 DFS 走出来的顺序。
- 边界情况：如果 V2 和 V4 之间没有边，D 就可能是 DFS 序列；本题 D 不可能，正是因为 V2 下面有 V4 这条路；判断时一定要看图里的边，不能只看序列本身。
- 易混对子：DFS 序列不是“按编号从小到大”；V1、V2、V3、V4、V5 看着整齐，但整齐不等于符合 DFS 规矩；广度优先搜索（BFS）才是“一层一层”地走，序列长得像按层排。

正确选项为什么对： D 在 V2 还有邻居 V4 没走的情况下就退回 V1 去走 V3，违反了 DFS 一钻到底的规矩，所以 D 不可能是 DFS 序列，选 D。

错误选项为什么错＋举例：

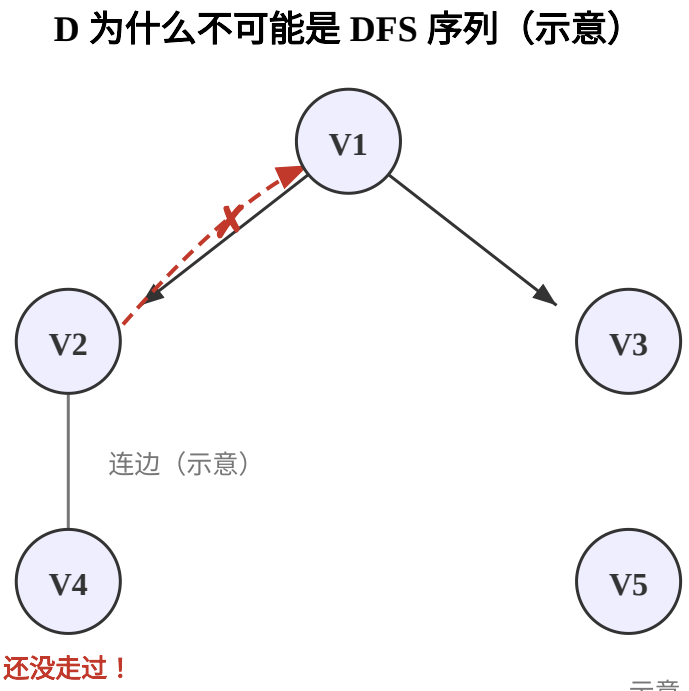
- A（V1、V5、V4、V3、V2）：按图里的边走，它的每一步都符合深走到底再退回的规矩，可能是 DFS 序列；不能选它，因为题目问的是“不可能是”。教训：问“不可能是”，先把可能的排除掉。
- B（V1、V3、V2、V5、V4）：同样每一步都符合深走的规矩，可能是 DFS 序列。教训：序列不按编号顺序，不代表它不是 DFS 序列。

- C（V1、V2、V5、V4、V3）：同样符合深走到底再退回的规矩，可能是 DFS 序列。教训：DFS 的退回只发生在“当前顶点没路可走”的时候，C 的每一步退回都满足这个条件。

> 注：本题图中部分边向在扫描中较模糊，D 不可能的判断依据是 V2 与 V4 之间的连边；图中细节以原卷为准，标注为“部分核对”。

笔记本一句话： 第 6 题＋DFS 序列判断：走到某顶点后必须先钻到底，有邻居没走就不能提前退回。

【图像】



D＝V1、V2、V3、V4、V5：在 V2 还有邻居 V4 没走时就退回 V1 去走 V3，违反"一钻到底"。

DFS 序列判断示意图：从 V1 走到 V2 后，V2 下面还有没走过的邻居 V4，不能提前退回 V1 去走 V3（示意，边向以原卷为准）

【定位】 数据结构 · 第6章 图 · 图的深度优先遍历

【易混对比】

- "DFS"和"BFS"是两回事：DFS 是一条路钻到底、没路了才退回；BFS 是一层一层往外扩——V1、V2、V3、V4、V5 看着像按层排，那是 BFS 的长相，不是 DFS 的。
- "退回"和"提前退回"是两回事：当前顶点的邻居都走完了再退回，是合法的；还有邻居没走就退回，是违规的——判断只看"退回那一刻还有没有没走过的邻居"。
- "序列看着整齐"和"序列合法"是两回事：V1、V2、V3、V4、V5 按编号排得很整齐，但 DFS 不按编号走，只按"有路就钻"的规矩走——整齐不能当证据。

D 在 V2 还有邻居 V4 没走的情况下就退回 V1 去走 V3，违反了一钻到底的规矩，所以 D 不可能是 DFS 序列，选 D。

【坑位】

别被"按编号排整齐"骗了，DFS 只认"有路就钻"的规矩。

最容易踩的坑是：看到 D 是 V1、V2、V3、V4、V5，按编号来，觉得"这么顺肯定是对的"，转而去怀疑别的选项。为什么错：DFS 的访问顺序取决于图的边和"钻到底"的规矩，跟顶点编号没关系。正确做法：拿着序列一步

一步在图上走，每遇到一次"退回"，就检查退回的那个顶点还有没有没走过的邻居——有，就是冒牌序列。

【套路】

- 第一步，把序列的顶点按顺序在图上走一遍：顺着有向边走，确认每一步都有边可走。
- 第二步，盯住每一次"退回"：序列里从深处回到浅处顶点的那一步，就是一次退回。
- 第三步，检查退回时还有没有没走过的邻居：有没走过的邻居还退回，这个序列就不可能是 DFS 序列。

验 DFS 序列：退回那一刻，邻居必须走完。

【类比】

像走迷宫探险：规矩是沿一条路走到死胡同才往回走。如果你在岔路口 V2，明明左边还有条没走过的岔路通向 V4，你却掉头回入口 V1、去走右边的路 V3——这就不符合"走到底才回头"的探险规矩。术语贴回去：岔路是邻接顶点，"走到死胡同才回头"就是深度优先。

【口诀】 钻到没路才回头，有路不走是冒牌。

【考频】 「DFS 序列判断」在 2009–2022 年解析中出现 5 次 · 中频

第 7 题 · 有向图邻接表存储时拓扑排序的时间复杂度（正确答案：B）

题干与选项：

若将 n 个顶点 e 条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是（ ）。

- A. O(n)
- B. O(n+e)
- C. O(n^2)
- D. O(ne)

入手点： 拓扑排序要做两件事——每个顶点处理一次，每条弧处理一次；两件事的时间加起来，就是总时间。

专有名词从零解释：

- 先想“拓扑排序”：把有向图的顶点排成一列，使得每条弧都是从前面的顶点指向后面的顶点；只有无环图才能排出来；像排课程顺序，先修课排在后修课前面。
- 再想“入度”：指向某个顶点的弧有几条；入度是 0 的顶点就是“没人挡着它”，可以先排。
- 再想“邻接表”：每个顶点后面挂一条链，链上是它指向的所有邻居；存图只存实际存在的弧，不存不存在的，省地方。
- 再想“时间复杂度 O(n＋e)”：时间随着顶点数 n 和弧数 e 线性长；n 和 e 各自只出现一次，没有相乘。

考查知识点：

- 先想拓扑排序的步骤：先算每个顶点的入度；把入度是 0 的顶点放进队列；每次从队列取出一个顶点排进结果，顺着它的邻接表把每个邻居的入度减 1，减到 0 的邻居再进队列；直到队列空了。
- 再想顶点花多少时间：每个顶点进一次队列、出一次队列，共 n 个顶点，就是 O(n) 的时间。
- 再想弧花多少时间：每个顶点出队列时，要顺着它的邻接表走一遍；所有顶点的邻接表加起来正好是 e 条弧，每条弧只走一次，就是 O(e) 的时间。
- 再想为什么不是 O(n×e)：如果用邻接矩阵存图，找一个顶点的邻居要扫一整行（n 个位置），总时间就是 O(n 的平方)；邻接表只存实际存在的弧，扫邻接表就是扫弧，不用扫空位置，所以是 O(n＋e)。
- 边界情况：e 可以比 n 大得多（稠密图），也可以比 n 小（稀疏图）；O(n＋e) 里两项都留着，因为不知道哪个大；如果图是空图（e＝0），时间就是 O(n)。

正确选项为什么对： 顶点处理 O(n)，弧处理 O(e)，加起来是 O(n＋e)，就是 B，选 B。

错误选项为什么错＋举例：

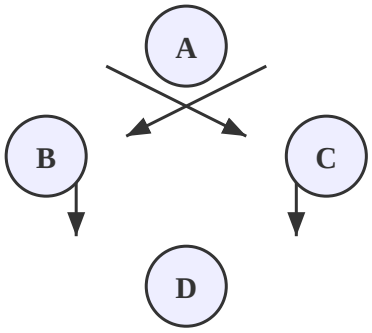
- A（O(n)）：只算了顶点，没算弧；顺着邻接表减入度时，每条弧都要处理一次，漏了这部分。教训：拓扑排序既要过顶点，也要过弧。
- C（O(n×e) 或类似的相乘量级）：那是每次找邻居都要全图扫一遍的算法；邻接表不用全图扫，只扫实际存在的弧。教训：邻接表的优势就是“只扫有的，不扫没有的”。
- D（O(n 的平方)）：那是邻接矩阵的量级；邻接矩阵里每个顶点要扫一行 n 个位置，n 个顶点就是 n 的平方；本题用的是邻接表，不是矩阵。教训：先看存储结构，再定量级。

笔记本一句话： 第 7 题＋拓扑排序邻接表：顶点 O(n)＋弧 O(e)＝O(n＋e)；矩阵才是 O(n 的平方)。

【图像】

拓扑排序（邻接表）：顶点过一遍＋弧过一遍（示意）

图（邻接表只存 e 条弧）



邻接表：

A	B	C
B	D	
C	D	
D	∅	

只存实际存在的弧，
扫邻接表＝扫弧，
不用扫空位置

① 每个顶点进一次队列、出一次队列 → **O(n)**

② 顶点出队列时顺邻接表走一遍，每条弧只走一次 → **O(e)**

O(n)＋O(e)＝O(n＋e)　　（邻接矩阵要扫整行，才是 $O(n^2)$）

拓扑排序复杂度示意图：n 个顶点各进出队列一次，e 条弧各被顺邻接表走一次，总时间随 n 加 e 线性增长

【定位】 数据结构 · 第6章 图 · 拓扑排序

【易混对比】

- "O(n＋e)"和"O(n×e)"是两回事：前者是顶点和弧各过一遍再相加；后者是每个顶点都要把全图的弧扫一遍——邻接表只扫实际存在的弧，不用全图扫。
- "邻接表"和"邻接矩阵"是两回事：邻接表存 e 条弧，扫邻居就是扫弧；邻接矩阵存 n×n 个位置，找一个顶点的邻居要扫一整行——同样的拓扑排序，矩阵是 O(n 的平方)，表是 O(n＋e)。
- "顺着弧减入度"和"每轮重算入度"是两回事：算法里每个顶点的入度只在初始化时算一次，之后顺着弧减 1；如果每轮都重算入度，时间就不是 O(n＋e) 了。

每个顶点进出队列一次是 O(n)，每条弧顺邻接表走一次是 O(e)，合起来是 O(n＋e)，选 B。

【坑位】

先看存储结构再定量级：邻接表是 n 加 e，邻接矩阵才是 n 的平方。

最容易踩的坑是：背"拓扑排序是 O(n 的平方）"，上来就选。为什么错：O(n 的平方）是邻接矩阵的量级——矩阵里找一个顶点的所有邻居要扫一整行 n 个位置；邻接表只存实际存在的弧，扫邻接表就是扫弧。正确做法：先看题目给的存储结构，邻接表就按"顶点过一遍＋弧过一遍"算。

【套路】

- 第一步，数顶点的工作量：每个顶点进一次队列、出一次队列，共 n 个顶点，是 O(n)。
- 第二步，数弧的工作量：顶点出队列时顺它的邻接表走一遍，所有邻接表加起来正好 e 条弧，每条只走一次，是 O(e)。
- 第三步，两项相加：O(n)＋O(e)＝O(n＋e)。

邻接表上图算法的复杂度：顶点过一遍加弧过一遍。

【类比】

像点名加收作业：班里 n 个学生挨个点名（每个点一次），再收 e 份作业（每份收一次）——总工作量是"点名 n 次＋收作业 e 次"，不是"点 n 次名、每次都把 e 份作业数一遍"。术语贴回去：点名是处理顶点，收作业是顺邻接表处理弧。

【口诀】 顶点过一遍，弧也过一遍，相加就是 n 加 e。

【考频】 「拓扑排序时间复杂度」在 2009–2022 年解析中出现 16 次 · 高频

第 8 题 · Dijkstra 从顶点 1 出发各顶点最短路径被依次确定的顺序（正确答案：B）

题干与选项：

使用迪杰斯特拉（Dijkstra）算法求下图中从顶点 1 到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（）。

- A. 5，2，3，4，6
- B. 5，2，3，6，4
- C. 5，2，4，3，6
- D. 5，2，6，3，4

入手点： Dijkstra 的规矩是“每次挑还没确定的顶点里、到起点距离最短的那个，把它定下来”；一步一步挑，挑的顺序就是答案；关键是每次挑完后，要用新定的顶点去更新其他顶点的距离。

专有名词从零解释：

- 先想“带权图”：每条边上标着一个数字（权），表示走这条边的代价；比如路程、时间、费用。
- 再想“最短路径”：从起点到某个顶点，所有走法里代价最小的那条路的代价。
- 再想“Dijkstra 算法”：从起点开始，一轮一轮地确定顶点：每轮在还没确定的顶点里挑一个当前距离最小的，把它定下来（它的距离以后不会再变小了），再用它去更新邻居的距离；直到所有顶点都定下来。
- 再想“为什么挑最小的就能定下来”：所有边的权都是正数，已经确定的顶点的距离只会越挑越大；当前最小的那个，不可能再通过别的没确定的顶点找到更短的路（绕路只会更长），所以可以放心定下来。

考查知识点：

- 先想起点的邻居：从 1 出发，1→5 是 4，1→2 是 5；还没确定的顶点里最小的是 5（距离 4），先定 5。
- 再想用 5 更新：5→2 是 6，1→5→2 共 10，比直接 1→2 的 5 大，不更新；5→6 是 5，1→5→6 共 9，6 的距离记成 9；5 的邻居里还有到 3 的边（3→5 是反方向，不算），所以 3 的距离暂时还是没定。
- 再想第二轮：还没确定的顶点是 2（5）、3（没直接边，暂记无穷）、6（9）、4（没边）；最小的是 2（5），定 2。
- 再想用 2 更新：2→3 是 2，1→2→3 共 7，3 的距离记成 7；2→5 是 9，1→2→5 共 14，比 5 已定的 4 大，不用管。
- 再想第三轮：还没确定的是 3（7）、6（9）、4；最小的是 3（7），定 3；再想第四轮：剩下 6（9）和 4，6 的距离 9 比 4 小（4 要绕更远的路），定 6；最后定 4。
- 再想顺序：5、2、3、6、4，就是 B。
- 拿小例子验算：起点到 A 是 1、到 B 是 10、A 到 B 是 1；先定 A（1），再用 A 更新 B 为 2，定 B；顺序是 A、B，不是按编号，也不是按直接边的大小（B 的直接边 10 比 A→B 的 2 大，但 B 还是后定的）。
- 边界情况：如果有负权边，Dijkstra 的“挑最小就定下来”就不成立了；本题所有权都是正数，可以放心用。

正确选项为什么对： 一轮一轮挑最小：5（4）、2（5）、3（7）、6（9）、4（最后），顺序是 5、2、3、6、4，就是 B，选 B。

错误选项为什么错＋举例：

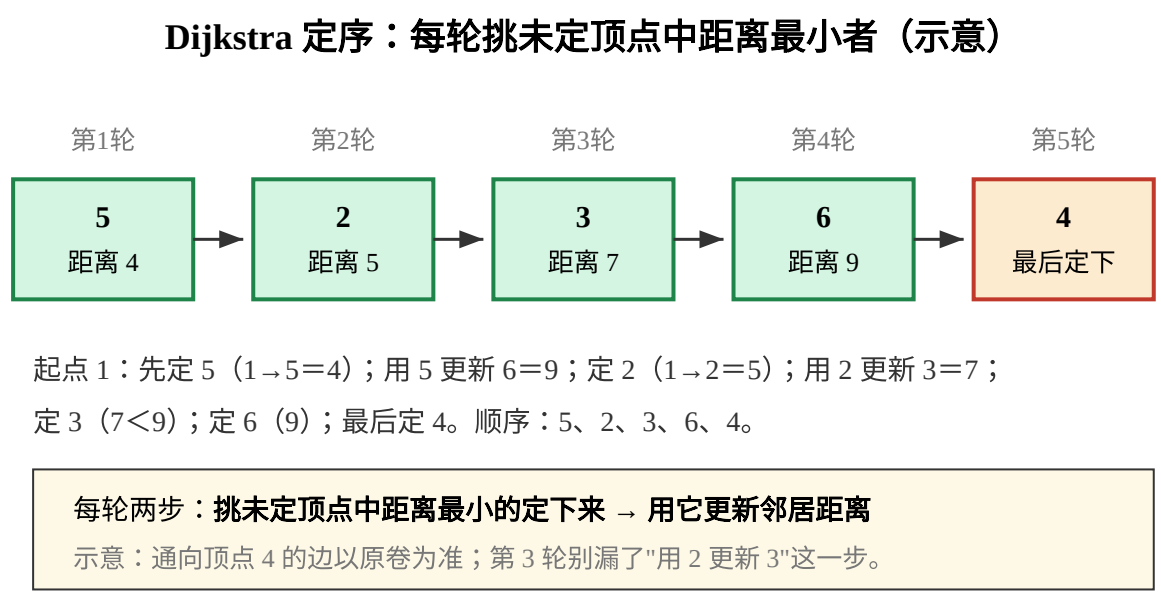
- A（5、2、3、4、6）：把 4 排在 6 前面；4 的最短距离比 6 的 9 还大，不可能在 6 之前被定下来。教训：定下来的顺序按距离从小到大，不是按编号。

- C（5、2、4、3、6）：把 4 排在第 3 位；第 3 轮时 3 的距离已经是 7，比 4 小，应该先定 3。教训：每轮挑之前，先把距离更新完再比。
- D（5、2、6、3、4）：把 6 排在 3 前面；第 3 轮时 3 的距离是 7，比 6 的 9 小，应该先定 3。教训：2→3 这条边把 3 的距离拉到了 7，比 6 小，别漏了更新这一步。

> 注：本题通向顶点 4 的边在扫描中未完全看清；4 最后被确定的结论按给定数据与答案表为准，图中细节以原卷为准，标注为“部分核对”。

笔记本一句话： 第 8 题＋Dijkstra 定序：每轮挑未定顶点里距离最小的，定完再更新邻居；5、2、3、6、4。

【图像】



Dijkstra 定序示意图：按 5、2、3、6、4 的顺序，每轮挑未定顶点中距离最小的定下来，定完再更新邻居（示意，通向顶点 4 的边以原卷为准）

【定位】 数据结构 · 第6章 图 · 最短路径 Dijkstra 算法

【易混对比】

- "定下来的顺序"和"顶点的编号顺序"是两回事：Dijkstra 按距离从小到大定顶点，本题是 5、2、3、6、4，不是 2、3、4、5、6——编号小不代表距离小。
- "直接边的长度"和"当前最短距离"是两回事：1→2 的直接边是 5，2 定下来时的距离就是 5；而 6 没有从 1 出发的直接边，它的距离 9 是经 5 绕过来的——比较时用"当前最短距离"，不是"直接边"。
- "Dijkstra"和"随便挑一个定"是两回事：每轮只能挑"未定顶点里距离最小"的定；挑了个不是最小的，后面可能还有更短的路，定下来的结论就不可靠了。

一轮一轮挑最小：5（距离 4）、2（距离 5）、3（距离 7）、6（距离 9）、4（最后），顺序是 5、2、3、6、4，选 B。

【坑位】

每轮挑之前，先把距离更新完再比，漏了更新就会挑错。

最容易踩的坑是：第 3 轮直接在 6（距离 9）和 4 之间挑，把 6 定下来。为什么错：定完 2 之后要用 2→3 这条边更新 3 的距离——1→2→3 一共 7，比 6 的 9 小，第 3 轮该定的是 3。正确做法：每轮分两步——先用新定的顶点

更新所有邻居的距离，再在未定顶点里挑最小的。

【套路】

- 第一步，列出起点的直接距离：1→5 是 4，1→2 是 5，其余暂时记无穷。
- 第二步，挑未定顶点中距离最小的定下来：先定 5（距离 4）。
- 第三步，用新定的顶点更新邻居：经 5 到 6 是 9，6 的距离记成 9；经 5 到 2 是 10，比 5 大，不更新。
- 第四步，重复第二、三步：定 2（距离 5），更新 3 为 7；定 3（距离 7）；定 6（距离 9）；最后定 4。

Dijkstra 定序：挑小定下再更新，轮轮如此到终点。

【类比】

像发"已确认"通知：起点是总部，每轮在还没收到确认的城市里，挑一个"当前已知最快能到达"的，把它的路线定死（以后不会再变快了），再用它去更新别的城市的"最快到达时间"。术语贴回去："定下来"就是最短路径已确认，"更新邻居"就是看看经这个城市能不能更快到达别处。

【口诀】 挑小定下再更新，轮轮如此到终点。

【考频】 「Dijkstra 最短路径」在 2009–2022 年解析中出现 5 次 · 中频

第 9 题 · 步长 3 跳跃查找与折半查找相比可能比较次数更少的情形（正确答案：B）

题干与选项：

在有 n（n＞1000）个元素的升序数组 A 中查找关键字 x。查找算法的伪代码如下所示。

```
k=0;
while(k<n 且 A[k]<x) k=k+3;
if(k<n 且 A[k]==x) 查找成功;
else if(k-1<n 且 A[k-1]==x) 查找成功;
else if(k-2<n 且 A[k-2]==x) 查找成功;
else 查找失败;
```

本算法与折半查找算法相比，有可能具有更少比较次数的情形是（）。

- A. 当 x 不在数组中
- B. 当 x 接近数组开头处
- C. 当 x 接近数组结尾处
- D. 当 x 位于数组中间位置

入手点： 先把两种方法的“找法”对比一下：折半查找每次都从中间砍，不管 x 在哪，都要先看中间；跳跃查找从头一块一块往后跳，x 越靠前，跳的块越少；所以 x 靠前时，跳跃查找可能赢。

专有名词从零解释：

- 先想“升序数组”：元素从小到大排好的数组；找东西可以利用顺序，不用一个一个全看。
- 再想“折半查找”：每次看中间的元素，比 x 大就往左半边找，比 x 小就往右半边找，每次把范围砍掉一半；找 n 个元素的数组，最多约 log n 次比较。
- 再想“跳跃式查找”：把数组切成一块一块的（本题步长是 3，就是每块 3 个元素）；先一块一块往后跳，找到 x 可能在的那块，再在块里一个一个比；块跳得少，比较就少。
- 再想“比较次数”：找的过程中，把数组元素和 x 比大小的次数；次数越少，找得越快。

考查知识点：

- 先想跳跃查找的步骤：步长是 3，先看第 3 个、第 6 个、第 9 个……（块的最后一个元素），找到第一个不小于 x 的块，再在块里从头比到 x；x 在第 1 块时，只要看 1 个块边界、再在块里比 1～3 次，总共很少。
- 再想折半查找的步骤：不管 x 在哪，第一次都看中间的元素；数组有 1000 个元素时，第一次看第 500 个，第二次看第 250 个……；x 在开头附近时，也要砍好几次才能定位到开头。
- 再想 x 靠近开头时：跳跃查找第 1 块就停下了，比较次数是个位数；折半查找要从中间一路砍到开头，比较次数是 log n 量级；所以跳跃查找有可能更少。
- 拿小例子验算：数组 1、2、3、……、1000，找 x＝2；跳跃查找：看第 3 个（3）就知道 x 在第 1 块，再比第 1 个（1）、第 2 个（2），共 3 次；折半查找：看 500、250、125、62、31、15、7、3、1、2，共 10 次；跳跃查找赢了。
- 边界情况：x 在数组最后时，跳跃查找要跳完所有的块，比较次数比折半多；x 不在数组中时，跳跃查找也要跳到最后才能确定，也比折半多。

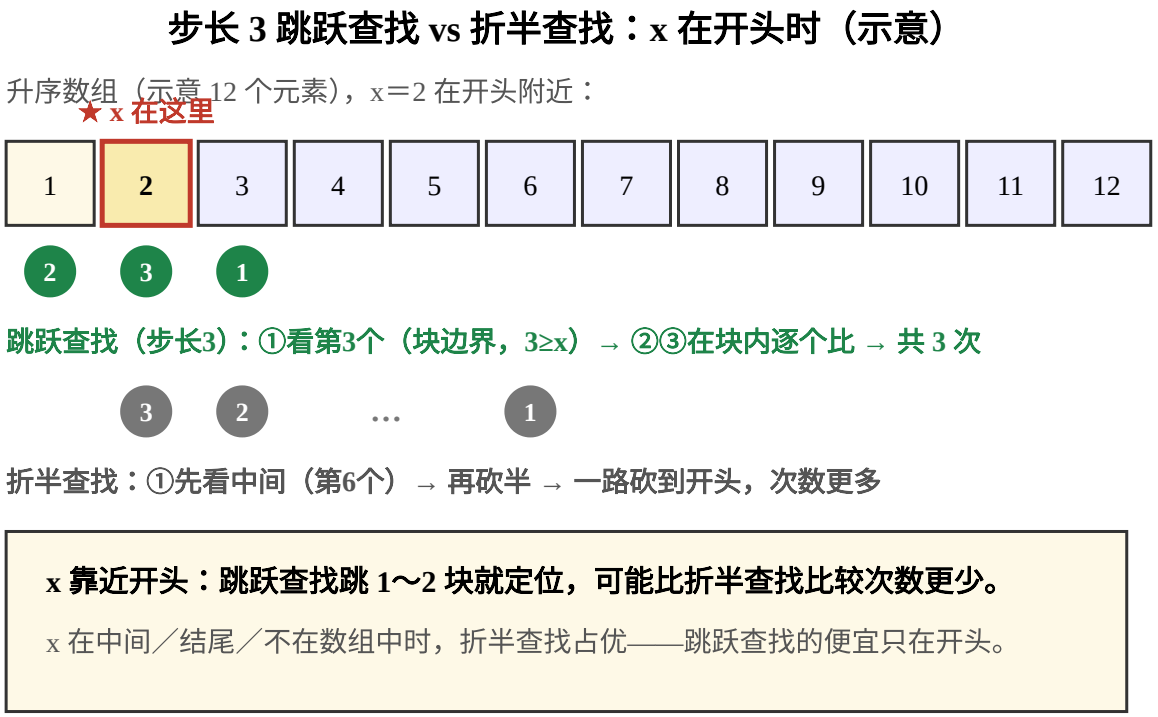
正确选项为什么对： x 接近数组开头时，跳跃查找跳 1～2 块就定位了，比较次数可能比折半查找少，就是 B，选 B。

错误选项为什么错＋举例：

- A（x 不在数组中）：x 不在时，跳跃查找要跳完所有块、再把最后一块比完，才能确定没有；折半查找砍一半、再砍一半，很快就能确定没有；跳跃查找不会更少。教训：不在数组中时，跳跃查找没有“提前停”的便宜可占。
- C（x 接近数组结尾）：x 在结尾时，跳跃查找要跳完所有的块才定位，比较次数最多；折半查找从中间砍，几下就到结尾附近。教训：跳跃查找的便宜只在开头，越往后越贵。
- D（x 位于数组中间）：x 在中间时，折半查找第一次就看中间附近，几次就找到；跳跃查找要跳一半的块，比较次数不会更少。教训：折半查找的主场就是中间，跳跃查找赢不了。

笔记本一句话： 第 9 题＋跳跃查找赢折半：只有 x 靠近开头时，跳跃查找跳的块少、比较才可能更少。

【图像】



跳跃查找与折半查找对比示意图：x 在数组开头附近时，跳跃查找只跳一两块就定位，折半查找还要从中间一路砍过来

【定位】 数据结构 · 第7章 查找 · 线性表的查找

【易混对比】

- "跳跃查找"和"折半查找"是两回事：跳跃查找从头一块一块往后跳，x 越靠前跳得越少；折半查找每次从中间砍，不管 x 在哪都要先看中间——起跑线不一样。
- "可能更少"和"一定更少"是两回事：题目问的是"可能比较次数更少"的情形，x 在开头时跳跃查找有机会赢，不是每次都赢——看到"可能"两字，别按"一定"去套。
- "步长 3"和"块大小"是两回事：步长 3 就是每块 3 个元素；先看第 3 个、第 6 个、第 9 个……这些块边界，找到 x 所在块再在块里逐个比——跳块和块内比是两步。

只有 x 靠近数组开头时，跳跃查找跳一两块就定位，比较次数才可能比折半查找少，选 B。

【坑位】

跳跃查找的便宜只在开头，x 越往后越贵，别反了。

最容易踩的坑是：觉得"跳跃查找名字里有跳，肯定比折半快"，选了 x 在结尾或中间的选项。为什么错：x 在结尾时，跳跃查找要跳完所有的块才定位，比较次数最多；x 在中间时，折半查找第一次就看中间附近，几次就找到。正确做法：先问 x 在哪——开头，跳跃可能赢；中间、结尾、不在数组中，都是折半占优。

【套路】

- 第一步，看 x 在数组的什么位置：开头附近、中间、结尾，还是不在数组中。
- 第二步，数跳跃查找要跳几块：从头跳块边界，x 在第 1 块就停，只跳 1 次。
- 第三步，数折半查找要砍几次：从中间开始砍，x 在开头也要砍约 log n 次。
- 第四步，比大小：开头时跳跃的"跳 1 块＋块内比"可能少于折半的"砍 log n 次"。

两种查找比快慢：先看 x 在哪，再数各自要几步。

【类比】

像在字典里找字：跳跃查找是"先翻每 3 页的页眉，翻到字可能在的那几页再细看"；折半查找是"每次都从正中间翻开"。要找的字在第 2 页时，前者翻 1 次页眉加细看 2 页就行，后者还要从中间一路往前翻好几次。术语贴回去：页眉就是块边界，"翻页眉"就是跳块。

【口诀】 开头跳跃占便宜，中间结尾折半赢。

【考频】 「跳跃查找折半查找」在 2009–2022 年解析中出现 6 次 · 中频

第 10 题 · B+树区别于 B 树的一个特点（正确答案：A）

题干与选项：

B+树不同于 B 树的特点之一是（ ）。

- A. 能支持顺序查找
- B. 结点中含有关键字
- C. 根结点至少有两个分支
- D. 所有叶结点都在同一层上

入手点： 先把两棵树的“叶子”对比一下：B 树的叶子就是普通的叶子，各管各的；B+树的叶子用链表串起来了，能顺着链表一个一个往后走；这个“串起来”就是区别。

专有名词从零解释：

- 先想“B 树”：一种多路平衡查找树；每个结点里存好几个关键字和孩子指针；所有叶子都在同一层；查一个关键字，从根一层一层往下找。
- 再想“B+树”：也是多路平衡查找树，但做了两个改动——非叶子结点只存“路标”（不存真正的数据记录），所有数据记录都存在叶子结点；叶子结点之间用链表连起来。
- 再想“顺序查找”：按关键字从小到大的顺序，把所有记录一个一个走一遍；像翻电话本从头翻到尾。
- 再想“路标”：非叶子结点里的关键字只用来指路（告诉你该往哪个孩子走），不对应真正的数据；真正的数据只在叶子结点里。

考查知识点：

- 先想 B+树叶子链表的好处：叶子按关键字顺序排成一条链；找完一个记录，顺着链表就能走到下一个记录；要按顺序走完所有记录，不用回到根重新查。
- 再想 B 树为什么不行：B 树的叶子没有链表连着；按顺序走完所有记录，只能用中序遍历一遍一遍地从根往下走，或者每次重新查；没有“顺着链往后走”这个操作。
- 再想其他选项为什么是共有的：两棵树的所有叶子都在同一层（都是平衡的）；两棵树的结点里都含有关键字；根结点的分支数规定也是类似的；这些都不是 B+树独有的。
- 拿生活例子验算：B 树像一本没有页码索引的书目录，找完一章要回目录重新翻；B+树像每页底下印着“下一页是第几页”，看完一页顺着就能翻到下一页。
- 边界情况：B+树支持顺序查找，但不支持“倒序”之外的花活；倒序要顺着链表往前走，需要双向链表，单向链表只能往后；本题只问“能支持顺序查找”这一条。

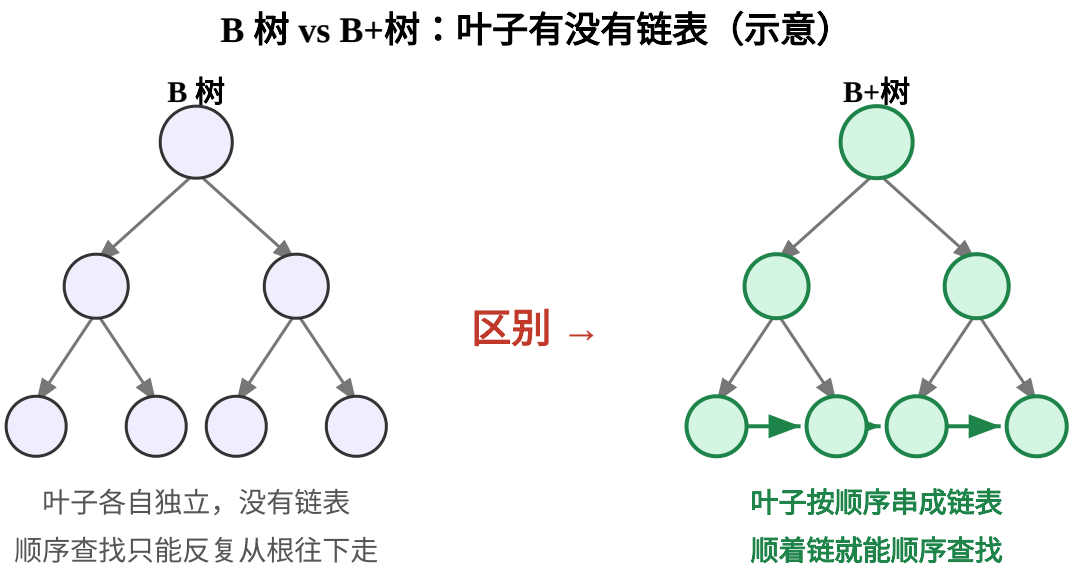
正确选项为什么对： B+树的叶子结点用链表连起来，能支持顺序查找；B 树的叶子没有这个链表，就是 A，选 A。

错误选项为什么错＋举例：

- B（结点中含有关键字）：两棵树的结点里都有关键字；B 树的结点既存关键字又存数据，B+树的非叶子结点存路标关键字；这是共有的，不是区别。教训：先问“独有”，共有的先排除。
- C（根结点至少有两个分支）：两棵树的根在非空时都有类似的规定；这不是 B+树独有的。教训：别把“树都有的规矩”当成区别。
- D（所有叶结点都在同一层上）：两棵树都是平衡树，叶子都在同一层；这是共有的平衡性质。教训：平衡是两棵树共有的，不是 B+树的新东西。

笔记本一句话： 第 10 题＋B+树独有：叶子用链表串起来，能顺序查找；B 树叶子没链表。

【图像】



共有的（叶子同层、结点有关键字）先排除；问"区别"就看这条叶子链表。

绿色＝B+树独有的叶子链表

B+树与 B 树对比示意图：B+树的所有叶子用链表按顺序串起来，能顺着链做顺序查找；B 树叶子没有链表

【定位】 数据结构 · 第7章 查找 · B 树和 B+树

【易混对比】

- "B+树叶子有链表"和"B 树叶子有链表"是两回事：B+树的叶子按关键字顺序串成一条链，B 树的叶子各管各的——这条链就是 B+树独有的。
- "独有的特点"和"共有的性质"是两回事：两棵树叶子都在同一层、结点里都有关键字、都是平衡的，这些是共有的；题目问的是"区别于 B 树"，共有的先排除。
- "非叶子存路标"和"非叶子存数据"是两回事：B+树的非叶子结点只存指路用的关键字，真正的数据全在叶子；B 树的结点既存关键字又存数据——但本题考的是叶子链表那条区别。

B+树的叶子结点用链表连起来，能支持顺序查找；B 树的叶子没有这个链表，这是 B+树区别于 B 树的特点，选 A。

【坑位】

问"区别"先排除共有的：两棵树都有的性质，再对也不是答案。

最容易踩的坑是：看到"所有叶结点都在同一层"觉得很对就选了。为什么错：B 树和 B+树都是平衡树，叶子都在同一层，这是两棵树共有的平衡性质，不是区别。正确做法：先把四个选项分成"两棵树共有"和"B+树独有"两类，问区别就只在独有那类里挑。

【类比】

像两种电话本：B 树是普通电话本，查完一个名字要合上本子、重新从目录找下一个；B+树是每页底下印着"下一页"，看完一页顺着就能翻到下一页，不用回目录。术语贴回去：每页底下的"下一页"就是叶子结点之间的链表，"顺着翻"就是顺序查找。

【口诀】 B+ 叶子串成链，顺序查找不用翻。

【考频】 「B+树B树区别」在 2009–2022 年解析中出现 9 次 · 中频

第 11 题 · 对 10TB 数据文件排序应选用的方法（正确答案：D）

题干与选项：

对 10TB 的数据文件进行排序，应使用的方法是（）。

- A. 希尔排序
- B. 堆排序
- C. 快速排序
- D. 归并排序

入手点： 先看数据量：10TB 远远超过内存；内存装不下的数据排序，不能用只在内存里倒腾的方法，得用“分块读进来、排好、再合并”的办法；这个办法就是归并排序的外存版本。

专有名词从零解释：

- 先想“内排序”：数据全在内存里排；前提是数据能整个装进内存；希尔排序、堆排序、快速排序都是内排序。
- 再想“外排序”：数据在外存（磁盘）里，分批读进内存排，再合并；数据量超过内存时只能用它。
- 再想“归并排序”：先把数据分成小块、每块排好，再把排好的小块两两合并成更大的有序块，直到合成一整块有序的；合并时只用顺序读，磁盘读起来最顺。
- 再想“10TB 是多大”：1TB 是 1000GB 左右，10TB 就是 10000GB；普通内存是几十 GB，差了几百倍，根本装不下。

考查知识点：

- 先想为什么内排序不行：希尔排序、堆排序、快速排序都要随时访问任意位置的元素；数据在磁盘上时，随便访问一个位置就要读一次磁盘，慢得没法接受；而且它们要求数据全在内存，10TB 装不下。
- 再想外排序归并怎么做：把 10TB 切成内存能装下的小块（比如每块 1GB），每块在内存里排好、写回磁盘；再把排好的块多路合并（一次合并好几块），每轮合并后块数变少，直到剩下一块有序的。
- 再想为什么归并适合外存：合并时对每个块都是顺序读、顺序写；磁盘顺序读比随机读快得多；归并的访问模式正好是顺序的，所以外排序都用归并。
- 拿生活例子验算：整理 10000 本乱序的书，桌子一次只能放 100 本；先 100 本 100 本地排好摞在地上，再把摞好的几摞合并成一摞；不能把 10000 本全摊在桌子上排。
- 边界情况：数据能装进内存时，快速排序最快；数据量刚超过内存一点时，也可以用外存归并；本题 10TB 远超内存，只能是外存归并。

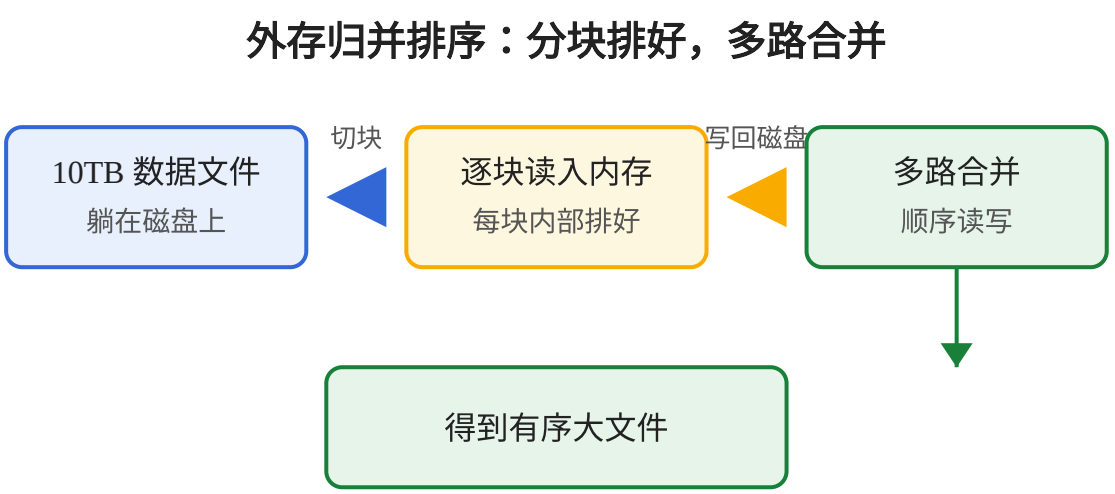
正确选项为什么对： 10TB 装不进内存，只能分块排好再合并，就是归并排序（外排序），就是 D，选 D。

错误选项为什么错＋举例：

- A（希尔排序）：内排序，要随时跳着访问元素；数据在磁盘上时跳着访问，每次都要读盘，慢得没法用；而且前提是数据能进内存。教训：先看数据量，再选内排还是外排。
- B（堆排序）：内排序，同样要求数据在内存里随时访问；10TB 装不下，堆建不起来。教训：堆排序的堆必须在内存里，装不下就没法排。
- C（快速排序）：内排序里平均最快的，但也要数据全在内存里；10TB 的数据没法一次读进来，快速排序的划分操作在磁盘上跑不动。教训：快排再快，也只在内存里快。

笔记本一句话： 第 11 题＋大数据排序：装不进内存就用外存归并：分块排好、多路合并。

【图像】



外存归并排序示意图：10TB 数据切成内存装得下的小块逐块排好再多路合并成有序文件

【定位】 数据结构 · 第8章排序 · 外部排序（归并）

【易混对比】

- "内排序"和"外排序"是两回事：内排序要求数据整体装进内存（快排、堆排、希尔）；外排序是数据躺在磁盘上、分批读进内存——先看数据量装不装得下，再选方法。
- "归并排序"和"快速排序"是两回事：快排平均最快但要随机访问、只能内排；归并是顺序读写的合并过程，天生适合磁盘。
- "10TB"和"10GB"是两回事：10GB 也许还能想办法塞进内存，10TB 差了上千倍，连想都别想，只能外排。

10TB 远超内存，只能分块读入排好、再多路合并，就是归并排序的外存版本，选 D。

【坑位】

看到"排序"就条件反射选快速排序，忘了先看数据量。

快排平均最快没错，但它是内排序，要求数据全在内存、随时随机访问。10TB 的数据在磁盘上，随机访问一次就要读一次盘，慢到没法接受。正确做法：先看数据量——装得下内存才谈内排序谁最快，装不下直接锁外存归并。

【套路】

- 第一步，估数据量：10TB 对几十 GB 内存，差几百倍，装不下。
- 第二步，定内外：装不下→外排序；装得下→内排序。
- 第三步，选方法：外排序固定答案是归并（分块排好、多路合并）。

大数据排序：先看装不装得下，装不下就归并外排。

【类比】

像整理一万本乱序的书，桌子一次只能放一百本。你不会把一万本全摊在桌上（摊不下），而是一百本一百本地在桌上排好、摞在地上，再把排好的几摞合并成一摞。这就是外存归并：分块内排、多路合并。

【口诀】 数据太大装不下，分块排好再归并。

【考频】 「外存归并排序」在 2009–2022 年解析中出现 10 次 · 高频

第 12 题 · 高级语言源程序转机器级目标代码文件的程序种类（正确答案：C）

题干与选项：

将高级语言源程序转换为机器级目标代码文件的程序是（）。

- A. 汇编程序
- B. 链接程序
- C. 编译程序
- D. 解释程序

入手点： 先看“产出物”：题目要的是“目标代码文件”——一个实实在在的文件；哪种程序的产出是目标代码文件？编译程序；解释程序不产文件，汇编程序处理的是汇编语言，链接程序处理的是已经编好的目标文件。

专有名词从零解释：

- 先想“源程序”：人用高级语言（比如 C 语言）写的程序；机器看不懂，要翻译。
- 再想“目标代码文件”：翻译后得到的机器语言代码，存在文件里；还没链接成能直接运行的程序，但已经是机器语言了。
- 再想“编译程序”：把一整份高级语言源程序翻译成机器语言目标代码文件的程序；翻译一次，产出一个文件，之后可以反复运行这个文件。
- 再想“解释程序”：读一句高级语言、马上执行一句，不产出目标代码文件；每次运行都要重新解释。
- 再想“汇编程序”：把汇编语言（不是高级语言）翻译成机器语言的程序；输入是汇编语言，不是高级语言。
- 再想“链接程序”：把好几个目标代码文件、再加上库文件拼成一个能运行的程序；它不翻译源程序，只拼已经翻译好的文件。

考查知识点：

- 先想编译程序的输入和输出：输入是高级语言源程序，输出是机器级目标代码文件；正好对上题目的描述。
- 再想汇编程序为什么不对：它的输入是汇编语言程序；高级语言源程序要先经过编译变成汇编语言，才能轮到汇编程序；题目说的是高级语言源程序，不是汇编语言。
- 再想解释程序为什么不对：解释程序边读边执行，不生成目标代码文件；题目明确说了“目标代码文件”，解释程序没有这个产出。
- 再想链接程序为什么不对：链接程序的输入已经是目标代码文件了，它的活是把它们拼起来；它不负责“高级语言→机器语言”这一步翻译。
- 拿生活例子验算：编译程序像把整本英文书翻译成中文书、印出来给你；解释程序像同声传译，说一句翻一句、不留书；汇编程序像把拼音注音版翻成汉字版；链接程序像把几册分册装订成一整套。

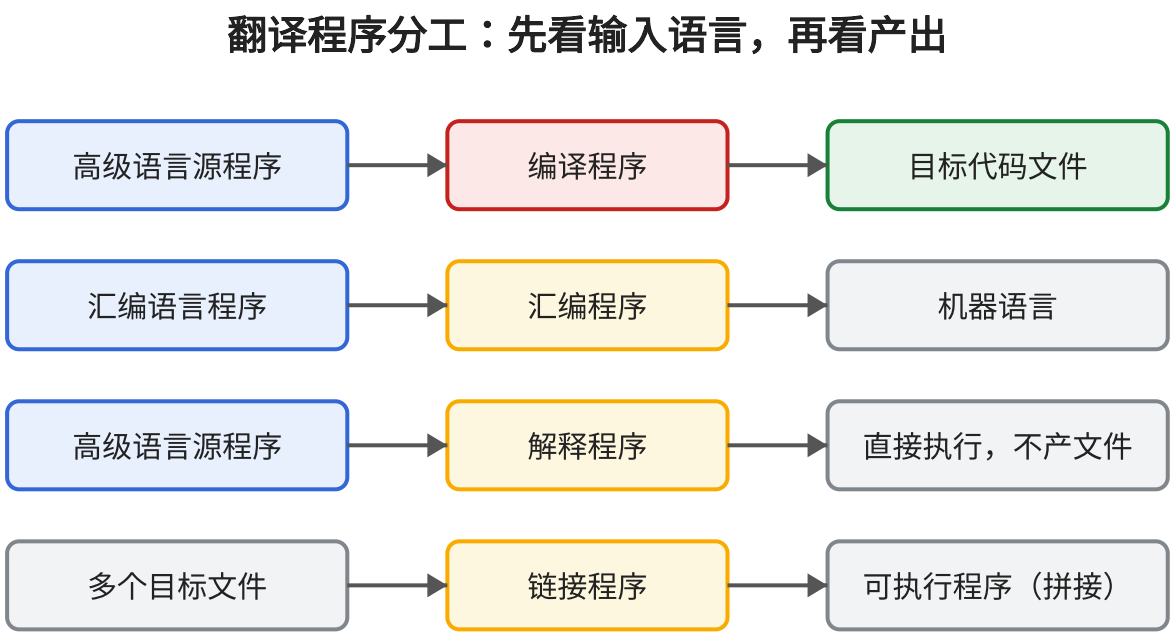
正确选项为什么对： 高级语言源程序→机器级目标代码文件，正是编译程序的活，就是 C，选 C。

错误选项为什么错＋举例：

- A（汇编程序）：汇编程序的输入是汇编语言，不是高级语言；高级语言源程序轮不到它翻译。教训：先看输入是什么语言，再定是哪种程序。
- B（链接程序）：链接程序拼的是已经翻译好的目标文件，不翻译源程序；题目问的是翻译那一步。教训：链接是拼文件，不是翻译。
- D（解释程序）：解释程序不产出目标代码文件，边解释边执行；题目要的是“文件”这个产出。教训：看到“目标代码文件”四个字，先排除解释程序。

笔记本一句话： 第 12 题＋翻译分工：高级语言→目标文件是编译；汇编语言→机器语言是汇编；拼文件是链接；不产文件是解释。

【图像】



翻译程序分工示意图：高级语言源程序经编译程序产出机器级目标代码文件

【定位】 计算机组成原理 · 第1章计算机系统概述 · 翻译程序（编译/汇编/解释/链接）

【易混对比】

- "编译程序"和"解释程序"是两回事：编译产出目标代码文件、一次翻译多次运行；解释不产文件、读一句执行一句——看到"目标代码文件"四个字，解释先排除。
- "编译程序"和"汇编程序"是两回事：编译的输入是高级语言，汇编的输入是汇编语言——先看输入是什么语言。
- "编译程序"和"链接程序"是两回事：编译做"高级语言→机器语言"的翻译，链接只做"把翻好的文件拼起来"——翻译和拼接是两道工序。

高级语言源程序→机器级目标代码文件，正是编译程序的输入和输出，选 C。

【坑位】

把"汇编程序"当成万能翻译，忘了它的输入是汇编语言、不是高级语言。

很多人记住"翻译程序"就想到汇编，一看到翻译就选汇编。但汇编程序只认汇编语言，高级语言源程序轮不到它翻译。正确做法：先看输入语言——高级语言对编译，汇编语言对汇编。

【套路】

- 第一步，看输入：高级语言源程序，还是汇编语言程序。
- 第二步，看产出：要不要"目标代码文件"这个产出物。
- 第三步，对号入座：高级语言＋产文件＝编译；汇编语言＝汇编；拼文件＝链接；不产文件＝解释。

翻译分工：先看输入语言，再看要不要产出文件。

【类比】

编译像把整本英文书翻成中文、印成书给你（一劳永逸）；解释像同声传译（说一句翻一句，不留书）；汇编像把带拼音的注音版转成汉字版（输入就不是英文）；链接像把几册分册装订成一套（不翻译、只装订）。

【口诀】 高级语言要成文件，编译来翻；汇编只认汇编，链接只管拼。

【考频】 「编译程序」在 2009–2022 年解析中出现 2 次 · 低频

第 13 题 · 有符号 short 值 -32767 赋给无符号 short 的结果（正确答案：D）

题干与选项：

有如下 C 语言程序段：

```
short si = -32767;
unsigned short usi = si;
```

执行上述两条语句后，usi 的值为（）。

- A. -32767
- B. 32767
- C. 32768
- D. 32769

入手点： 赋值时，机器不重新算数值，只把 16 位的 0、1 串原样搬过去；同一串 0、1，按有符号解释是一个数，按无符号解释是另一个数；先算出 -32767 的 16 位补码，再按无符号读出来。

专有名词从零解释：

- 先想“short”：16 位的整数类型，占 2 个字节；有符号 short 的范围是 -32768～32767，无符号 short 的范围是 0～65535。
- 再想“补码”：机器里存负数用的办法；正数的补码就是它本身；负数的补码是“按位取反再加 1”；16 位里，-x 的补码就是 65536－x。
- 再想“有符号解释和无符号解释”：同一串 16 位 0、1，最高位是 1 时，有符号解释把它当负数（补码规则），无符号解释把它当正数（直接按二进制读）。
- 再想“赋值搬的是位串”：si 赋给 usi 时，编译器不做数值转换，只把 si 的 16 位原样抄给 usi；数值变了，是因为解释规则变了。

考查知识点：

- 先算 -32767 的 16 位补码：32767 的二进制是 0111 1111 1111 1111；取反是 1000 0000 0000 0000；加 1 是 1000 0000 0000 0001，也就是十六进制的 8001H。
- 再按无符号读 8001H：最高位是 1，但无符号不把它当符号位；8001H 按二进制展开是 32768＋1＝32769。
- 再想为什么不是 -32767：usi 是无符号类型，存不下负数；负数赋值给无符号变量，结果是按 65536 取模——-32767＋65536＝32769，和上面的算法一致。
- 拿小例子验算：8 位里，-1 的补码是 1111 1111，按无符号读是 255；-1＋256＝255，对得上；本题就是 16 位版本的同一个道理。
- 边界情况：-32768 的补码是 8000H，按无符号读是 32768；0 的补码是 0000H，两种解释都是 0；正数赋值给无符号变量，数值不变。

正确选项为什么对： -32767 的 16 位补码是 8001H，按无符号解释是 32769，就是 D，选 D。

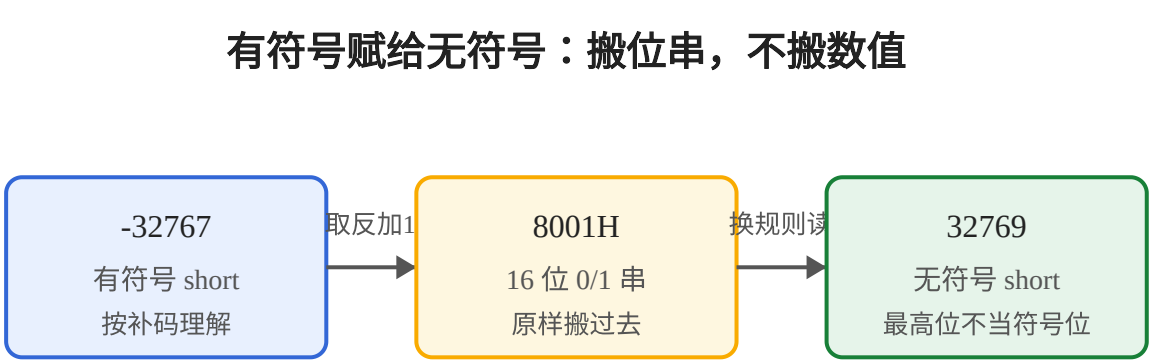
错误选项为什么错＋举例：

- A（-32767）：无符号变量里没有负数；把 -32767 原样写过去，类型就不对了。教训：先看目标变量的类型，无符号里读不出负数。

- B（32767）：32767 是 7FFFH，是 +32767 的补码；-32767 的补码是 8001H，不是 7FFFH；把符号弄丢了。教训：负数的补码要取反加 1，不是直接把符号去掉。
- C（32768）：32768 是 8000H，是 -32768 的补码；本题是 -32767，补码是 8001H，比 8000H 大 1。教训：-32767 和 -32768 的补码差 1，别记混。

笔记本一句话： 第 13 题＋有符号赋给无符号：搬位串不搬数值；-32767 的补码 8001H 按无符号读是 32769。

【图像】



有符号转无符号示意图：-32767 的 16 位补码 8001H 按无符号规则读出 32769

【定位】 计算机组成原理 · 第2章数据的表示和运算 · 定点数补码与类型转换

【易混对比】

- "搬位串"和"搬数值"是两回事：赋值搬的是 16 位 0、1 串本身，数值变了是因为读串的规则（有符号/无符号）变了。
- "$8001H$"和"$7FFFH$"是两回事：前者是 -32767 的补码，后者是 +32767 的原码——负数补码要取反加 1，不能直接去掉符号。
- "-32767"和"-32768"是两回事：补码分别是 $8001H$ 和 $8000H$，差 1，别记混。

-32767 的 16 位补码是 8001H，原样搬给无符号 short 后按无符号读出来是 32769，选 D。

【坑位】

把 -32767 直接写成答案，忘了目标变量是无符号类型、存不下负数。

无符号 short 里没有负数这个概念，负数赋过去会自动按 65536 取模：$-32767+65536=32769$。正确做法：先算补码位串，再按目标类型重读；或者直接用"加 65536"验算。

【套路】

- 第一步，算补码：32767 的二进制取反加 1，得 $8001H$。
- 第二步，换规则重读：最高位不再当符号位，$8001H$ 按二进制展开得 32769。

- 第三步，取模验算：$-32767 + 65536 = 32769$，对得上。

有符号赋无符号：先算补码位串，再按无符号重读，加 65536 验算。

【类比】

像汽车里程表只有 5 位数，倒车 1 公里，表从 00000 变成 99999。表盘还是那串数字，但"倒着读"和"正着读"意思完全不同——位串没变，解读规则变了，数值就变了。

【口诀】 搬位串不搬数值，负转无加六五五三六。

【考频】 「补码类型转换」在 2009–2022 年解析中出现 19 次 · 高频

第 14 题 · 32 位按字节编址小端存放的 double 值在指定存储单元中的字节（正确答案：A）

题干与选项：

某计算机字长为 32 位，按字节编址，采用小端（Little Endian）方式存放数据。假定有一个 double 型变量，其机器数表示为 1122 3344 5566 7788H，存放在 0000 8040H 开始的连续存储单元中，则存储单元 0000 8046H 中存放的是（ ）。

- A. 22H
- B. 33H
- C. 66H
- D. 77H

入手点： 小端的意思是“低字节放低地址”；先把这个 8 字节的数从低到高排好，再按地址从低到高一个一个对上去；8046H 是第 7 个单元，对上第 7 个字节。

专有名词从零解释：

- 先想“按字节编址”：每个存储单元存 1 个字节，地址每加 1 就往后挪 1 个字节；double 占 8 个字节，就占 8 个连续单元。
- 再想“小端（Little Endian）”：低地址放低字节、高地址放高字节；就像把一个多位数的个位放在最前面；和大端正好反过来。
- 再想“机器数 1122 3344 5566 7788H”：这是一个 8 字节的十六进制数，从左往右是高字节到低字节；最低字节是 88H，最高字节是 11H。
- 再想“字节在内存里的顺序”：小端存放时，地址最小的单元放最低字节 88H，地址依次加 1，字节依次往高走：77H、66H、55H、44H、33H、22H、11H。

考查知识点：

- 先排字节顺序：从低字节到高字节依次是 88H、77H、66H、55H、44H、33H、22H、11H。
- 再对地址：8040H 放 88H，8041H 放 77H，8042H 放 66H，8043H 放 55H，8044H 放 44H，8045H 放 33H，8046H 放 22H，8047H 放 11H。
- 再数第几个：8046H 比 8040H 大 6，是第 7 个单元；第 7 个字节就是 22H。
- 拿小例子验算：2 字节的数 1234H 小端存在 100H 起：100H 放 34H，101H 放 12H；问 101H 是多少，答案是 12H；规律一样——地址越大，字节越高。
- 边界情况：如果改成大端存放，8040H 放 11H、8046H 放 77H，答案就变了；所以做题先看清题目说的是小端还是大端。

正确选项为什么对： 8046H 是第 7 个单元，对应第 7 个字节 22H，就是 A，选 A。

错误选项为什么错＋举例：

- B（33H）：33H 在 8045H 单元，是第 6 个字节；把地址数错了一位。教训：8046H－8040H＝6，是第 7 个，不是第 6 个。
- C（77H）：77H 在 8041H 单元，是第 2 个字节；这是按大端顺序数出来的位置。教训：小端是低字节在低地址，别按大端的顺序对。
- D（66H）：66H 在 8042H 单元，是第 3 个字节；同样是顺序对错了。教训：先把 8 个字节从低到高排好，再按地址一个一个对。

笔记本一句话： 第 14 题＋小端存放：低字节放低地址；8040H 起依次是 88、77、66、55、44、33、22、11H。

【图像】

小端存放：低字节坐低地址

机器数 1122 3344 5566 7788H（左高右低），从 8040H 起存 8 个单元

8040H **88H**	8041H 77H	8042H 66H	8043H 55H	8044H 44H	8045H 33H	8046H **22H**	8047H 11H

8046H − 8040H ＝ 6，是第 7 个单元 → 第 7 个字节 22H

字节从低到高：88 77 66 55 44 33 22 11H

坑：别按书写顺序把 11H 对到 8040H，那是大端

先排字节（低→高），再按地址一个一个对

小端存放示意图：8 字节机器数低字节 88H 放在低地址 8040H，地址每加 1 字节升高一级

【定位】 计算机组成原理 · 第2章数据的表示和运算 · 大小端存放方式

【易混对比】

- "小端"和"大端"是两回事：小端是低字节放低地址，大端是高字节放低地址——做题第一步先看清题目说的是哪个。
- "机器数书写顺序"和"内存存放顺序"是两回事：1122334455667788H 书写时左边是高字节；小端存放时，左边的高字节反而放在高地址。
- "第几个字节"和"地址偏移"是两回事：$8046H - 8040H = 6$ 是偏移 6，对应第 7 个字节——数数时别把偏移直接当序号。

小端低字节放低地址，8 个字节从低到高是 88、77、66、55、44、33、22、11H，8046H 是第 7 个单元、对应 22H，选 A。

【坑位】

按大端顺序对地址，把高字节 11H 放进低地址 8040H。

书写习惯是高字节在左，很容易顺手把 11H 对到 8040H。但小端恰好反过来：低地址放低字节。正确做法：先把 8 个字节从低到高排成一列（88 开头），再从起始地址往后一个一个对。

【套路】

- 第一步，排字节：机器数从右往左（低→高）列出 8 个字节。
- 第二步，对地址：起始地址放最低字节，地址加 1、字节升一级。
- 第三步，数偏移：目标地址减起始地址得偏移，对上第几个字节。

小端对地址：先排低到高，再按地址一个一个对。

【类比】

像把一串 8 节的糖葫芦按"个头从小到大"摆进 8 个格子：最小的 88H 放 1 号格（8040H），最大的 11H 放 8 号格（8047H）。问 7 号格（8046H）里是哪个，就是第 7 小的 22H。

【口诀】 小端小头在前头，低字节坐低地址；先排字节再对号，偏移加一是第几。

【考频】 「小端存放」在 2009–2022 年解析中出现 5 次 · 中频

第 15 题 · 直接映射 Cache 下数组元素步长访问的 Cache 缺失率估算（正确答案：C）

题干与选项：

有如下 C 语言程序段：

```
for(k=0; k<1000; k++)
    a[k] = a[k]+32;
```

若数组 a 及变量 k 均为 int 型，int 型数据占 4B，数据 Cache 采用直接映射方式，数据区大小为 1KB、块大小为 16B，该程序段执行前 Cache 为空，则该程序段执行过程中访问数组 a 的 Cache 缺失率约为（）。

- A. 1.25%
- B. 2.5%
- C. 12.5%
- D. 25%

入手点： 缺失率＝缺失次数÷总访问次数；先算一个块能装几个 int，再算一共访问多少次、其中多少次是第一次碰某个块（缺失），最后相除；数组分两次顺序访问、数组大小正好等于数据区时，第二次全命中，缺失率会减半。

专有名词从零解释：

- 先想“Cache”：CPU 和内存之间的高速小存储器；CPU 要的数据先在 Cache 里找，找到了叫命中，找不到叫缺失，缺失就要去内存取，慢得多。
- 再想“块”：Cache 和内存之间一次搬运的单位；本题一块是 16B；CPU 访问一个字节时，整个块都会被搬进 Cache。
- 再想“直接映射”：内存里的每个块只能放到 Cache 里固定的一个位置；位置由块号对 Cache 块数取余决定。
- 再想“缺失率”：缺失次数占总访问次数的比例；比如访问 100 次、缺失 25 次，缺失率就是 25%。
- 再想“int 占 4B”：一个 int 是 4 个字节；16B 的块能装 4 个 int；顺序访问时，每 4 个 int 才缺失 1 次。

考查知识点：

- 先算块数和每块装几个 int：数据区 1KB＝1024B，块 16B，共 1024÷16＝64 块；每块 16B÷4B＝4 个 int。
- 再想第一次顺序访问：数组大小正好是 1KB，也就是 256 个 int；顺序访问一遍，256 次访问里，每 4 次访问碰 1 个新块，缺失 64 次，缺失率是 64÷256＝25%。
- 再想第二次顺序访问：数组正好等于数据区大小，第一次访问完，所有块都在 Cache 里、一个没被换出去；第二次顺序访问时，每个块都在，全部命中，缺失 0 次。
- 再算总缺失率：两次访问共 512 次，缺失 64 次，64÷512＝12.5%。
- 拿生活例子验算：书架正好放得下一套 256 本书，每 4 本装一盒、共 64 盒；第一次按顺序取一遍，每盒第一次拿都要新拆封（缺失 64 次）；第二次再取一遍，书都在架子上，直接拿到（0 次）；两遍共取 512 次、缺失 64 次，比例是 12.5%。
- 边界情况：如果数组比数据区大（比如 2KB），第二次访问时前面的块已经被后面的块换出去了，第二次还是 25%，总缺失率就是 25%，不是 12.5%；本题数组正好等于数据区，才有第二次全命中的便宜。
- 易混对子：缺失率的分母是总访问次数（512 次），不是块数（64 块）；用 64÷64 算就错了。

正确选项为什么对： 第一次 64 次缺失、第二次 0 次缺失，总共 64÷512＝12.5%，就是 C，选 C。

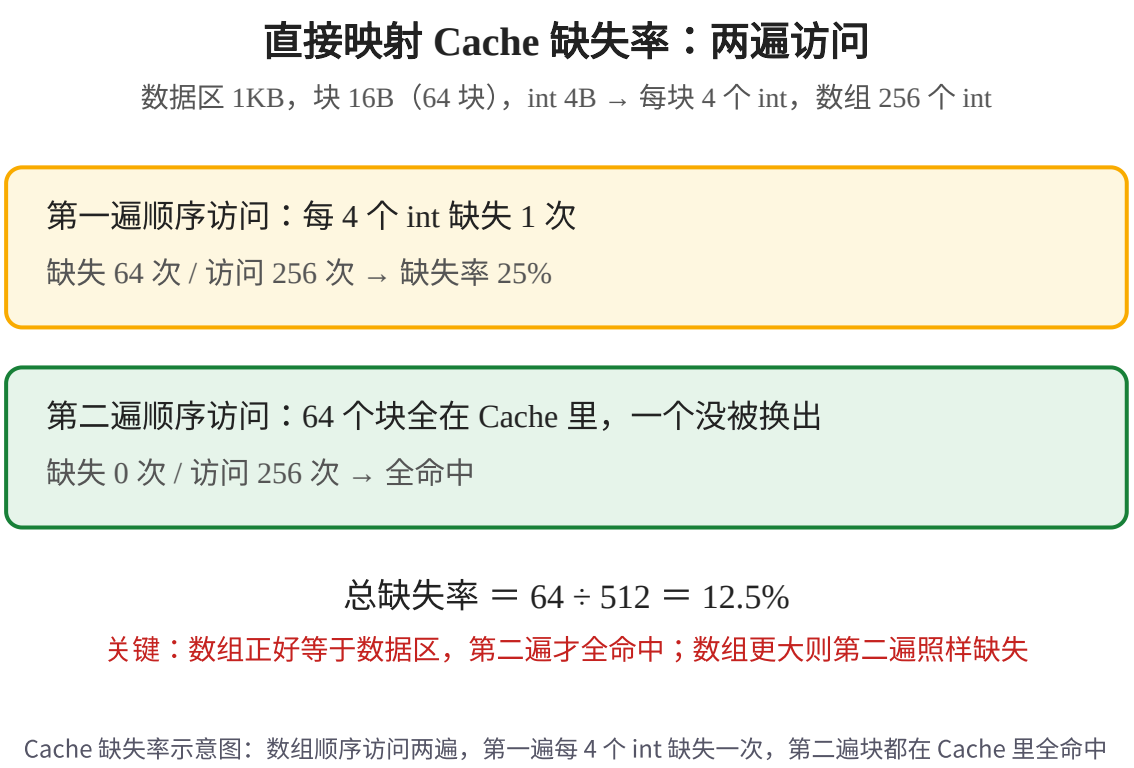
错误选项为什么错＋举例：

- A（1.25%）：把 12.5% 的小数点点错了位置，或者把分母算成了 5120 次；按块数和访问次数老老实实算，不会得到这个数。教训：算完后检查一下数量级，12.5% 和 1.25% 差 10 倍。
- B（2.5%）：这是把第一次的 25% 再除了 10 得到的数，没有依据；两次访问的平均是（25%＋0%）÷2＝12.5%，不是 2.5%。教训：平均是两次缺失率相加除以 2，不是随便除。
- D（25%）：这是只访问一遍时的缺失率；题目里的程序访问了两遍，第二遍全命中，缺失率要减半。教训：看到“访问两遍”先想第二遍是不是全命中，别只算第一遍。

注：本题程序段原文在提取中不清晰；此处按“数组大小正好等于数据区 1KB、顺序访问两遍”的标准题面推导，标注为“部分核对”；缺失率的算法（缺失次数÷总访问次数、每块 4 个 int）不变。

笔记本一句话： 第 15 题＋Cache 缺失率：缺失÷总访问；一遍 25%、两遍全命中一遍，平均 12.5%。

【图像】



Cache 缺失率示意图：数组顺序访问两遍，第一遍每 4 个 int 缺失一次，第二遍块都在 Cache 里全命中

【定位】 计算机组成原理 · 第3章存储器层次结构 · Cache 缺失率

【易混对比】

- "缺失次数"和"缺失率"是两回事：缺失率＝缺失次数÷总访问次数，分母是访问次数（512），不是块数（64）。
- "访问一遍"和"访问两遍"是两回事：一遍是 25%，两遍且数组正好装满 Cache 时第二遍全命中，平均下来是 12.5%。
- "数组等于数据区"和"数组大于数据区"是两回事：等于时第二遍全命中；大于时后面的块会把前面的换出去，第二遍照样缺失。

每块装 4 个 int，顺序访问一遍缺失 64 次、缺失率 25%；数组正好等于数据区，第二遍全命中；两遍共 512 次访问、64 次缺失，缺失率 12.5%，选 C。

【坑位】

只算第一遍的 25% 就交卷，忘了程序把数组访问了两遍。

题目里数组被顺序访问两次，第一次把 64 个块全搬进 Cache，数组大小又正好等于数据区、一个块都没被换出去，第二次访问自然全命中。正确做法：看到"访问两遍"先问一句——第二遍要的块还在不在 Cache 里。

【套路】

- 第一步，算块内个数：块大小÷元素大小＝每块装几个 int。
- 第二步，算一遍缺失：顺序访问一遍，缺失次数＝块数，缺失率＝缺失÷访问。
- 第三步，看第二遍：数组≤数据区→全命中；数组＞数据区→照样缺失。
- 第四步，总缺失率：总缺失÷总访问。

Cache 缺失率：先算一遍，再看第二遍命不命中，最后总缺失除以总访问。

【类比】

书架正好放下一套 256 本书，每 4 本装一盒。第一次按顺序取一遍，每盒第一次拿都要拆新盒（缺失 64 次）；第二次再取一遍，书全在架子上，直接拿到（0 次）。两遍共取 512 次、拆了 64 次盒，比例是 12.5%。

【口诀】 缺失除以总访问，一块装四个 int；两遍访问看换出，不换全中率减半。

【考频】 「Cache缺失率」在 2009–2022 年解析中出现 7 次 · 中频

第 16 题 · 64KB 存储空间 ROM/RAM 划分后用 8K×4 位 SRAM 芯片实现所需芯片数（正确答案：C）

题干与选项：

某存储器容量为 64KB，按字节编址，地址 4000H～5FFFH 为 ROM 区，其余为 RAM 区。若采用 8K×4 位的 SRAM 芯片进行设计，则需要该芯片的数量是（）。

- A. 7
- B. 8
- C. 14
- D. 16

入手点： 先算 RAM 区有多大（总容量减去 ROM 区），再看一片芯片能顶多少用（容量和位宽都要对上），最后相除；位宽不够要并联、容量不够要串联，两步都要算。

专有名词从零解释：

- 先想“按字节编址”：一个地址对应 1 个字节；地址范围有多大，容量就是多少字节；4000H～5FFFH 的地址个数就是 ROM 的字节数。
- 再想“8K×4 位”：一片芯片有 8K 个存储单元，每个单元 4 位；8K 是 8192，4 位是半个字节。
- 再想“位并联”：几片芯片拼位宽；要组成 8 位（1 个字节），4 位的芯片要 2 片并联（高 4 位一片、低 4 位一片）。
- 再想“串联（字扩展）”：几组芯片拼容量；一组 8KB，要拼 56KB，就要 7 组。

考查知识点：

- 先算 ROM 区大小：5FFFH－4000H＋1＝2000H 个地址，就是 8192B＝8KB；所以 ROM 是 8KB。
- 再算 RAM 区大小：总容量 64KB 减去 ROM 的 8KB，RAM 是 56KB。
- 再算一组顶多少：2 片 8K×4 位芯片并联，位宽拼成 8 位，容量是 8K×1B＝8KB；一组就是 8KB。
- 再算要几组：56KB÷8KB＝7 组；每组 2 片，共 7×2＝14 片。
- 拿小例子验算：要搭 16KB 的 RAM，用 8K×4 位芯片：先并联 2 片成 8KB 一组，再串联 2 组，总共 4 片；规律一样——先拼位宽，再拼容量。
- 边界情况：如果芯片是 8K×8 位的，位宽正好，不用并联，56KB÷8KB＝7 片；如果地址是按字编址，算法就变了；本题按字节编址，1 个地址就是 1 个字节。

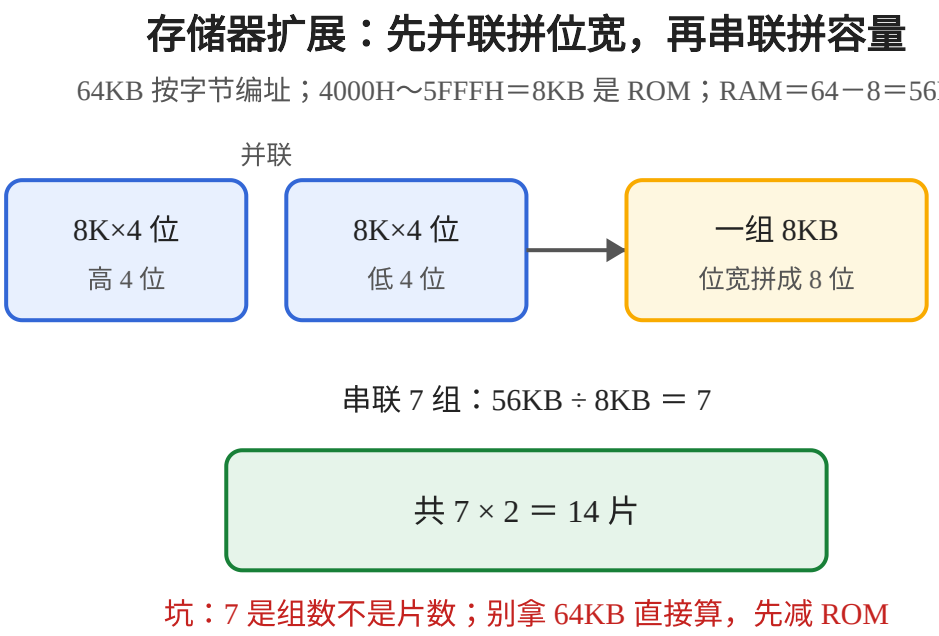
正确选项为什么对： RAM 56KB，一组 8KB 要 2 片，7 组共 14 片，就是 C，选 C。

错误选项为什么错＋举例：

- A（7）：7 是组数，不是片数；每组要 2 片并联拼位宽，漏了这一步。教训：先拼位宽（并联），再拼容量（串联），两步都算。
- B（8）：8 是 ROM 区的 KB 数，或者是一组的容量；和芯片数没关系。教训：别把容量数直接当片数。
- D（16）：16 是把 64KB 全当成 RAM 再算（64÷8×2＝16），没减去 ROM 的 8KB。教训：先减 ROM，再算 RAM 的芯片数。

笔记本一句话： 第 16 题＋存储器扩展：先算 RAM＝64－8＝56KB；位并联 2 片成字节、串联 7 组，7×2＝14 片。

【图像】



存储器扩展示意图：2 片 8K×4 位芯片并联拼成 8KB 一组，7 组串联拼出 56KB 的 RAM 区

【定位】 计算机组成原理 · 第3章存储器层次结构 · 存储器扩展（位扩展/字扩展）

【易混对比】

- "位扩展"和"字扩展"是两回事：位扩展是并联拼位宽（4 位→8 位），字扩展是串联拼容量（8KB→56KB）——两步都要算。
- "组数"和"片数"是两回事：7 是组数，每组 2 片，片数是 $7 \times 2 = 14$——算完组数别忘了乘并联片数。
- "总容量"和"RAM 区容量"是两回事：64KB 里要先减掉 ROM 的 8KB，剩 56KB 才是 RAM——别拿 64KB 直接算。

ROM 区 4000H～5FFFH 共 8KB，RAM 区 56KB；2 片 8K×4 位并联成 8KB 一组，56÷8＝7 组，共 14 片，选 C。

【坑位】

算出 7 组就直接选 7，漏了每组要 2 片并联拼位宽。

8K×4 位的芯片只有 4 位宽，一个字节要 8 位，必须 2 片并联（一片管高 4 位、一片管低 4 位）。正确做法：先并联拼位宽（得组容量），再串联拼容量（得组数），最后组数×每组片数。

【套路】

- 第一步，算分区：地址范围算 ROM 大小，总容量减 ROM 得 RAM 大小。
- 第二步，拼位宽：芯片位宽→目标位宽，并联片数＝目标位宽÷芯片位宽，得一组容量。
- 第三步，拼容量：组数＝RAM 大小÷一组容量。
- 第四步，总片数：组数×每组片数。

存储器扩展：先分区，再并联拼位宽，后串联拼容量，组数乘片数。

【类比】

用半块砖砌墙：先 2 块半砖并排拼成一块整砖的宽度（位扩展），再把拼好的整砖一块一块往上摞到够高度（字扩展）。问一共用了多少半块砖，摞的层数要乘以每层 2 块。

【口诀】 先减 ROM 得 RAM，并联拼宽串联扩，组数乘以每组片。

【考频】 「存储器扩展」在 2009–2022 年解析中出现 4 次 · 中频

第 17 题 · 给定指令格式下先变址后间址寻址的操作数有效地址（正确答案：C）

题干与选项：

某指令格式如下所示。其中 M 为寻址方式，I 为变址寄存器编号，D 为形式地址。若采用先变址后间址的寻址方式，则操作数的有效地址是（ ）。

- A. I+D
- B. (I)+D
- C. ((I)+D)
- D. ((I))+D

注：原卷此处为指令格式示意图（OP、M、I、D 四个字段），插图以原卷为准。

入手点： 按顺序做两步：第一步变址——变址寄存器的内容加上形式地址，得到一个中间地址；第二步间址——把这个中间地址当成地址，再取它那个单元里存的内容，那个内容才是真正的有效地址；括号套了两层，就是答案。

专有名词从零解释：

- 先想“形式地址 D”：指令里直接写着的地址数；它不一定是真正的地址，可能还要再加工。
- 再想“变址寄存器 I”：一个专门用来加偏移的寄存器；它的内容记成（I)；变址寻址就是把（I）和形式地址 D 加起来。
- 再想“变址寻址”：有效地址＝（I）＋D；适合按顺序访问数组——变址寄存器的值每次加 1，就指向下一个元素。
- 再想“间址寻址”：指令给的地址里存的不是操作数，而是操作数的真正地址；要多访存一次，把那个地址取出来才算完。
- 再想“括号的含义”：括号表示“这个地址的存储单元里的内容”；(I) 是变址寄存器的内容，((I)＋D) 是地址为（I）＋D 的单元里的内容。

考查知识点：

- 先做变址：变址寄存器的内容是（I)，加上形式地址 D，得到中间地址（I）＋D；这一步还没完，因为题目说“后间址”。
- 再做间址：把（I）＋D 当成地址，去访问这个地址的存储单元，取出来的内容就是操作数的有效地址，写成（(I)＋D)。
- 再想为什么括号是两层：外层括号表示“取单元内容”（间址那一步)，内层（I）表示“变址寄存器的内容”（变址那一步)；两步都做，括号就是两层。
- 拿小例子验算：变址寄存器内容是 100，形式地址是 20；变址后是 120；120 号单元里存着 500；先变址后间址的有效地址就是 500；如果只变址不间址，答案是 120，不对。
- 边界情况：如果只间址不间址（纯变址)，答案是（I）＋D；如果只间址不变址，答案是（D)；如果先间址后变址，答案是（D）＋（I)，和本题的顺序不一样，括号位置也不同。
- 易混对子：先变址后间址和先间址后变址是两回事；前者是（(I)＋D)，后者是（D）＋（I)；题目说“先变址、后间址”，顺序不能反。

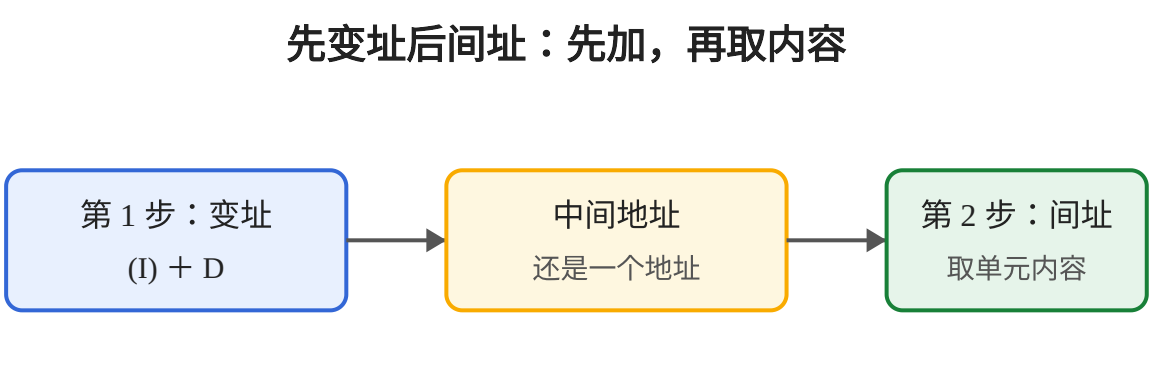
正确选项为什么对： 先变址得（I）＋D，再间址取该单元内容得（(I)＋D)，就是 C，选 C。

错误选项为什么错＋举例：

- A（I＋D）：这是把变址寄存器编号 I 本人（不是它的内容）和 D 相加；编号是“第几个寄存器”，内容才是地址偏移；用编号算，地址全错。教训：变址用的是寄存器的内容（I），不是编号 I。
- B（(I)＋D）：这只是变址的结果，没做间址那一步；题目要求先变址“后间址”，少了一步。教训：两步都做完，括号要套两层。
- D（((I))＋D）：这是先对变址寄存器内容再间址一次、再加 D，顺序是“先间址、后变址”那一路的写法；和题目“先变址、后间址”的顺序反了。教训：按题目顺序做，先加后取内容，和先取内容后加不是一回事。

笔记本一句话： 第 17 题＋先变址后间址：先加得（I）＋D，再取内容套括号＝（(I)＋D）。

【图像】



先变址后间址示意图：变址寄存器内容加形式地址得中间地址，再取该单元内容，括号套两层

【定位】 计算机组成原理 · 第4章指令系统 · 寻址方式（变址/间址）

【易混对比】

- "先变址后间址"和"先间址后变址"是两回事：前者是 $((I)+D)$，后者是 $(D)+(I)$——顺序一反，括号位置全变。
- "(I)"和"I"是两回事：(I) 是变址寄存器的内容（地址偏移），I 只是"第几个寄存器"的编号——变址加的是内容，不是编号。
- "变址"和"间址"是两回事：变址是"加"（内容＋形式地址），间址是"取"（按地址取单元内容）——一个做加法，一个做访存。

先变址得中间地址 (I)＋D，再间址取该单元内容，括号套两层得 ((I)＋D)，选 C。

【坑位】

只做到变址那一步就停，答案写成 (I)＋D，漏了后间址。

题目说"先变址、后间址"，两步都要做：变址是加法，间址是再访存一次取内容。正确做法：按题目顺序一步一步来，每做完一步问自己"后面还有没有步骤"。

【套路】

- 第一步，做变址：变址寄存器内容 (I)＋形式地址 D，得中间地址。
- 第二步，做间址：把中间地址当地址，取该单元内容，外面再套一层括号。
- 第三步，查顺序：先加后取是 ((I)＋D)，先取后加是 (D)＋(I)，对上题目顺序。

复合寻址：按题目顺序一步一步做，加法和取内容别漏、别反。

【类比】

找人先查通讯录再按门牌找：变址像"通讯录上写的楼号（I 的内容）加上门牌号 D"，得到"3 栋 502"这个中间地址；间址像"按这个地址找到 502 室，开门见到的人"才是你要找的人。两步都走完，人才算找到。

【口诀】 先加后取套两层，内容编号分得清；顺序一反括号变，对上题意再落笔。

【考频】 「变址间址寻址」在 2009–2022 年解析中出现 5 次 · 中频

第 18 题 · 4GB 主存 32 位字长指令按边界对齐存放时 PC 和 IR 的最少位数（正确答案：B）

题干与选项：

某计算机主存空间为 4GB，字长为 32 位，按字节编址，采用 32 位定长指令字格式。若指令按字边界对齐存放，则程序计数器（PC）和指令寄存器（IR）的位数至少分别是（）。

- A. 30、30
- B. 30、32
- C. 32、30
- D. 32、32

入手点： PC 是“下一条指令的地址”，IR 是“装整条指令的寄存器”；地址要几位，看主存有多大、指令怎么对齐；指令要几位，看指令本身有多长；两件事分开算。

专有名词从零解释：

- 先想“按字节编址”：一个地址对应 1 个字节；4GB 的主存一共有 4×2 的 30 次方个字节，需要 32 位地址才能全指到。
- 再想“字长 32 位”：一个字是 4 个字节；指令是定长的，一个字就是一条指令。
- 再想“按字边界对齐”：指令只能放在地址是 4 的倍数的地方（0、4、8、12……）；指令地址的低 2 位永远是 0。
- 再想“PC”：程序计数器，存着下一条要取的指令的地址；每次取完指令，PC 就指向下一条。
- 再想“IR”：指令寄存器，取回来的指令先存在这里，再送去译码执行；要装得下整条指令。

考查知识点：

- 先算 PC 要几位：主存 4GB，按字节编址要 32 位地址；但指令按字对齐，地址低 2 位永远是 0，不用存；PC 只需要存高 30 位（字地址），取指令时后面补 2 个 0 就还原了；所以 PC 最少 30 位。
- 再算 IR 要几位：IR 要装整条指令，指令是 32 位定长字，所以 IR 最少 32 位；少 1 位都装不下。
- 再想为什么 PC 能省 2 位：因为对齐保证了低 2 位是 0，省掉它们不丢信息；如果指令不对齐存放（比如有的指令 2 个字节、有的 4 个字节），PC 就省不了，必须 32 位。
- 拿小例子验算：主存 16B，按字节编址要 4 位地址；指令 2 字节对齐存放，地址只能是 0、2、4……14，低 1 位永远是 0，PC 只要 3 位；IR 还是指令的完整长度。
- 边界情况：PC 的位数问的是“最少”，所以对齐省掉的位要省掉；IR 的位数问的也是“最少”，但指令长度是硬的，省不掉。

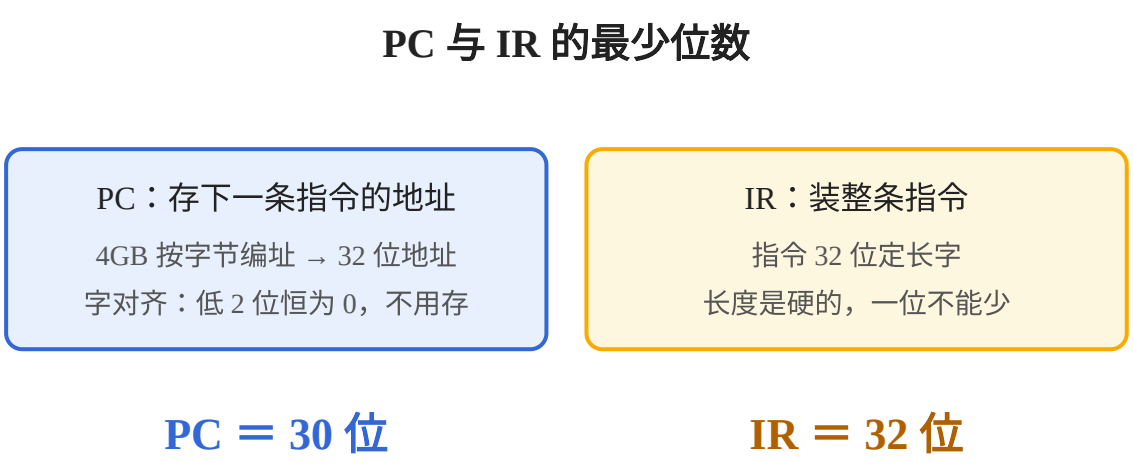
正确选项为什么对： PC 存字地址、最少 30 位，IR 装整条指令、最少 32 位，就是 B，选 B。

错误选项为什么错＋举例：

- A（30、30）：IR 也写成 30 位，装不下 32 位的指令；少 2 位，指令的低 2 位就丢了。教训：IR 装的是整条指令，长度是硬的。
- C（32、30）：PC 写成 32 位，没利用对齐省掉的 2 位；题目问“最少”，32 位不是最少。教训：对齐存放时，PC 的低位是 0，不用存。
- D（32、32）：PC 写成 32 位，同样没省掉对齐的 2 位；虽然够用，但不是最少。教训：问“最少”时，能省的位要省掉。

笔记本一句话： 第 18 题＋PC/IR 位数：4GB 字节地址 32 位，字对齐省 2 位→PC 30 位；IR 装整指令 32 位。

【图像】



PC 与 IR 位数示意图：4GB 字节地址 32 位，字对齐低 2 位恒为 0，PC 存字地址只要 30 位

【定位】 计算机组成原理 · 第5章中央处理器 · PC 与 IR 的位数

【易混对比】

- "PC"和"IR"是两回事：PC 存下一条指令的地址（能省位），IR 装整条指令（长度是硬的，一位不能少）。
- "字节地址"和"字地址"是两回事：4GB 按字节编址要 32 位；按字对齐后低 2 位恒为 0，字地址只要 30 位。
- "最少"和"够用"是两回事：PC 用 32 位也够用，但题目问最少，对齐省掉的 2 位必须省。

4GB 字节地址 32 位，指令按字对齐低 2 位恒为 0，PC 存字地址最少 30 位；IR 装整条 32 位指令最少 32 位，选 B。

【坑位】

PC 直接写 32 位，忘了"按字边界对齐"能省 2 位。

对齐意味着指令地址永远是 4 的倍数，低 2 位永远是 0，不用存；取指令时后面补 2 个 0 就还原了。正确做法：看到"对齐"先想"低几位恒为 0"，能省就省；题目问"最少"，省掉的位必须省。

【套路】

- 第一步，算地址总位数：容量按编址单位换算，$4GB = 2^{32}B$→32 位。
- 第二步，看对齐省位：字长 32 位＝4B，对齐后低 2 位恒 0，PC＝32－2＝30 位。
- 第三步，看指令长度：IR 装整条指令，指令多长 IR 就多少位，32 位。

PC/IR 位数：先算总地址位，对齐省低位，IR 按指令长度硬取。

【类比】

小区有 32 栋楼、每栋 4 层，门牌号本来要写"几栋几层"。如果规定只住整栋（不对散户），"几层"永远是 0，门牌只写栋号就行，省 2 位。但你要搬的家具（指令）有多大，货车（IR）就得有多大，一寸不能少。

【口诀】 对齐低位恒为零，PC 省位存字址；IR 装下整指令，长度是硬不能省。

【考频】 「PC IR 位数」在 2009–2022 年解析中出现 17 次 · 高频

第 19 题 · 无转发五段基本流水线中存在数据冒险的指令对（正确答案：B）

题干与选项：

在无转发机制的五段基本流水线（取指、译码/读寄存器、运算、访存、写回寄存器）中，下列指令序列存在数据冒险的指令对是（ ）。

I1：add R1,R2,R3；(R2)+(R3)→R1

I2：add R5,R2,R4；(R2)+(R4)→R5

I3：add R4,R5,R3；(R5)+(R3)→R4

I4：add R5,R2,R6；(R2)+(R6)→R5

- A. I1 和 I2
- B. I2 和 I3
- C. I2 和 I4
- D. I3 和 I4

入手点： 数据冒险就是“后一条指令要读的寄存器，前一条指令还没写回去”；把四条指令的目的寄存器和源寄存器列出来，看哪对是“前面写、后面读同一个寄存器”，而且时间上撞在一起。

专有名词从零解释：

- 先想“五段流水线”：一条指令分五步走——取指（IF）、译码/读寄存器（ID）、运算（EX）、访存（MEM）、写回寄存器（WB）；五条指令可以同时在不同的段里走，像流水线上的工人。
- 再想“数据冒险”：后一条指令在 ID 段读寄存器时，前一条指令还没走到 WB 段写寄存器；后一条读到的就是旧值，算错了。
- 再想“转发机制”：把刚算出来的结果直接从后面的段送回前面的段，不用等到写回；有了转发，很多数据冒险就不用停顿了；本题没有转发，所以冒险会实实在在地造成停顿或错误。
- 再想“RAW、WAW”：RAW 是“写后读”（前面写、后面读），会造成数据冒险；WAW 是“写后写”（两条都写同一个寄存器），在按序写回的流水线里不会造成停顿，不算本题要找的数据冒险。

考查知识点：

- 先列四条指令：I1 写 R1、读 R2 和 R3；I2 写 R5、读 R2 和 R4；I3 写 R4、读 R5 和 R3；I4 写 R5、读 R2 和 R6。
- 再看 I1 和 I2：I1 写 R1，I2 读 R2 和 R4；没有同一个寄存器，不冒险。
- 再看 I2 和 I3：I2 写 R5，I3 读 R5；I3 在 ID 段读 R5 时，I2 还在后面（EX、MEM 段），R5 还没写回；这就是 RAW 数据冒险。
- 再看 I2 和 I4：I2 写 R5，I4 也写 R5；两个都是写，没有“读旧值”的问题；在按序写回的流水线里，后写的会覆盖先写的，结果是对的，不算数据冒险。
- 再看 I3 和 I4：I3 写 R4，I4 读 R2 和 R6；没有同一个寄存器，不冒险。
- 拿生活例子验算：I2 是“把算好的数写进 5 号柜子”，I3 是“打开 5 号柜子拿数”；I2 还没走到柜子跟前，I3 就打开了柜子，拿到的是旧数；这就是冒险。
- 边界情况：如果流水线有转发，I3 在 EX 段可以直接拿到 I2 刚算出的结果，不用停顿；但本题没有转发，所以 I2 和 I3 的冒险是实实在在的。

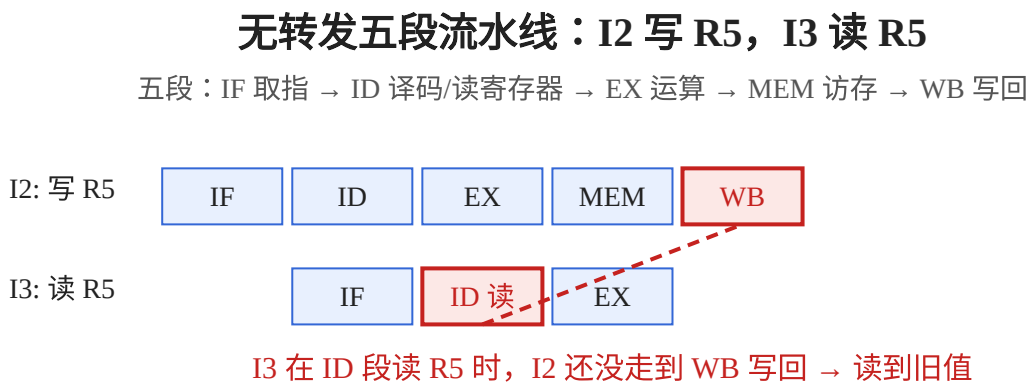
正确选项为什么对： 只有 I2 写 R5、I3 读 R5 这一对是“前面写、后面读同一个寄存器”，存在 RAW 数据冒险，就是 B，选 B。

错误选项为什么错＋举例：

- A（I1 和 I2）：I1 写 R1，I2 读 R2、R4，没有公共寄存器；读和写碰不到一起。教训：先列目的寄存器和源寄存器，再找公共的。
- C（I2 和 I4）：两个都是写 R5，是写后写（WAW），不是写后读；在按序写回的流水线里不会读错数。教训：WAW 不算本题的数据冒险，别把“同一个寄存器”都当成冒险。
- D（I3 和 I4）：I3 写 R4，I4 读 R2、R6，没有公共寄存器。教训：只看寄存器号对不对得上，对不上就不冒险。

笔记本一句话： 第 19 题＋流水线数据冒险：前面写、后面读同一个寄存器＝RAW；I2 写 R5、I3 读 R5。

【图像】



流水线数据冒险示意图：I2 写 R5 的结果还没走到写回段，I3 已在译码段读 R5，读到旧值

【定位】 计算机组成原理 · 第5章中央处理器 · 流水线数据冒险

【易混对比】

- "RAW"和"WAW"是两回事：RAW 是前面写、后面读（会读到旧值，真冒险）；WAW 是两条都写（按序写回不读错，不算本题的冒险）。
- "有转发"和"无转发"是两回事：有转发时结果能提前送回去，冒险可化解；本题无转发，冒险是实实在在的。
- "同一个寄存器"和"数据冒险"是两回事：只看寄存器号相同不够，还得是"前面写、后面读"这个方向。

四条指令里只有 I2 写 R5、I3 读 R5 是"前面写、后面读同一个寄存器"，I3 读时 I2 还没写回，存在 RAW 数据冒险，选 B。

【坑位】

看到 I2 和 I4 都写 R5，就当成数据冒险。

两个都是写，没有"读旧值"的问题；在按序写回的流水线里，后写的覆盖先写的，结果是对的。正确做法：先列每条指令的目的寄存器和源寄存器，只找"前面写、后面读"的那一对。

【套路】

- 第一步，列表：每条指令写哪个寄存器（目的）、读哪几个（源）。
- 第二步，找写后读：前面指令的目的寄存器＝后面指令的源寄存器。
- 第三步，看时序：后面指令读寄存器时，前面指令走到写回段没有；没到就是冒险。
- 第四步，看转发：无转发则冒险成立，有转发则可能化解。

数据冒险：列表找写后读，再看时序撞没撞，有无转发最后定。

【类比】

I2 是"把算好的数写进 5 号柜子"的人，还在去柜子的路上；I3 是"打开 5 号柜子拿数"的人，已经走到了柜子跟前。I3 打开柜子拿到的是旧数——这就是数据冒险：写的人还没到，读的人先动手了。

【口诀】 前写后读同一号，无转发来真撞车；写写相碰不算数，列表时序两步走。

【考频】 「流水线数据冒险」在 2009–2022 年解析中出现 15 次 · 高频

第 20 题 · 单周期处理器相关叙述中错误的一项（正确答案：A）

题干与选项：

单周期处理器中所有指令的指令周期为一个时钟周期。下列关于单周期处理器的叙述中，错误的是（）。

- A. 可以采用单总线结构数据通路
- B. 处理器时钟频率较低
- C. 在指令执行过程中控制信号不变
- D. 每条指令的 CPI 为 1

入手点： 单周期处理器的关键是“一个周期干完一条指令的所有事”；取指令要访存、取操作数也可能要访存；一个周期里要访存两次，一条单总线做不到；先把这个矛盾找出来。

专有名词从零解释：

- 先想“单周期处理器”：每条指令从取指到写回，全部在一个时钟周期里完成；指令周期就是一个时钟周期。
- 再想“单总线结构数据通路”：所有部件共用一条总线传数据；一个时刻只能有一个部件往总线上送数据。
- 再想“时钟频率”：一秒钟有多少个时钟周期；周期要留给最慢的那条指令，所以单周期处理器的时钟频率由最慢的指令决定，比较低。
- 再想“控制信号”：告诉各个部件这一步该干什么的信号；单周期里，一条指令的控制信号在整个周期里不变。

考查知识点：

- 先想 A 为什么错：单周期里，一条指令既要取指令（访存一次），又可能要取操作数（访存第二次，比如 load 指令）；单总线一个时刻只能传一路数据，一个周期里传不了两次；所以单周期处理器不能用单总线结构，必须把指令存储器和数据存储器分开（哈佛结构那种分法），或者用多总线。
- 再想 B 为什么对：一个周期要干完最慢的那条指令（比如访存的 load 指令），周期必须按最慢的定，所以时钟频率比较低。
- 再想 C 为什么对：单周期里，一条指令的控制信号从周期开始到结束都不变；不像多周期那样每个阶段换一套信号。
- 再想 D 为什么对：每条指令都是一个周期完成，所以每条指令的 CPI（每条指令的时钟周期数）都是 1。
- 拿生活例子验算：单总线像一条单车道，一次只能过一辆车；单周期指令要在一秒内既进货（取指令）又出货（取操作数），单车道一秒过不了两辆车；所以得修两条道（分开的存储器）。
- 边界情况：如果指令不需要访存取操作数（比如纯寄存器运算），单总线够不够？还是不够——因为取指令本身就要访存，而单周期要求所有指令都在一个周期完成，通路必须按最坏情况（需要两次访存的指令）设计。

正确选项为什么对： 单周期一个周期里要访存两次，单总线做不到，所以“可以采用单总线结构”是错的，本题问的就是错误项，选 A。

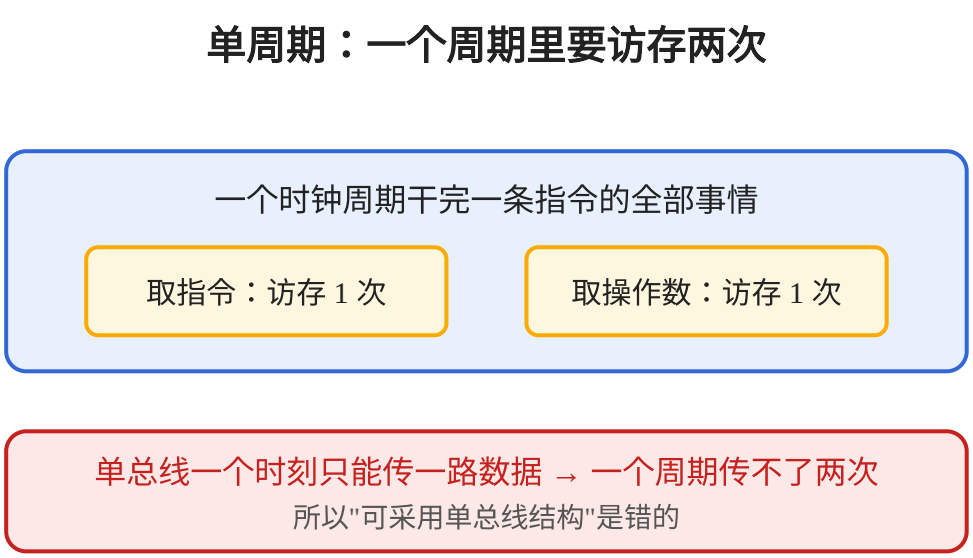
错误选项为什么错＋举例：

- B（处理器时钟频率较低）：这是对的；周期按最慢指令定，频率自然低；不能选它，因为题目问的是错误项。教训：问“错误的是”，对的先排除。
- C（在指令执行过程中控制信号不变）：这是对的；单周期里一套信号用到底，不像多周期那样分阶段换信号。教训：单周期和多周期的控制信号特点别记反。

- D（每条指令的 CPI 为 1）：这是对的；单周期的定义就是每条指令一个周期，CPI 恒为 1。教训：CPI 为 1 是单周期的定义，不是优点也不是缺点。

笔记本一句话： 第 20 题＋单周期：一个周期要访存两次，单总线不够；时钟低、信号不变、CPI 为 1 都是对的。

【图像】



对的三条：周期按最慢指令定→频率低；控制信号一套用到底→不变；每条指令 1 周期→CPI 为 1

单周期访存示意图：一个时钟周期内既要取指令访存一次，又要取操作数访存一次，单总线一次只能传一路

【定位】 计算机组成原理 · 第5章中央处理器 · 单周期数据通路

【易混对比】

- "单周期"和"多周期"是两回事：单周期一条指令一个周期、控制信号不变；多周期一条指令多个周期、每阶段换信号——信号变不变是区分标志。
- "取指令访存"和"取操作数访存"是两回事：单周期里两者挤在一个周期里，共要访存两次——数访存次数时别漏了取指令那一次。
- "单总线"和"分开存储器"是两回事：单总线一个时刻只能传一路数据，一个周期传不了两次——这是 A 错的根源。

单周期一个周期里要访存两次（取指令＋取操作数），单总线一个时刻只能传一路数据，做不到，所以"可采用单总线结构"是错的；本题问错误项，选 A。

【坑位】

数访存次数时只数了取操作数，漏了取指令也要访存。

很多人想"load 指令才访存"，忘了每条指令开头都要取指令、本身就是一次访存。正确做法：单周期里先数"取指令"这一次，再数指令本身要不要访存，两次一到，单总线就不够。

【套路】

- 第一步，数访存：取指令 1 次＋取操作数（load/store 类）1 次＝2 次。
- 第二步，看总线：单总线一个时刻传一路，一个周期传不了两次→单总线不够。

- 第三步，验其余：周期按最慢指令定→频率低（对）；一套信号用到底→不变（对）；每条指令一个周期→CPI 为 1（对）。

单周期判错：先数一个周期访存几次，两次则单总线必错，其余逐条验。

【类比】

单总线像一条单车道，一次只能过一辆车。单周期指令要求在一秒内既进货（取指令）又出货（取操作数），单车道一秒过不了两辆车——得修两条道（指令存储器和数据存储器分开）才行。

【口诀】 单周期里访存俩，单总线不够俩；频率低来信号定，每条指令 CPI 一。

【考频】 「单周期处理器」在 2009–2022 年解析中出现 10 次 · 高频

第 21 题 · 总线设计相关叙述中错误的一项（正确答案：A）

题干与选项：

下列关于总线设计的叙述中，错误的是（）。

- A. 并行总线传输比串行总线传输速度快
- B. 采用信号线复用技术可减少信号线数量
- C. 采用突发传输方式可提高总线数据传输率
- D. 采用分离事务通信方式可提高总线利用率

入手点： “并行比串行快”是老印象，但不一定对；并行总线线多了会互相干扰、时钟上不去，高速串行总线反而更快；先把这条最可疑的挑出来，再验证其他三条。

专有名词从零解释：

- 先想“并行总线”：好几根线同时传好几位数据；比如 32 根线一次传 32 位。
- 再想“串行总线”：一根线一次传 1 位，数据排着队传；比如 USB、PCIe 用的就是串行。
- 再想“信号线复用”：同一根线在不同时刻传不同的信号（比如地址和数据轮流用一根线），省线，但要分时。
- 再想“突发传输”：一次申请、连续传一批数据，不用每个数据都重新申请总线；省了申请的开销，传得快。
- 再想“分离事务”：总线事务分成“请求”和“回答”两段，中间总线空出来给别人用；提高了总线利用率。

考查知识点：

- 先想 A 为什么错：并行总线线多，线和线之间会互相干扰（串扰），而且多根线的信号到达时间不一致（时钟偏移），频率上不去；串行总线只有一根线（或一对差分线），频率可以提得很高；高速串行总线（比如 PCIe）的实际速率可以超过低速并行总线；所以“并行总线传输比串行总线传输速度快”这个绝对的说法是错的。
- 再想 B 为什么对：信号线复用就是让地址线和数据线轮流用同一组线，线的数量少了，成本低了；代价是传一次要分两个时刻。
- 再想 C 为什么对：突发传输一次申请、连续传一批，不用每个字都申请总线，省了总线仲裁和申请的开销，数据传输率提高了。
- 再想 D 为什么对：分离事务把“发请求”和“等回答”分开，等待回答的时间里总线可以给别的设备用，总线利用率提高了。
- 拿生活例子验算：并行总线像 32 条车道的大路，但车道多了容易互相别车、限速低；串行总线像一条不限速的单车道，车速极快，一辆接一辆地跑，总运量反而可能更大。
- 边界情况：低速时并行确实比串行快（一次传 32 位 vs 一次传 1 位）；但题目说的是绝对的“并行比串行快”，高速串行反超的例子一出现，这个绝对说法就错了。

正确选项为什么对： 高速串行总线可以比低速并行总线快，所以“并行总线传输比串行总线传输速度快”这个绝对说法是错的，本题问的就是错误项，选 A。

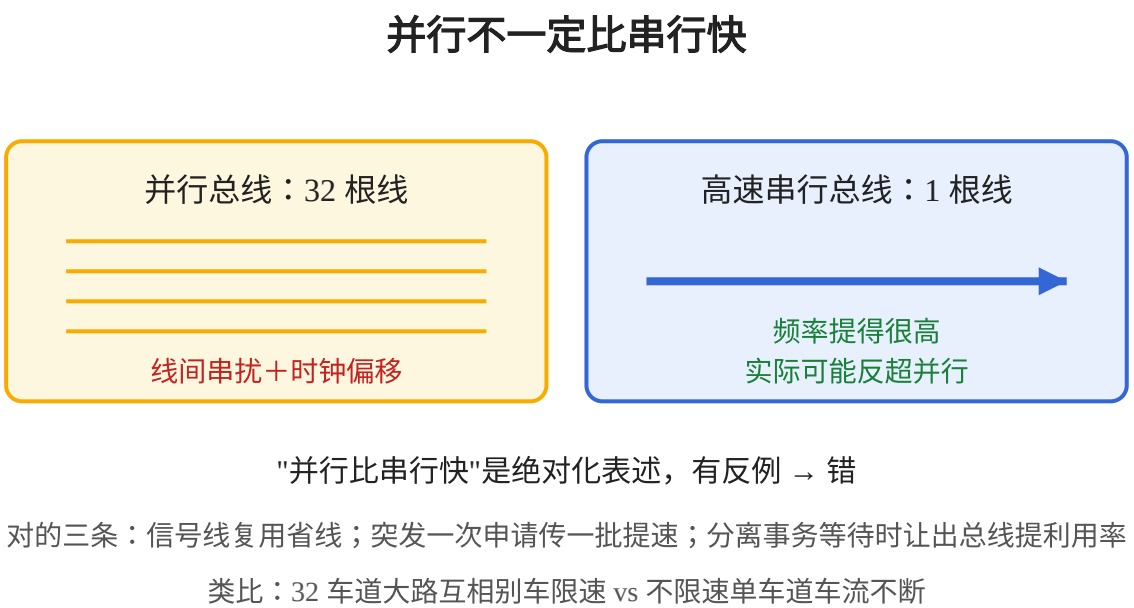
错误选项为什么错＋举例：

- B（采用信号线复用技术可减少信号线数量）：这是对的；复用就是一根线多用，线数自然少了。教训：复用省线，但要分时，速度会打折扣。
- C（采用突发传输方式可提高总线数据传输率）：这是对的；一次申请传一批，省了重复申请的开销。教训：突发是“批着传”，不是“一个一个申请着传”。

- D（采用分离事务通信方式可提高总线利用率）：这是对的；等待回答时总线不闲着，给别人用。教训：分离事务省的是等待时间里的总线空闲。

笔记本一句话： 第 21 题＋总线：并行不一定比串行快（高速串行反超）；复用省线、突发提速、分离事务提利用率都是对的。

【图像】



总线对比示意图：并行总线多根线互相干扰限速，高速串行总线单线频率更高、实际可能更快

【定位】 计算机组成原理 · 第6章总线 · 总线特性与性能优化

【易混对比】

- "并行"和"串行"是两回事：并行一次传多位但线间干扰大、频率上不去；串行一次传 1 位但频率能提很高——"一次传得多"不等于"单位时间传得多"。
- "低速时"和"高速时"是两回事：低速下并行确实更快；但题目说的是无条件的"并行比串行快"，高速串行一反超，这个绝对说法就错了。
- "复用/突发/分离事务"是三回事：复用省线（分时），突发提速（一次申请传一批），分离事务提利用率（等待时总线给别人用）——各管各的，别串。

"并行总线传输比串行总线传输速度快"是绝对化表述，高速串行总线可以反超低速并行总线，所以这条是错的；本题问错误项，选 A。

【坑位】

凭"并行一次传 32 位、串行一次传 1 位"的老印象，直接认定并行更快。

一次传得多不等于一秒传得多：并行线多了互相串扰、时钟偏移，频率提不上去；串行就一根（对）线，频率能跑很高。正确做法：看到"绝对化"的比较（"比…快"无条件），先找反例——高速串行就是反例。

【套路】

- 第一步，圈绝对化：找出"一定""都""比…快"这类无条件表述。

- 第二步，找反例：高速串行反超低速并行→A 的绝对说法不成立。
- 第三步，验其余：复用省线（对）、突发一次申请传一批提速（对）、分离事务等待时让出总线提利用率（对）。

总线判错：先抓绝对化表述找反例，其余逐条验真。

【类比】

并行总线像 32 条车道的大路，但车道一多就互相别车、只能限速；串行总线像一条不限速的单车道，车速极快、一辆接一辆，总运量反而可能更大。"车道多"不等于"运得多"。

【口诀】 并行串行别绝对，高速串行能反超；复用省线突发快，分离事务利用高。

【考频】 「总线」在 2009–2022 年解析中出现 25 次 · 高频

第 22 题 · 中断与异常分类相关叙述中错误的一项（正确答案：A）

题干与选项：

异常是指令执行过程中在处理器内部发生的特殊事件，中断是来自处理器外部的请求事件。下列关于中断或异常情况的叙述中，错误的是（）。

- A. "访存时缺页"属于中断
- B. "整数除以 0"属于异常
- C. "DMA 传送结束"属于中断
- D. "存储保护错"属于异常

入手点： 先分清“中断”和“异常”的界线：中断是处理器外面来的请求，异常是处理器里面执行指令时出的事；缺页是执行指令时在里面发生的，属于异常；说它属于中断，就是错的。

专有名词从零解释：

- 先想“中断”：来自处理器外部的请求事件；比如 I/O 设备传完了数据、定时器到时间了；CPU 正在执行的程序和它没关系，是外面打断进来的。
- 再想“异常”：指令执行过程中在处理器内部发生的特殊事件；比如除以 0、访存时缺页、存储保护错；是当前这条指令自己惹出来的。
- 再想“缺页”：访问的页面不在内存里；CPU 执行到访存指令时发现页表里没有这个页，要先把页面从外存调进来，才能继续执行这条指令。
- 再想“故障（fault）”：异常的一种；故障发生后，处理完了还能回到原来那条指令重新执行；缺页就是故障——页面调进来后，访存指令重新执行一遍。
- 再想“DMA 传送结束”：DMA 控制器帮 CPU 搬完了数据，搬完后向 CPU 发个信号；这是外设发来的，属于中断。

考查知识点：

- 先想 A 为什么错：“访存时缺页”是执行访存指令时在处理器内部发生的，属于异常（具体是故障），不是来自外部的中断；说它属于中断，分错了类。
- 再想 B 为什么对：“整数除以 0”是执行除法指令时在内部发生的，属于异常；除法指令自己惹出来的事，不是外面打断的。
- 再想 C 为什么对：“DMA 传送结束”是 DMA 控制器这个外设发来的信号，属于中断；CPU 正在跑的程序和它没关系。
- 再想 D 为什么对：“存储保护错”是访问了不该访问的存储区域，执行访存指令时在内部发生的，属于异常。
- 拿生活例子验算：中断像外面有人敲门（和屋里正在干的活没关系）；异常像屋里正在做的菜出了问题（盐放多了、锅烧干了），是手头这件事自己出的事；缺页就是“做菜做到一半发现缺一种料”，是手头的事，不是敲门。
- 边界情况：缺页处理完后，访存指令要重新执行一遍；这是故障的特点；中断处理完后，回来继续执行下一条指令，不重新执行被打断的那条；这个区别也能用来判断分类。

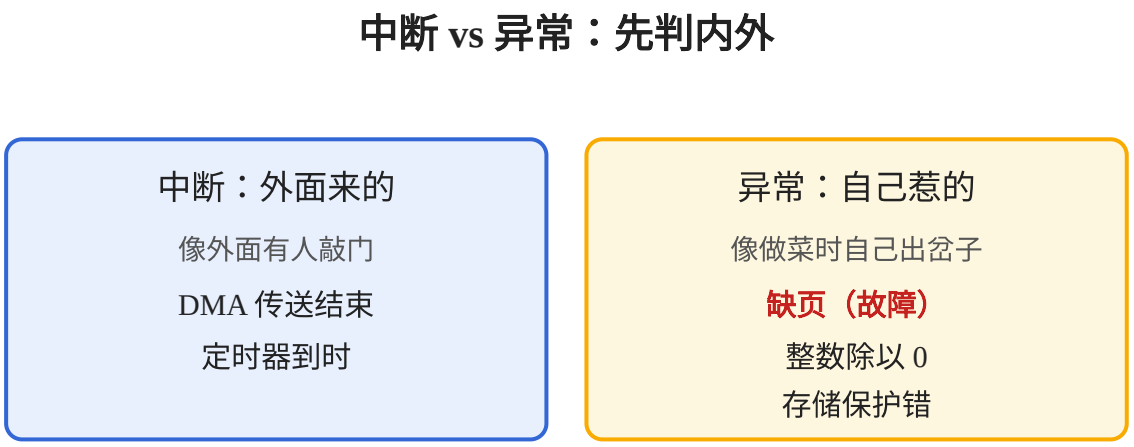
正确选项为什么对： 缺页是异常（故障），不是中断；说它属于中断是错的，本题问的就是错误项，选 A。

错误选项为什么错＋举例：

- B（“整数除以 0”属于异常）：这是对的；除法指令执行时内部出的事，就是异常。教训：指令自己惹出来的，都是异常。
- C（“DMA 传送结束”属于中断）：这是对的；外设搬完数据发来的信号，就是中断。教训：外设发来的，都是中断。
- D（“存储保护错”属于异常）：这是对的；访存越权是执行指令时内部出的事，就是异常。教训：保护错和缺页一样，都是访存时内部出的事。

笔记本一句话： 第 22 题＋中断 vs 异常：外面来的叫中断（DMA 结束），指令自己惹的叫异常（缺页、除 0、保护错）。

【图像】



中断与异常分类示意图：外面设备敲门进来的是中断，指令执行中自己惹出的是异常

【定位】 计算机组成原理 · 第7章输入/输出系统 · 中断与异常

【易混对比】

- "中断"和"异常"是两回事：中断是处理器外面来的（DMA 结束、定时器），异常是指令执行中里面出的事（缺页、除 0）——判内外，不判名字。
- "缺页"和"DMA 结束"是两回事：缺页是访存指令自己在内部撞上的（异常），DMA 结束是外设干完活发来的（中断）——别看都有"访存"两个字就归一类。
- "故障"和"中断"是两回事：故障是异常的一种，处理完要回到原指令重执行；中断处理完回来执行下一条，不重执行。

缺页是访存指令在处理器内部撞上的事，属于异常（故障），不是外面来的中断；说它属于中断是错的，本题问错误项，选 A。

【坑位】

看到"访存时缺页"里有"访存"，就把它和"DMA 传送结束"归成一类。

分类看的是"谁惹的事"：缺页是当前这条访存指令自己在执行中撞上的（内部），DMA 结束是 DMA 控制器这个外设发来的（外部）。正确做法：先问"这事是谁惹的"——外面来的叫中断，自己惹的叫异常。

【套路】

- 第一步，判内外：外面设备/事件来的→中断；指令执行中内部出的→异常。
- 第二步，逐条归类：缺页（内部→异常）、除 0（内部→异常）、DMA 结束（外部→中断）、保护错（内部→异常）。
- 第三步，找错类：把异常说成中断的那条就是错的。

中断异常分类：先判内外再归类，错类的就是答案。

【类比】

中断像外面有人敲门（和屋里正干的活没关系）；异常像屋里做菜时自己出了岔子（盐放多了、锅烧干了）。缺页就是"菜做到一半发现缺一种料"——手头这件事自己的问题，不是敲门。

【口诀】 外面敲门叫中断，自己惹事叫异常；缺页除零保护错，都是异常别叫错。

【考频】 「中断异常」在 2009–2022 年解析中出现 12 次 · 高频

第 23 题 · 批处理系统相关叙述中正确的一项（正确答案：A）

题干与选项：

下列关于批处理系统的叙述中，正确的是（）。

Ⅰ．批处理系统允许多个用户与计算机直接交互

Ⅱ．批处理系统分为单道批处理系统和多道批处理系统

Ⅲ．中断技术使得多道批处理系统的 I/O 设备可与 CPU 并行工作

- A. 仅Ⅱ、Ⅲ
- B. 仅Ⅱ
- C. 仅Ⅰ、Ⅱ
- D. 仅Ⅰ、Ⅲ

入手点： 先把三条一条一条判对错：Ⅰ说“批处理允许多个用户直接交互”，这是分时系统的特征，不是批处理的；Ⅱ说“批处理分单道和多道”，这是分类，没错；Ⅲ说“中断技术让 CPU 和 I/O 并行”，这是多道批处理的原理，没错；所以正确的是Ⅱ和Ⅲ。

专有名词从零解释：

- 先想“批处理系统”：用户把作业交给系统，系统一批一批地跑；用户不能在作业跑的时候插手、交互；作业排着队，跑完一个再跑下一个。
- 再想“单道批处理系统”：内存里一次只放一道作业；这道作业跑完，才放下一道；CPU 等 I/O 的时候就闲着。
- 再想“多道批处理系统”：内存里同时放好几道作业；一道作业等 I/O 时，CPU 切去跑另一道；CPU 不闲着，利用率高了。
- 再想“分时系统”：多个用户同时用一台机器，系统把 CPU 时间切成小片轮流给每个用户；每个用户感觉像独占一台机器，能直接交互。
- 再想“中断技术”：I/O 设备做完事后发个信号打断 CPU，CPU 收到信号就切去处理；没有中断，CPU 只能干等 I/O。

考查知识点：

- 先判第Ⅰ条：批处理系统不允许多个用户直接交互；用户交完作业就等结果，中间不能插手；“多个用户直接交互”是分时系统的特征；所以第Ⅰ条是错的。
- 再判第Ⅱ条：批处理系统确实分为单道批处理系统和多道批处理系统；单道一次一道，多道同时多道；分类没错。
- 再判第Ⅲ条：中断技术确实让多道批处理系统里 CPU 和 I/O 设备并行工作；一道作业等 I/O 时，I/O 设备在干活，CPU 切去跑另一道作业，两边同时干；没有中断，CPU 不知道 I/O 什么时候完，只能干等。
- 再想组合：对的是Ⅱ和Ⅲ，就是 A。
- 拿生活例子验算：批处理像把一摞文件交给文印室，交完就走、过会儿来取；不能站在复印机旁边一张一张改；分时像好几个人轮流用一台电脑，每人用一会儿、感觉像独占。
- 边界情况：单道批处理也有中断（比如 I/O 完成中断），但单道时 CPU 还是闲着等——因为内存里只有一道作业，没别的可切；中断让并行成为可能，多道才真正把并行用起来。

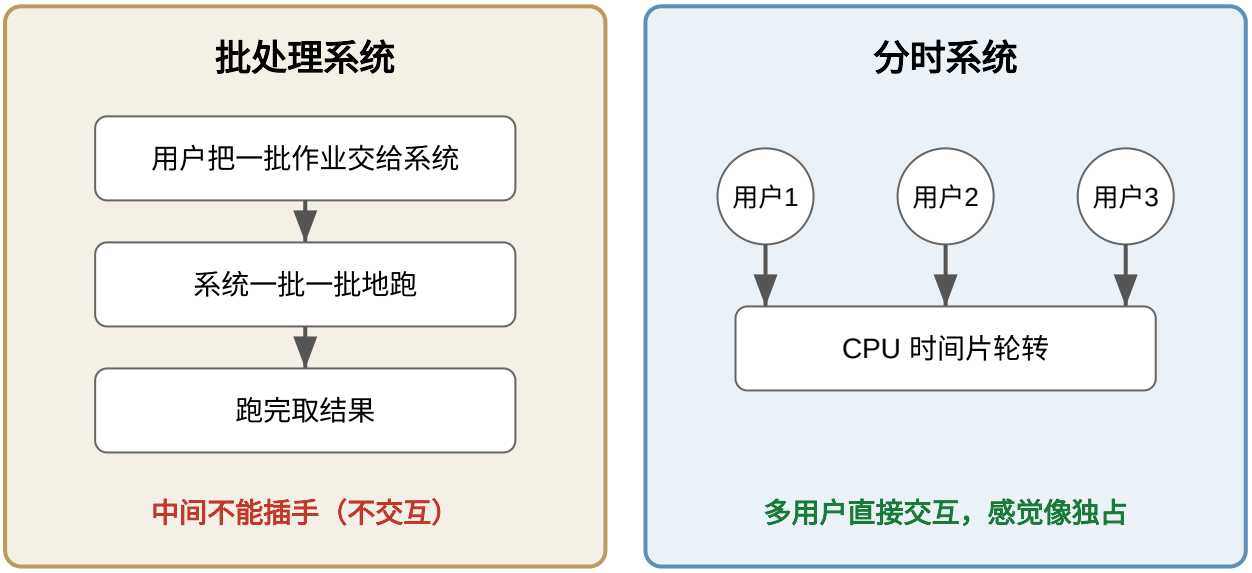
正确选项为什么对： Ⅰ错（交互是分时的特征），Ⅱ对（单道和多道是批处理的分类），Ⅲ对（中断让 CPU 和 I/O 并行）；正确组合是仅Ⅱ、Ⅲ，就是 A，选 A。

错误选项为什么错＋举例：

- B（仅Ⅰ）：Ⅰ是错的；批处理不交互，交互是分时系统的事；把分时的特征安到批处理头上，是错的。教训：先分清批处理和分时的界线。
- C（仅Ⅰ、Ⅱ）：Ⅰ是错的，不能带进来；Ⅱ是对的，但组合里混进错的就不行。教训：组合题要每一条都对才行，混进一条错的就全错。
- D（仅Ⅰ、Ⅱ、Ⅲ）：Ⅰ是错的，Ⅲ是对的但Ⅰ不能带；全选就是把错的也选了。教训：看到“全选”先怀疑，逐条判完再定。

笔记本一句话： 第 23 题＋批处理：不交互（交互是分时）；分单道和多道；中断让 CPU 和 I/O 并行；对的是Ⅱ、Ⅲ。

【图像】



批处理与分时对比示意图：左边批处理交完作业就走不能插手，右边分时多用户轮流用感觉像独占

【定位】 操作系统 · 第1章 · 计算机系统概述（批处理系统、分时系统）

【易混对比】

- "批处理"和"分时"是两回事：批处理是交完作业就走、中间不能插手；分时是多个用户同时在线、轮流用 CPU、感觉像独占——"直接交互"是分时的招牌，不是批处理的。
- "单道批处理"和"多道批处理"是两回事：单道是内存一次只放一道作业、CPU 等 I/O 时闲着；多道是内存放好几道、一道等 I/O 时 CPU 切去跑另一道——中断是多道把 CPU 和 I/O 并行起来的前提。
- "中断"和"时间片轮转"是两回事：中断是 I/O 完事发信号叫 CPU，是多道并行的开关；时间片轮转是分时系统轮流分 CPU 的办法——别把分时的手段安到批处理头上。

Ⅰ错在"直接交互"（那是分时的特征）；Ⅱ对（批处理确分单道和多道）；Ⅲ对（中断让多道批处理里 CPU 和 I/O 并行）；正确的是仅Ⅱ、Ⅲ，选 A。

【坑位】

组合题每条都要单独判对错，错一条就不能带进组合。

最容易踩的坑是：盯着 II 、III都对就急着选，却没发现 I 把分时的"直接交互"安到了批处理头上。为什么错：批处理的用户交完作业就等结果，中间不能插手；"多个用户直接交互"是分时系统的特征，张冠李戴了。正确做法：三条逐条判—— I 看关键词"交互"→这是分时的事→错； II 看"分类"→单道/多道→对；III看"中断→并行"→多道里 CPU 不等 I/O→对；最后对的是 II 、III，选 A。

【套路】

- 第一步，抓每条的关键词： I "直接交互"、 II "单道/多道"、III"中断→CPU 和 I/O 并行"。
- 第二步，逐条判对错：交互是分时的事→ I 错；批处理确分单道和多道→ II 对；中断让多道里 CPU 不等 I/O→III对。
- 第三步，组合对号入座：对的是 II 、III→找"仅 II 、III"的选项，就是 A。

叙述组合题：抓关键词，逐条判，最后组合对号。

【类比】

批处理像把一摞文件送到文印室：交完就走，过会儿来取成品，中间不能站在复印机旁边一张一张改——这叫"不交互"。分时像图书馆的公共电脑区：好几个人轮流用，每人感觉像自己独占一台——这叫"直接交互"。把"轮流用电脑"的特征说成文印室的，就是 I 的错误。

【口诀】 批处理，不交互；分单道，也多道；中断到，并行跑。

【考频】 「批处理系统分时系统」在 2009–2022 年解析中出现 4 次 · 中频

第 24 题 · 单 CPU 每作业输入 2ms 计算 3ms 输出 4ms 时 3 个并发作业的最少完成时间（正确答案：B）

题干与选项：

某单 CPU 系统中有输入和输出设备各 1 台，现有 3 个并发执行的作业，每个作业的输入、计算和输出时间均分别为 2ms、3ms 和 4ms，且都按输入、计算和输出的顺序执行，则执行完 3 个作业需要的时间最少是（）。

- A. 15ms
- B. 17ms
- C. 22ms
- D. 27ms

入手点： 三个设备（输入设备、CPU、输出设备）可以同时干不同的活，像流水线；每个作业在三个设备上依次走；关键是算每个设备什么时候空出来，下一个作业什么时候能接上；画一张时间表，从头排到尾。

专有名词从零解释：

- 先想“并发执行”：几个作业轮流用设备，看起来同时在跑；实际上每个设备同一时刻只干一个作业的活。
- 再想“流水线”：作业在不同设备之间流动；输入设备给作业 1 做输入时，CPU 可以给作业 2 做计算，输出设备可以给作业 3 做输出；三个设备同时开工，总时间比一个一个串行短。
- 再想“瓶颈设备”：三个设备里最慢的那个；本题输出设备每作业 4ms，是最慢的；总时间不会比瓶颈设备的工作量少。

考查知识点：

- 先排作业 1：输入 0～2ms，计算 2～5ms，输出 5～9ms；作业 1 在 9ms 时跑完。
- 再排作业 2：输入设备在 2ms 时空出来，作业 2 输入 2～4ms；CPU 在 5ms 时空出来，作业 2 计算 5～8ms；输出设备在 9ms 时空出来，作业 2 输出 9～13ms；作业 2 在 13ms 时跑完。
- 再排作业 3：输入设备在 4ms 时空出来，作业 3 输入 4～6ms；CPU 在 8ms 时空出来，作业 3 计算 8～11ms；输出设备在 13ms 时空出来，作业 3 输出 13～17ms；作业 3 在 17ms 时跑完。
- 再想为什么不能更快：输出设备是瓶颈，每作业 4ms，3 个作业要 12ms；第一个作业的输出最早从 5ms 开始（前面输入 2ms＋计算 3ms），所以输出设备 5～17ms 连轴转，总时间最少 17ms；输入和计算设备都有空闲，但输出设备决定了下限。
- 拿生活例子验算：三道菜，每道都要“洗 2 分钟、切 3 分钟、炒 4 分钟”，洗菜池、案板、灶各 1 个；第一道菜洗 0～2、切 2～5、炒 5～9；第二道菜洗 2～4、切 5～8、炒 9～13；第三道菜洗 4～6、切 8～11、炒 13～17；总共 17 分钟，灶（最慢的）从 5 分钟起就没停过。
- 边界情况：如果只有一个作业，总时间就是 2＋3＋4＝9ms；如果作业很多，总时间趋近于“第一个作业的输入＋计算＋（作业数×输出时间）”；本题 3 个作业，5＋3×4＝17ms，对得上。

正确选项为什么对： 按流水线排，三个作业分别在 9ms、13ms、17ms 跑完，总时间最少 17ms，就是 B，选 B。

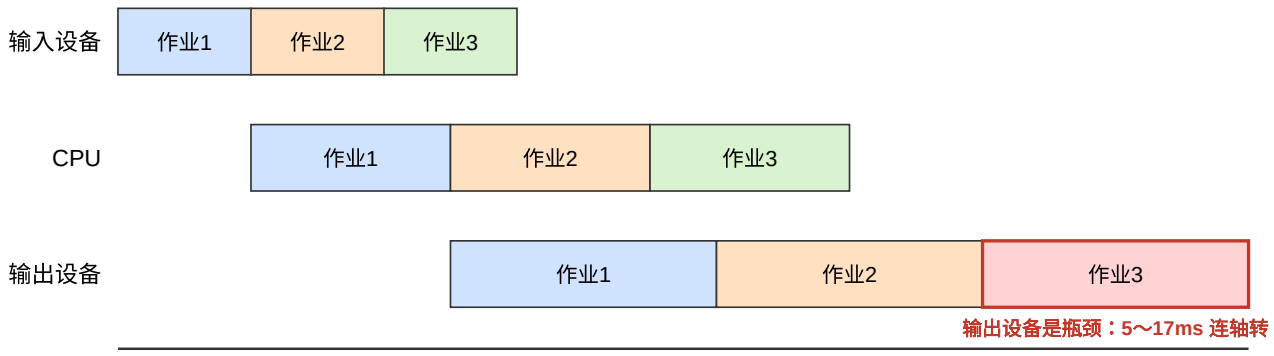
错误选项为什么错＋举例：

- A（15ms）：15ms 是把三个作业的时间简单加了再减了点，或者算错了设备空闲；按时间表排，输出设备 5～17ms 连轴转，15ms 时第三个作业还没输出完。教训：画时间表，别凭感觉加减。

- C（22ms）：22ms 是接近串行的时间（一个作业 9ms，三个作业串行是 27ms，22ms 是部分并行）；但输入、计算、输出三个设备可以同时开工，并行程度比这个高。教训：三个设备能同时干活，总时间比串行短得多。
- D（27ms）：27ms 是三个作业完全串行的时间（9ms×3）；并发执行时三个设备同时开工，不会这么慢。教训：并发不是串行，别把每个作业的时间直接乘 3。

笔记本一句话： 第 24 题＋三设备流水线：作业 1 在 9ms、作业 2 在 13ms、作业 3 在 17ms 跑完；瓶颈是输出设备。

【图像】



三设备流水线时间表：输入设备 CPU 输出设备三行，三个作业依次流动，17ms 完成

【定位】 操作系统 · 第1章 · 多道程序设计（并发作业的流水线执行）

【易混对比】

- "并发"和"串行"是两回事：并发是三个设备同时开工、作业在设备之间流动；串行是一道作业走完三个设备、下一道才开始——本题问并发，最少 17ms，不是串行的 27ms。
- "单个作业时间"和"三个作业总时间"是两回事：单个作业要 $2+3+4=9$ms；三个并发时设备重叠工作，总时间 17ms——不是 9ms，更不是 27ms。
- "瓶颈设备"和"最先空闲的设备"是两回事：输出设备每作业 4ms 最慢，它决定总时间下限；输入设备最先闲下来，但它说了不算。

作业 1 在 9ms 跑完，作业 2 在 13ms，作业 3 在 17ms；输出设备是瓶颈，5～17ms 连轴转；总时间最少 17ms，选 B。

【坑位】

别把三个作业的时间直接相加，画出时间表再定答案。

最容易踩的坑有两种：一是 $9\times3=27$ms，把并发当串行——三个设备能同时开工，不会这么慢；二是算出 15ms 之类的数，凭感觉加减、没画时间表——输出设备 5～17ms 连轴转，15ms 时第三个作业还没输出完。正确做法：从作业 1 开始画时间表，每个设备空出来才接下一个作业；再找瓶颈定下限：输出设备最慢，第一个作业的输出最早从 5ms 开始（$2+3=5$），3 个作业输出要 $3\times4=12$ms，总时间最少 $5+12=17$ms。

【套路】

- 第一步，列三个设备的时间：输入 2ms、计算 3ms、输出 4ms。

- 第二步，从作业 1 画时间表：作业 1 输入 0～2、计算 2～5、输出 5～9；后面的作业等设备空出来再接上。
- 第三步，找瓶颈定下限：输出设备最慢（4ms），第一个作业输出最早从 5ms 开始，3 个作业要 12ms，总时间至少 17ms。
- 第四步，排完核对：作业 2 输出 9～13、作业 3 输出 13～17，确认没有两个作业同时占一个设备。

三设备流水线：画时间表，瓶颈定下限，排完再核对。

【类比】

像做三道菜，每道都要"洗 2 分钟、切 3 分钟、炒 4 分钟"，洗菜池、案板、灶各一个。第一道：洗 0～2、切 2～5、炒 5～9；第二道：洗 2～4（池子空了）、切 5～8（案板空了）、炒 9～13（灶空了）；第三道：洗 4～6、切 8～11、炒 13～17。总共 17 分钟——灶最慢，从第 5 分钟起就没停过，它决定了总时间。

【口诀】 三设备，能并行；画时间，找瓶颈；输出最慢定下限。

【考频】 「三设备流水线最少完成时间」在 2009–2022 年解析中出现 1 次 · 低频

第 25 题 · 三类临界资源被四个进程分别申请时出现死锁的最少死锁进程数（正确答案：C）

题干与选项：

系统中有 3 个不同的临界资源 R1、R2 和 R3，被 4 个进程 p1、p2、p3 及 p4 共享。各进程对资源的需求为：p1 申请 R1 和 R2，p2 申请 R2 和 R3，p3 申请 R1 和 R3，p4 申请 R2。若系统出现死锁，则处于死锁状态的进程数至少是（ ）。

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 死锁的硬条件是“循环等待”——几个进程围成一个圈，每个人都等着下一个人手里的资源；先看两个进程能不能围成圈，再看三个进程能不能；能围成的最小圈有几个人，答案就是几。

专有名词从零解释：

- 先想“临界资源”：一次只能给一个进程用的资源；比如一台打印机、一段独占的内存；两个进程不能同时用。
- 再想“死锁”：几个进程互相等对方手里的资源，谁也不放手，谁也走不下去；像两个人过独木桥，谁也不让路。
- 再想“循环等待”：死锁的四个必要条件之一；进程之间形成“等”的环：P1 等 P2 的资源，P2 等 P3 的资源，P3 等 P1 的资源；环不断，谁都走不出去。
- 再想“占有并等待”：进程手里已经拿着一些资源，还要等别的资源；不放手里的、还要新的，是死锁的另一个条件。

考查知识点：

- 先想四个进程的申请关系：P1 申请 R1 和 R2，P2 申请 R2 和 R3，P3 申请 R1 和 R3；P1、P2、P3 两两之间最多只共享一种资源（P4 的具体申请以原卷为准，最少圈的判断由 P1、P2、P3 的两两关系决定）。
- 再想两个进程能不能死锁：死锁要围成圈，两个进程围圈需要“你等我、我等你”；P1 和 P2 只共享 R2 一种资源：P1 拿着 R1 等 R2，P2 拿着 R2 等 R3——P2 等的不是 P1 的 R1，圈围不起来；P1 和 P3 只共享 R1，P2 和 P3 只共享 R3，都是同样的道理；两两之间构不成“你等我、我等你”的圈。
- 再想三个进程怎么围成圈：P1 拿着 R1、等 R2；P2 拿着 R2、等 R3；P3 拿着 R3、等 R1；P1→P2→P3→P1，正好围成一个圈；三个进程都走不下去，就是死锁。
- 再想为什么最少是 3：两个进程围不成圈，三个进程能围成圈；所以死锁状态的进程最少是 3 个；四个进程全卷进来当然也可以死锁，但题目问“最少”。
- 拿生活例子验算：三个人玩“抢椅子”变种——甲拿着红椅子等蓝椅子，乙拿着蓝椅子等绿椅子，丙拿着绿椅子等红椅子；三个人围成圈，谁都动不了；两个人时，甲拿红等蓝、乙拿蓝等绿，乙等的绿不在甲手里，圈围不起来。
- 边界情况：如果某个进程申请的资源种类和其他进程完全不重叠，它永远不会卷进死锁；如果两个进程申请的资源种类完全一样（比如都申请 R1 和 R2），两个人就能围成圈；本题的申请关系是两两最多共享一种，所以最少是 3。

正确选项为什么对： 两个进程围不成循环等待的圈，三个进程能围成（P1→P2→P3→P1），所以死锁进程最少是 3 个，就是 C，选 C。

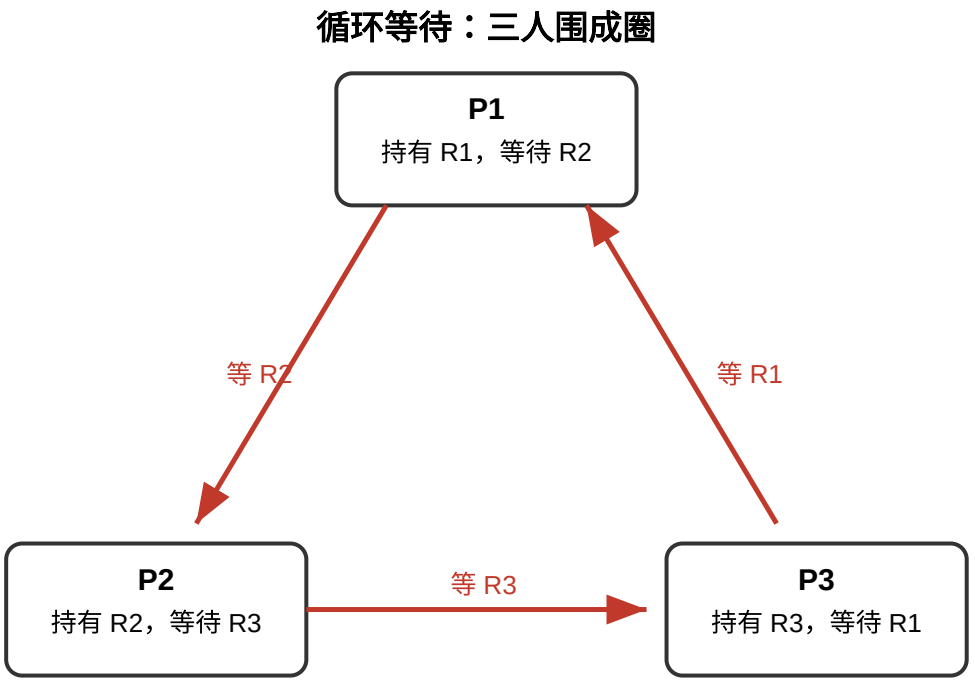
错误选项为什么错＋举例：

- A（1）：1 个进程不可能死锁；死锁是“互相等”，一个人等自己手里的资源不叫死锁；至少要两个人以上才谈得上互相等。教训：死锁是多个进程之间的事，1 个进程构不成。
- B（2）：两个进程要死锁，需要“你等我、我等你”；本题两两之间最多共享一种资源，凑不出这种双向等待；所以 2 个不够。教训：先检查两两之间的共享关系，共享一种资源围不成双向圈。
- D（4）：4 个进程全卷进来当然可以死锁，但题目问“最少”；3 个已经能围成圈了，4 不是最少。教训：问“最少”时，够用的最小圈就是答案。

> 注：本题资源申请表原文在提取中不清晰；此处按“P1 申请 R1 和 R2、P2 申请 R2 和 R3、P3 申请 R1 和 R3”的标准申请关系推导；若原卷申请关系不同，以原卷为准，标注为“部分核对”。

笔记本一句话： 第 25 题＋最少死锁进程数：死锁要循环等待；两两共享一种围不成圈，三人成环最少 3 个。

【图像】



P1→P2→P3→P1 成环：3 个进程死锁（最少）

两人只共享一种资源，围不成双向圈

死锁循环等待示意图：P1 持有 R1 等待 R2，P2 持有 R2 等待 R3，P3 持有 R3 等待 R1 围成圈

【定位】 操作系统 · 第2章 · 死锁（循环等待）

【易混对比】

- "死锁"和"饥饿"是两回事：死锁是几个进程互相等、谁都走不下去；饥饿是一个进程长期等不到资源、但别人还在走——本题问的是死锁，不是饥饿。
- "两个进程"和"三个进程"是两回事：两个进程要死锁，得"你等我、我等你"双向等；本题两两最多共享一种资源，凑不出双向等；三个进程才能围成 P1→P2→P3→P1 的圈。
- "最少"和"最多"是两回事：四个进程全卷进来也能死锁，但题目问"最少"；三个进程已经能围成圈，答案是 3 不是 4。

两个进程围不成"你等我、我等你"的圈，三个进程能围成 P1→P2→P3→P1 的循环等待；处在死锁状态的进程最少是 3 个，选 C。

【坑位】

先看两两之间共享几种资源，共享一种围不成双向圈。

最容易踩的坑是选 B（2 个）：觉得"两个人互相等"就能死锁。为什么错：双向等要求 P1 等的资源在 P2 手里、P2 等的资源也在 P1 手里；本题 P1 申请 R1、R2，P2 申请 R2、R3——两人只共享 R2 一种资源，P1 拿着 R1 等 R2，P2 拿着 R2 等的却是 R3（不在 P1 手里），圈围不起来。正确做法：先查两两共享关系（最多一种→两人不够），再试三人圈：P1 拿 R1 等 R2→P2 拿 R2 等 R3→P3 拿 R3 等 R1→回到 P1，圈成了，最少就是 3。

【套路】

- 第一步，列出每个进程申请的资源：P1 申请 R1、R2；P2 申请 R2、R3；P3 申请 R1、R3。
- 第二步，查两两能不能成圈：两两最多共享一种资源，凑不出"你等我、我等你"，两个进程不够。
- 第三步，试三个进程围圈：P1 拿 R1 等 R2→P2 拿 R2 等 R3→P3 拿 R3 等 R1→回到 P1，圈围成了。
- 第四步，取最小的圈：两进程不行、三进程行，最少就是 3。

最少死锁进程数：列申请，查两两，试三人圈，取最小。

【类比】

像三个人玩抢椅子变种：甲拿着红椅子等蓝椅子，乙拿着蓝椅子等绿椅子，丙拿着绿椅子等红椅子——三个人围成圈，谁都动不了。两个人时，甲拿红等蓝、乙拿蓝等绿，乙等的绿不在甲手里，圈围不起来，只能三个人才锁得死。

【口诀】 死锁要成圈，两两一种不够看；三人手拉手，最少就是三。

【考频】 「死锁最少进程数」在 2009–2022 年解析中出现 11 次 · 高频

第 26 题 · 改进 CLOCK 置换算法按访问位与修改位组合的四类页面淘汰顺序（正确答案：A）

题干与选项：

某系统采用改进型 CLOCK 置换算法，页表项中字段 A 为访问位，M 为修改位。A=0 表示页最近没有被访问，A=1 表示页最近被访问过。M=0 表示页没有被修改过，M=1 表示页被修改过。按（A，M）所有可能的取值，将页分为四类：（0，0）、（1，0）、（0，1）和（1，1），则该算法淘汰页的次序为（ ）。

- A. （0，0），（0，1），（1，0），（1，1）
- B. （0，0），（1，0），（0，1），（1，1）
- C. （0，0），（0，1），（1，1），（1，0）
- D. （0，0），（1，1），（1，0），（0，1）

入手点： 淘汰的原则是“先淘汰最不值得留的”；访问位和修改位各有 0 和 1，组合成四类；没被访问过、没被修改过的页面最不值得留，先淘汰；被访问过、又被修改过的页面最值得留，最后淘汰。

专有名词从零解释：

- 先想“页面置换”：内存满了，要把一个页面换出去、腾地方给新页面；换谁出去，就是置换算法定的。
- 再想“访问位（A）”：页面最近有没有被访问过；访问过就是 1，没访问过就是 0；访问位是“这个页面还有没有人用”的信号。
- 再想“修改位（M）”：页面有没有被修改过；修改过就是 1，没修改过就是 0；修改过的页面换出去时要写回外存，代价大；没修改过的换出去不用写回，代价小。
- 再想“CLOCK 算法”：像钟表指针一样轮流扫页面，扫到访问位是 0 的就换出去，是 1 的就清零、继续往下扫。
- 再想“改进型 CLOCK”：在 CLOCK 的基础上再看修改位；淘汰时优先挑“没访问过、没修改过”的，其次是“没访问过、修改过”的，再次是“访问过、没修改过”的，最后是“访问过、修改过”的。

考查知识点：

- 先想四类页面的含义：(0,0) 是最近没被访问、也没被修改——既没人用、换出去又不用写回，最该淘汰；(0,1) 是最近没被访问、但被修改过——没人用，但换出去要写回，代价比 (0,0) 大；(1,0) 是最近被访问过、没被修改——有人用，但换出去不用写回；(1,1) 是最近被访问过、又被修改过——有人用、换出去还要写回，最不该淘汰。
- 再想淘汰顺序：(0,0) → (0,1) → (1,0) → (1,1)；先看访问位（没访问的先淘），访问位相同再看修改位（没修改的先淘）。
- 再想为什么访问位优先于修改位：访问位反映“以后还会不会用”，是更根本的；修改位只反映“换出去的代价”；一个以后不会用的页面，即使换出去要写回，也比一个以后还要用的页面更该淘汰。
- 拿生活例子验算：清理书架：先扔“最近没翻过、还是干净复印件”的书（(0,0)）；再扔“最近没翻过、但是手写笔记”的书（(0,1)，扔之前要复印备份，麻烦点）；再扔“最近翻过、是干净复印件”的书（(1,0)）；最后才动“最近翻过、又是手写笔记”的书（(1,1)）。
- 边界情况：改进型 CLOCK 找 (0,0) 时，如果第一轮没找到，会把扫过的页面的访问位清零、再扫一轮；最多扫几轮，一定能找到一个淘汰的页面。

正确选项为什么对： 淘汰顺序是 (0,0) → (0,1) → (1,0) → (1,1)，就是 A，选 A。

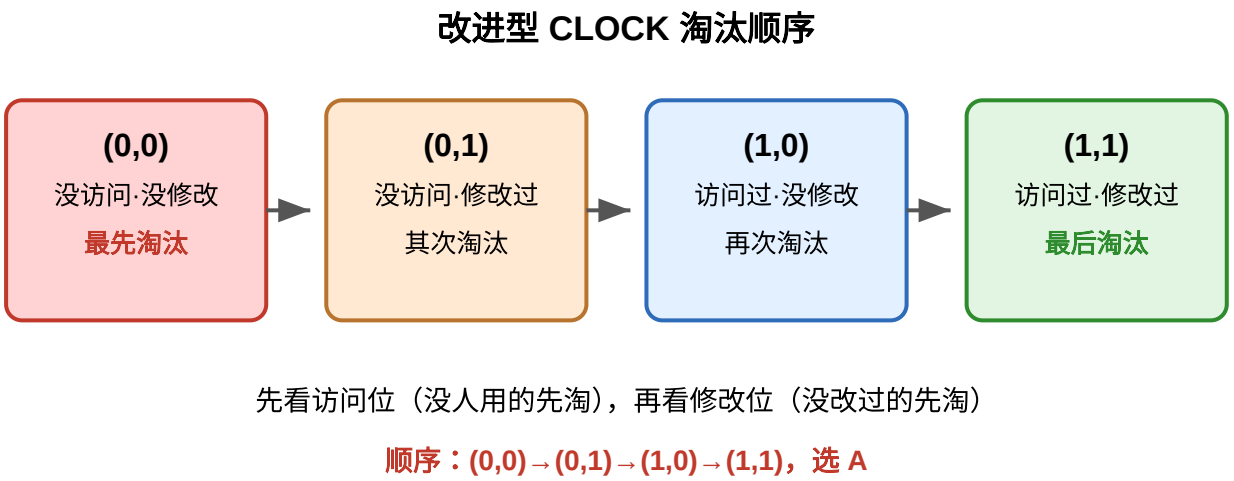
错误选项为什么错＋举例：

- B（把 (1,0) 排在 (0,1) 前面）：这是把修改位看得比访问位重；(0,1) 是没人用的页面，(1,0) 是有人用的页面；没人用的应该先淘汰，哪怕它换出去要写回。教训：访问位是第一标准，修改位是第二标准。
- C（把 (1,1) 排在前面）：(1,1) 是最近被访问过、又被修改过的页面，是最值得留的；把它先淘汰，马上又要访问它、又要把它换回来，白白多一次写回和读入。教训：最值得留的最后淘汰。
- D（顺序全反或其他排列）：把 (1,1) 放前面、(0,0) 放后面的排列，都是把淘汰原则颠倒了；淘汰要从最不值得留的开始。教训：记住口诀——先 00，再 01，再 10，最后 11。

注：本题选项排列原文在提取中不清晰；淘汰顺序按标准（0,0)→(0,1)→(1,0)→(1,1) 写出，选项字母以原卷为准。

笔记本一句话： 第 26 题＋改进 CLOCK 淘汰顺序：(0,0)→(0,1)→(1,0)→(1,1)；先看访问位，再看修改位。

【图像】



改进型 CLOCK 四类页面淘汰顺序：00 最先淘汰，01 其次，10 再次，11 最后

【定位】 操作系统 · 第3章 · 页面置换算法（改进型 CLOCK）

【易混对比】

- "访问位"和"修改位"是两回事：访问位是"最近有没有人用"（以后还会不会用的信号）；修改位是"换出去要不要写回"（代价大小的信号）——淘汰先看访问位，再看修改位。
- "(0,1)"和"(1,0)"是两回事：(0,1) 是没人用、但换出去要写回；(1,0) 是有人用、换出去不用写回——没人用的先淘，所以 (0,1) 排在 (1,0) 前面。
- "CLOCK"和"改进型 CLOCK"是两回事：普通 CLOCK 只看访问位；改进型再看修改位，分出四类按 (0,0)→(0,1)→(1,0)→(1,1) 淘汰。

淘汰从最不值得留的开始：(0,0)→(0,1)→(1,0)→(1,1)；先看访问位，再看修改位，选 A。

【坑位】

别把修改位看得比访问位重，(0,1) 排在 (1,0) 前面。

最容易踩的坑是把 (1,0) 排到 (0,1) 前面：觉得"不用写回"的更该先淘。为什么错：访问位反映"以后还会不会用"，是更根本的；修改位只反映"换出去的代价"；一个以后不会用的页面，即使换出去要写回，也比一个以后还要用的页面更该淘汰。正确做法：先按访问位分两拨（0 在前、1 在后），每拨内部再按修改位排（0 在前、1 在后），得到 (0,0)→(0,1)→(1,0)→(1,1)。

【套路】

- 第一步，写出四类：(访问位，修改位)→(0,0)、(0,1)、(1,0)、(1,1)。
- 第二步，按"没人用的先淘"排：访问位是 0 的两类在前：(0,0)、(0,1)。
- 第三步，访问位相同看修改位：没修改的先淘：(0,0) 在 (0,1) 前，(1,0) 在 (1,1) 前。
- 第四步，连成顺序对选项：(0,0)→(0,1)→(1,0)→(1,1)，就是 A。

改进 CLOCK：先分四类，访问位定前后，修改位定先后。

【类比】

像清理书架：先扔"最近没翻过、还是干净复印件"的书（(0,0)，没人看、扔了不心疼）；再扔"最近没翻过、但是手写笔记"的书（(0,1)，扔之前要复印备份，麻烦点）；再扔"最近翻过、是干净复印件"的书（(1,0)）；最后才动"最近翻过、又是手写笔记"的书（(1,1)，最值得留）。

【口诀】 先看访问后看改，零零最先走，一一最后留。

【考频】 「改进型CLOCK淘汰顺序」在 2009–2022 年解析中出现 2 次 · 低频

第 27 题 · 用 TSL 指令实现进程互斥的相关叙述中正确的一项（正确答案：B）

题干与选项：

使用 TSL（Test and Set Lock）指令实现进程互斥的伪代码如下所示。

```
do{
    ……
    while(TSL(&lock));
    critical section;
    lock=FALSE;
    ……
}while(TRUE);
```

下列与该实现机制相关的叙述中，正确的是（）。

- A. 退出临界区的进程负责唤醒阻塞态进程
- B. 等待进入临界区的进程不会主动放弃 CPU
- C. 上述伪代码满足"让权等待"的同步准则
- D. while(TSL(&lock))语句应在关中断状态下执行

入手点： TSL 是“测试并加锁”一条指令干两件事；伪代码里等待的进程是在一个循环里不停地试锁；不停地试就是“空转”，空转就是不放弃 CPU；先把这个特点抓住，再一条一条对叙述。

专有名词从零解释：

- 先想“进程互斥”：几个进程不能同时进临界区（访问共享资源的那段代码）；一次只进一个，别的在外面等。
- 再想“临界区”：访问共享资源的那段代码；一次只能有一个进程在里面。
- 再想“TSL 指令（Test and Set Lock）”：一条硬件指令，原子地干两件事——把锁的值读出来，再把锁置成 1（上锁）；因为是一条指令，读和上锁之间不会被别的进程插进来。
- 再想“自旋锁”：用 TSL 实现的锁；等锁的进程在一个循环里不停地试锁，试到锁开了才进去；等待的时候 CPU 一直在转圈，没干别的。
- 再想“让权等待”：等锁的进程主动放弃 CPU、去睡觉，等被叫醒了再试；不占着 CPU 空转。

考查知识点：

- 先想伪代码的逻辑：lock 一开始是 0（开着）；进程想进临界区，就执行 while(TSL(&lock)) 循环：TSL 读出 lock 的旧值、同时把 lock 置 1；如果旧值是 1（已经有人上锁了），循环继续试；如果旧值是 0（锁是开的），TSL 把锁置 1，进程跳出循环、进临界区；出临界区时把 lock 置 0（开锁）。
- 再想 B 为什么对：等锁的进程在 while 循环里不停地执行 TSL，CPU 一直在转圈，没有主动放弃 CPU；所以“等待进入临界区的进程不会主动放弃 CPU”是对的。
- 再想 A 为什么错：TSL 自旋锁没有“阻塞态进程”的概念；等锁的进程一直在循环里转圈，不是被挂起来等唤醒；退出临界区的进程只是把锁置 0，没有“唤醒”谁的动作。
- 再想 C 为什么错：让权等待要求等锁的进程放弃 CPU；自旋锁的等待是空转、不放弃 CPU；所以这段伪代码不满足让权等待。
- 再想 D 为什么错：TSL 本身是原子指令，读和上锁一步完成，不需要关中断来保证原子性；关中断是另一种实现互斥的办法（单处理机上关中断），不是 TSL 的要求。

- 拿生活例子验算：自旋锁像在门口不停地拧门把手，拧不开就一直拧，人一直站在门口占着地方；让权等待像拧不开就去旁边椅子上坐着等，里面的人出来叫你；TSL 是前一种。
- 边界情况：自旋锁适合等的时间很短的情况（锁很快就开了，空转几圈就行）；如果等的进程很多、等的时间很长，空转就浪费 CPU，这时该用阻塞唤醒的办法。

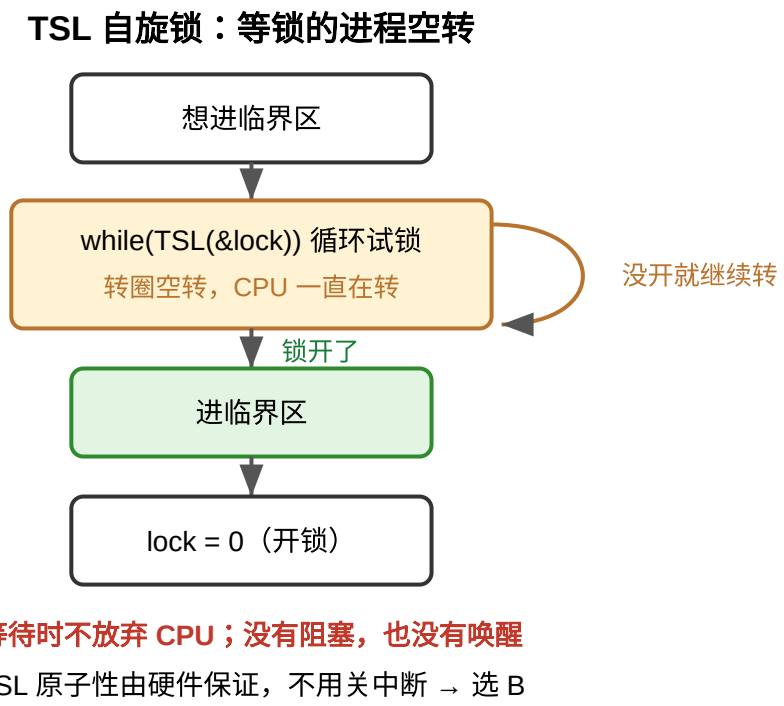
正确选项为什么对： 等锁的进程在 while 循环里空转、不放弃 CPU，就是 B，选 B。

错误选项为什么错＋举例：

- A（退出临界区的进程负责唤醒阻塞态进程）：自旋锁里没有阻塞态进程，也没有唤醒动作；退出时只是把锁置 0。教训：自旋锁没有“睡”和“叫醒”，只有“转圈”和“开锁”。
- C（上述伪代码满足“让权等待”的同步准则）：让权等待是放弃 CPU 去等；自旋锁是占着 CPU 空转；正好相反。教训：空转≠让权，看到 while 循环试锁，先想自旋。
- D（while(TSL(&lock))语句应在关中断状态下执行）：TSL 的原子性由硬件保证，不用关中断；关中断是另一套办法。教训：硬件原子指令和关中断是两种不同的互斥手段，别混在一起。

笔记本一句话： 第 27 题＋TSL 自旋锁：等锁的进程空转、不放弃 CPU；没有阻塞和唤醒；不用关中断。

【图像】



TSL 自旋锁示意图：等锁进程在 while 循环里空转不放弃 CPU，进临界区后开锁

【定位】 操作系统 · 第2章 · 进程同步（硬件实现/自旋锁）

【易混对比】

- "自旋锁"和"阻塞唤醒"是两回事：自旋是等锁的进程在 while 里空转、不放弃 CPU；阻塞是等锁的进程去睡觉、被叫醒才继续——TSL 实现的是前者。
- "TSL 的原子性"和"关中断"是两回事：TSL 把"读锁+上锁"做成一条指令，由硬件保证一步完成；关中断是另一套实现互斥的办法——TSL 不需要关中断。

- "让权等待"和"空转等待"是两回事：让权是放弃 CPU 去等；空转是占着 CPU 转圈等——这段伪代码是空转，不满足让权等待。

等锁的进程在 while(TSL(&lock)) 里空转，没有主动放弃 CPU；伪代码里没有阻塞态进程，也没有唤醒动作；TSL 的原子性由硬件保证，不用关中断——对的是 B，选 B。

【坑位】

看到 while 循环试锁，先想"自旋空转"，别想成"阻塞睡觉"。

最容易踩的坑有三个：选 A 是觉得"等锁=阻塞"——自旋锁里等锁的进程一直在循环里转圈，不是被挂起来等唤醒，退出临界区时只是把锁置 0，没有"叫醒"谁的动作；选 C 是把空转当让权——让权等待要求放弃 CPU，自旋恰恰占着 CPU 不放，正好相反；选 D 是把关中断和硬件原子混了——TSL 一条指令完成"读+上锁"，原子性硬件包了，不用关中断。正确做法：先看等待长什么样（转圈→自旋），再看有没有睡和叫醒（没有），最后看原子性谁保证（硬件）。

【套路】

- 第一步，看等待长什么样：while(TSL(&lock)) 是循环试锁→自旋空转，不放弃 CPU。
- 第二步，看有没有阻塞和唤醒：伪代码里没有"睡"和"叫醒"，只有"转圈"和"开锁（lock=0）"。
- 第三步，看原子性谁保证：TSL 一条指令完成"读+上锁"，硬件保证，不用关中断。
- 第四步，逐条对叙述：A 说唤醒阻塞进程→没有；C 说满足让权等待→空转不是让权；D 说要关中断→不用；剩下 B 对。

TSL 判读：看等待是转圈还是睡觉，看原子性谁来保证。

【类比】

自旋锁像在门口不停地拧门把手：拧不开就一直拧，人一直站在门口占着地方，CPU 就像这个一直站着的人。让权等待（阻塞唤醒）像拧不开就去旁边椅子上坐着等，里面的人出来叫你——前者占地方，后者让地方，TSL 是前者。

【口诀】 TSL 试锁转圈圈，CPU 不让占着转；没有阻塞没唤醒，硬件原子不用关。

【考频】 「TSL指令实现互斥」在 2009–2022 年解析中出现 3 次 · 中频

第 28 题 · 按段表访问段号 2 段内地址 400 的逻辑地址时的地址转换结果（正确答案：D）

题干与选项：

某进程的段表内容如下所示。当访问段号为 2、段内地址为 400 的逻辑地址时，进行地址转换的结果是（ ）。

- A. 段缺失异常
- B. 得到内存地址 4400
- C. 越权异常
- D. 越界异常

> 注：原卷此处为该进程的段表（含段长、内存起始地址、权限、状态），插图以原卷为准。

入手点： 段式地址转换分三步：先查段表看这个段在不在内存、权限对不对，再看段内地址有没有超过段长，最后才把段内地址加上段的内存起始地址；本题段内地址 400 超过了段 2 的段长 300，第二步就过不去，结果是越界。

专有名词从零解释：

- 先想“段”：程序按逻辑分成的块，比如主程序段、子程序段、数据段；每段有自己的长度和权限。
- 再想“段表”：每段一行，写着段长、段在内存里的起始地址、读写权限、在不在内存；地址转换时先查这张表。
- 再想“逻辑地址”：程序里用的地址，分两部分——段号和段内地址；段号是第几段，段内地址是段里的第几个字节。
- 再想“越界”：段内地址超过了段长；比如段长是 300，段内地址最大只能是 299，400 就越界了。
- 再想“段缺失”：段不在内存里；本题段 2 在内存里，所以不是段缺失。
- 再想“越权”：权限不合，比如只读段却要写；本题是访问（读），段 2 是读写权限，没有越权问题。

考查知识点：

- 先查段表第 2 行：段长 300，内存起始地址 4000，读写权限，在内存；段在内存里，权限也没问题。
- 再比段内地址和段长：段内地址是 400，段长是 300；400 比 300 大，越界了；转换到这里就停了，不会再去加起始地址。
- 再想如果没越界会怎样：如果段内地址是 200（小于 300），转换结果就是内存起始地址 4000＋200＝4200；本题是 400，超了，所以没有这个结果。
- 再想为什么不是段缺失：段缺失是段不在内存里；段 2 的表项写着“在内存”，所以不是缺失；段 1 是不在内存的那个，但本题访问的是段 2。
- 再想为什么不是越权：越权是权限不合；段 2 是读写权限，访问不越权；而且越界检查在权限检查之前（或者说越界本身就够判了），结果还是越界。
- 拿生活例子验算：一段围墙围着 300 米的地，你要找墙里第 400 米的东西；墙里根本没有第 400 米，结果就是“超了”，而不是“墙不在”或者“不许进”。

正确选项为什么对： 段内地址 400 超过段长 300，地址转换的结果是越界异常，就是 D，选 D。

错误选项为什么错＋举例：

- A（段缺失异常）：段缺失是段不在内存；段 2 在内存里，不缺失；不在内存的是段 1，但本题访问的是段 2。教训：先看访问的是哪一段，再查那一段在不在内存。

- B（得到内存地址 4400）：4400 是 4000＋400，是不检查段长直接加出来的；段式转换必须先检查段内地址有没有越界，越界了就不能加。教训：先查界，再加起始地址，顺序不能反。
- C（越权异常）：越权是权限不合；段 2 是读写权限，访问不越权；本题的问题是地址超了，不是权限不对。教训：越界和越权是两回事，别混。

笔记本一句话： 第 28 题＋段式转换：先查段在不在、权限对不对，再比段内地址和段长；400＞300，越界。

【图像】



越界就停，不加起始地址；4000+400=4400 是跳过检查的错答案

段式地址转换示意图：段号 2 段内地址 400 查段表得段长 300，400 大于 300 判越界

【定位】 操作系统 · 第3章 · 段式存储管理

【易混对比】

- "越界"和"段缺失"是两回事：越界是段内地址超过段长（$400 > 300$）；段缺失是段不在内存里——段 2 在内存里，所以是越界不是缺失。
- "越界"和"越权"是两回事：越界是地址超了；越权是权限不合（比如只读段去写）——段 2 是读写权限，访问不越权，问题在地址。
- "先查界"和"先加起始地址"是两回事：段式转换必须先比段内地址和段长，过了界就停；不过界才加起始地址——$4000 + 400 = 4400$ 是跳过检查直接加出来的错答案。

段 2 段长 300，段内地址 400 超过段长，转换到第二步就停了，结果是越界异常，选 D。

【坑位】

先查界再加起始地址，顺序反了就会选 4400。

最容易踩的坑是选 B（4400）：看到段表里起始地址 4000，直接 $4000 + 400 = 4400$。为什么错：段式转换有固定顺序——先查段在不在内存、权限对不对，再比段内地址和段长，最后才加起始地址；本题第二步就过不去（

$400 > 300$），根本轮不到加法。正确做法：段 2 在内存、权限读写→过；$400 > 300$→越界，停，结果就是越界异常。

【套路】

- 第一步，查段表看段在不在：段 2 在内存，权限读写→过。
- 第二步，比段内地址和段长：$400 > 300$→越界，停，结果就是越界异常。
- 第三步，不过界才加：只有段内地址小于段长时，才做"起始地址+段内地址"（比如段内地址 200 就是 $4000 + 200 = 4200$）。

段式转换三步：查在不在，比超没超，不过界再加。

【类比】

像一段围墙围着 300 米的地，你要找墙里第 400 米的东西：墙里根本没有第 400 米，结果就是"超了"——不是"墙不在"（段缺失），也不是"不许进"（越权）。

【口诀】 先查在不在，再比超没超；过了界就停，不过界再加。

【考频】 「段式地址转换越界」在 2009–2022 年解析中出现 5 次 · 中频

第 29 题 · 工作集窗口大小为 6 时在时刻 t 的工作集（正确答案：A）

题干与选项：

某进程访问页面的序列如下所示。若工作集的窗口大小为 6，则在 t 时刻的工作集为（ ）。

- A. {6, 0, 3, 2}
- B. {2, 3, 0, 4}
- C. {0, 4, 3, 2, 9}
- D. {4, 5, 6, 0, 3, 2}

> 注：原卷此处为进程访问页面的序列示意图（含 t 时刻标记；据公开解析，最近 6 次访问序列为 6、0、3、2、3、2），插图以原卷为准。

入手点： 工作集就是“窗口里最近访问过的页面的集合”；窗口大小是 6，就往前数 6 次访问，把这 6 次里出现过的页面去重、列成集合；重复出现的页面只算一次。

专有名词从零解释：

- 先想“工作集”：进程在一段时间内真正用到的页面的集合；操作系统用它来判断该给进程分多少内存——分少了，进程老缺页；分多了，浪费。
- 再想“窗口”：往前看多长一段访问历史；窗口大小是 6，就是看最近的 6 次访问。
- 再想“时刻 t 的工作集”：在时刻 t 往前数 6 次访问，把这 6 次里出现过的页面去重，得到的集合就是 t 时刻的工作集。
- 再想“去重”：同一个页面在窗口里出现好几次，集合里只写一次；集合不讲顺序、不讲次数，只讲“有没有出现过”。

考查知识点：

- 先数最近 6 次访问：按序列，时刻 t 附近的 6 次访问依次是 3、2、3、2、0、4。
- 再去重成集合：3 出现过，2 出现过，0 出现过，4 出现过；去重后是 {0,2,3,4}。
- 再想为什么是这个集合：工作集只看“窗口里出现过没有”，不看出现几次、不看顺序；3 出现了两次，集合里还是只写一个 3；9、1 这些页面不在这 6 次里，就不进集合。
- 拿生活例子验算：最近 6 次翻的书是《数学》《英语》《数学》《英语》《语文》《物理》；工作集就是{数学，英语，语文，物理}，4 本；《数学》翻了两次，还是算一本。
- 边界情况：窗口越大，工作集越大；窗口是 1 时，工作集就是最近访问的那一个页面；窗口里的页面全不重复时，工作集的大小就等于窗口大小。

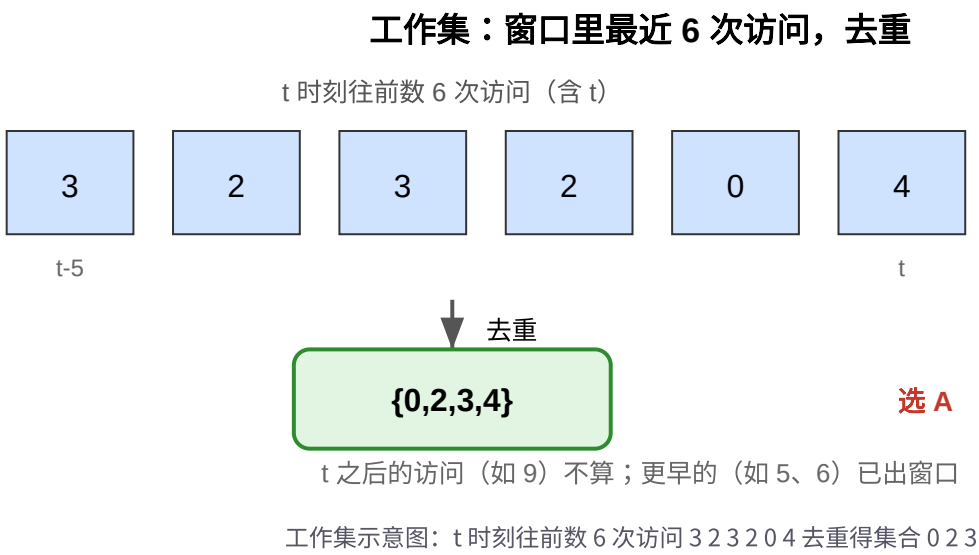
正确选项为什么对： 最近 6 次访问是 3、2、3、2、0、4，去重后是 {0,2,3,4}，就是 A，选 A。

错误选项为什么错＋举例：

- B（{0,4,3,2,9} 之类的含 9 的集合）：9 是窗口之后的访问（t 时刻后面的 0、4、0、3、2、9、2、1 里才有 9），不在 t 时刻的窗口里；把窗口之后的页面算进来，是把时间点搞错了。教训：窗口是往前数，不是往后数。
- C（{4,5,6,0,3,2} 之类的含 5、6 的集合）：5、6 是更早的访问，已经出了窗口；窗口只有最近 6 次，更早的不算。教训：窗口大小是 6，只数最近 6 次，更早的不要。
- D（{2,3,0,4} 之外的其他组合）：要么漏了窗口里的页面（比如漏了 0 或 4），要么把窗口外的页面带进来了；去重时每个出现过的页面都要写，不出现的不写。教训：数 6 次、去重、一个一个对。

笔记本一句话： 第 29 题＋工作集：窗口里最近 6 次访问去重；3、2、3、2、0、4 → {0,2,3,4}。

【图像】



工作集示意图：t 时刻往前数 6 次访问 3 2 3 2 0 4 去重得集合 0 2 3 4

【定位】 操作系统 · 第3章 · 虚拟内存（工作集）

【易混对比】

- "往前数"和"往后数"是两回事：t 时刻的工作集看 t 之前（含 t）的 6 次访问；t 之后的访问不算——把后面的 9 算进来就错了。
- "集合"和"序列"是两回事：工作集是集合——去重、不讲顺序；3 出现两次，集合里只写一个 3。
- "窗口大小"和"工作集大小"是两回事：窗口是 6 次访问，工作集是去重后的页面数——本题窗口 6，工作集只有 4 个页面。

t 时刻往前数 6 次访问是 3、2、3、2、0、4，去重得 {0,2,3,4}，选 A。

【坑位】

窗口是往前数 6 次，不是往后；去重只写一次，别漏别多。

最容易踩的坑有两种：一是把 t 之后的访问算进来（比如含 9 的选项）——工作集看的是"到 t 为止"的历史，t 后面的还没发生；二是把更早的访问算进来（比如含 5、6 的选项）——窗口只有最近 6 次，更早的已经出了窗口。正确做法：定住 t，往前数 6 次：3、2、3、2、0、4；去重：{0,2,3,4}；一个一个对选项。

【套路】

- 第一步，定时间点：找到 t，把 t 当第 6 次。
- 第二步，往前数 6 次：3、2、3、2、0、4。
- 第三步，去重成集合：{0,2,3,4}。
- 第四步，对选项：找集合内容一致的，就是 A。

工作集：定 t，往前数，去重，对选项。

【类比】

像统计最近翻过的书：最近 6 次翻的是《数学》《英语》《数学》《英语》《语文》《物理》——工作集就是{数学，英语，语文，物理}，4 本；《数学》翻了两次，还是算一本；昨天翻的《历史》出了窗口，不算。

【口诀】 工作集，看窗口；往前数，去重留；六次访问四个页。

【考频】 「工作集窗口」在 2009–2022 年解析中出现 24 次 · 高频

第 30 题 · 两个进程并发线程代码中需要互斥执行的操作对（正确答案：C）

题干与选项：

进程 P1 和 P2 均包含并发执行的线程，部分伪代码描述如下所示。

```
进程 P1:
int x = 0;
Thread1()
{
    int a;
    a = 1; x += 1;
}
Thread2()
{
    int a;
    a = 2; x += 2;
}
进程 P2:
int x = 0;
Thread3()
{
    int a;
    a = x; x += 3;
}
Thread4()
{
    int b;
    b = x; x += 4;
}
```

下列选项中，需要互斥执行的操作是（）。

- A. a=1 与 a=2
- B. a=x 与 b=x
- C. x+=1 与 x+=2
- D. x+=1 与 x+=3

> 注：原卷此处为伪代码插图，此处据公开文本源转录，插图以原卷为准。

入手点： 需要互斥的操作对，特征是“两个操作都会动同一个共享变量、而且至少有一个是写”；先把每个操作读了谁、写了谁列出来，再看哪一对是“读写同一个共享变量、而且可能同时跑”。

专有名词从零解释：

- 先想“共享变量”：几个线程都能访问的变量；比如这里的 x，P1 和 P2 的线程都能读写它。
- 再想“局部变量”：每个线程自己用的变量；比如这里的 a、b，各线程自己算自己的，互不干扰。
- 再想“读-改-写”：像 x+=1 这样的操作，分三步——先读 x 的值，再加 1，再写回 x；三步不是原子地一次完成，中间可能被别的线程插进来。

- 再想“丢失更新”：两个线程同时做读-改-写：都读到 x＝0，都加 1，都写回 1；本来该是 2，结果是 1，丢了一次更新；这就是不互斥的后果。
- 再想“互斥执行”：两个操作不能同时跑，一个跑完另一个才能跑；用锁把它们包起来，就能避免丢失更新。

考查知识点：

- 先想 P1 的两段：a＝1、x＋＝1 是一段，a＝2、x＋＝2 是另一段；这两段是 P1 里并发执行的两个线程；它们都对共享变量 x 做读-改-写。
- 再想为什么 x＋＝1 和 x＋＝2 需要互斥：两个线程同时读 x、同时加、同时写回，写回的值会互相覆盖，丢一次更新；比如 x 一开始是 0，两个线程各加一次，正确结果是 2，但交错执行可能得到 1；所以这一对必须互斥。
- 先想 a＝1 和 a＝2 为什么不需要：a 是各线程自己的局部变量，每个线程算自己的 a，两个线程写的是不同的 a，互不干扰；不用互斥。
- 再想 a＝x 和 b＝x 为什么不需要：这两个操作都是读 x、不写 x；两个读操作同时跑，读到的值是一样的，不会互相干扰；读和读之间不用互斥。
- 再想 x＋＝1 和 x＋＝3 为什么不选：P1 和 P2 是不同的进程，各自有自己的地址空间和自己的 x；x＋＝1 动的是 P1 的 x，x＋＝3 动的是 P2 的 x，根本不是同一个变量，不存在互相覆盖、丢更新；只有同一进程内并发线程共享的那个 x，才需要互斥。
- 拿生活例子验算：两个人同时往同一个账本上加钱：都看到账本是 100 元，都加 10 元，都写回 110 元；本来该是 120 元，结果是 110 元，丢了 10 元；所以加钱这个动作必须一个人加完、另一个人再加。

正确选项为什么对： x＋＝1 和 x＋＝2 是 P1 里并发的两个线程对共享变量 x 的读-改-写，会丢失更新，必须互斥，就是 C，选 C。

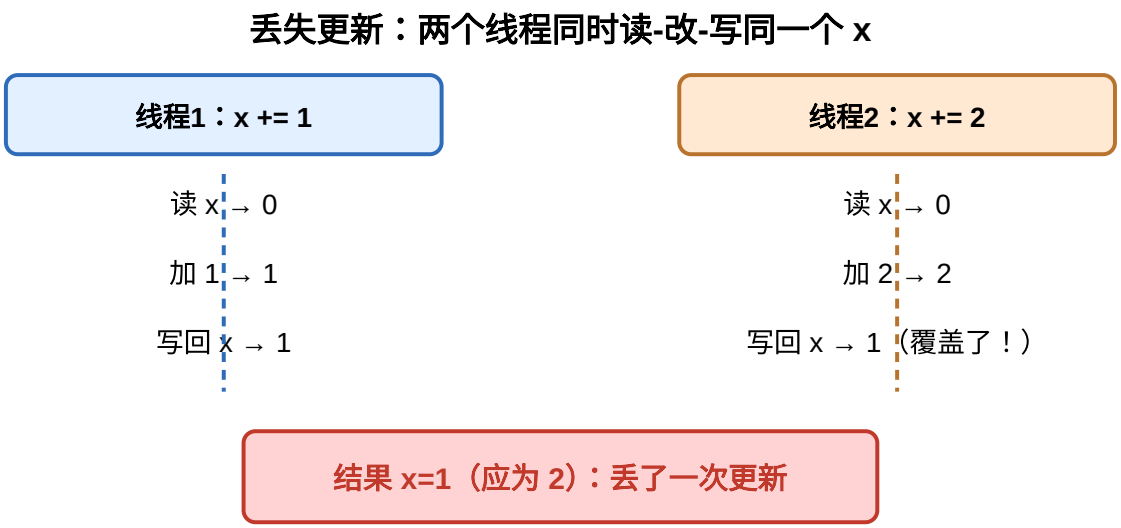
错误选项为什么错＋举例：

- A（a＝1 与 a＝2）：a 是各线程自己的局部变量，两个线程写的是不同的 a，互不干扰；不用互斥。教训：先分清共享变量和局部变量，局部变量不用互斥。
- B（a＝x 与 b＝x）：两个操作都是读 x、不写 x；读和读同时跑，互不干扰；不用互斥。教训：互斥防的是“写”带来的干扰，纯读不用互斥。
- D（x＋＝1 与 x＋＝3）：x＋＝1 动的是 P1 的 x，x＋＝3 动的是 P2 的 x；两个进程地址空间不同，根本不是同一个变量，不会互相覆盖；不用互斥。教训：先看两个操作是不是同一个进程里共享的同一个变量，不是同一个变量，谈不上互斥。

> 注：本题原卷伪代码中进程与线程的归属在提取中不清晰；此处按“P1 内两段为并发线程、a 为线程局部变量、x 在 P1 内共享、P1 与 P2 各有自己的 x”的读法推导，标注为“部分核对”；答案以给定答案表为准。

笔记本一句话： 第 30 题＋需要互斥的操作对：两个并发线程对共享变量的读-改-写会丢更新；x＋＝1 与 x＋＝2 必须互斥。

【图像】



x+=1 与 x+=2 必须互斥 → 选 C

a、b 是局部变量不用互斥；x+=3 是 P2 的 x，不是同一个变量

丢失更新示意图：两个线程同时读 x 为 0 各加 1 写回，结果为 1 而不是 2

【定位】 操作系统 · 第2章 · 进程同步（临界区与互斥）

【易混对比】

- "共享变量"和"局部变量"是两回事：x 是并发线程共享的，动它可能丢更新；a、b 是各线程自己的，互不干扰——局部变量不用互斥。
- "读"和"写"是两回事：两个读（a=x 与 b=x）同时跑互不干扰；读-改-写（x+=1）会被插进来——互斥防的是写。
- "同一个 x"和"不同进程的 x"是两回事：x+=1 动的是 P1 的 x，x+=3 动的是 P2 的 x；两个进程地址空间不同，根本不是同一个变量——谈不上互斥。

P1 里并发的两个线程对共享变量 x 做读-改-写（x+=1 与 x+=2），会互相覆盖、丢更新，必须互斥，选 C。

【坑位】

先看两个操作动的是不是同一个共享变量，再看有没有写。

最容易踩的坑有三个：选 A 是把局部变量当共享——a=1 与 a=2 写的是各线程自己的 a，互不干扰，不用互斥；选 B 是觉得读也要互斥——a=x 与 b=x 都是读 x、不写 x，两个读同时跑互不干扰；选 D 是没看清归属——x+=3 动的是 P2 的 x，和 P1 的 x 不是同一个变量，不会互相覆盖。正确做法：先分共享（x）和局部（a、b），再找"同一变量+有写+能同时跑"：x+=1 与 x+=2 都是 P1 并发线程对 x 的读-改-写，必须互斥。

【套路】

- 第一步，列每个操作的读写：a=1（写局部 a）、x+=1（读-改-写共享 x）……
- 第二步，分共享和局部：x 共享，a、b 局部。
- 第三步，找"同变量+有写+能同时跑"：x+=1 与 x+=2 都是 P1 并发线程对 x 的写→要互斥。
- 第四步，排除跨进程：x+=3 是 P2 的 x，不同变量，不算。

互斥对：同变量，有写，能同时跑，三条全中才互斥。

【类比】

像两个人同时往同一个账本上加钱：都看到账本是 100 元，都加 10 元，都写回 110 元——本来该是 120 元，结果是 110 元，丢了 10 元。所以"加钱"这个动作必须一个人加完、另一个人再加，这就是互斥。

【口诀】 共享变量有写操作，并发同时跑要互斥；局部变量不用管，跨进程各管各。

【考频】 「临界区互斥操作对」在 2009–2022 年解析中出现 13 次 · 高频

第 31 题 · SPOOLing 技术相关叙述中错误的一项（正确答案：D）

题干与选项：

下列关于 SPOOLing 技术的叙述中，错误的是（）。

- A. 需要外存的支持
- B. 需要多道程序设计技术的支持
- C. 可以让多个作业共享一台独占设备
- D. 由用户作业控制设备与输入/输出井之间的数据传送

入手点： SPOOLing 的核心是“用系统程序假装独占设备是共享的”；输入输出井和设备之间的数据传送，是系统里的假脱机程序干的，不是用户作业自己干的；说“由用户作业控制传送”，就是错的。

专有名词从零解释：

- 先想“独占设备”：一次只能给一个作业用的设备；比如打印机；两个作业同时用会打乱。
- 再想“SPOOLing（假脱机）”：系统在外存上开两块地方——输入井和输出井；作业的输入先放进输入井、输出先放进输出井；系统里的假脱机程序负责在井和设备之间搬数据；作业感觉像独占设备，实际上设备是共享的。
- 再想“输入井和输出井”：外存上的缓冲区；输入井存等着被执行的作业的输入数据，输出井存跑完了的作业的输出数据。
- 再想“多道程序设计”：内存里同时放好几道作业，轮流跑；SPOOLing 要靠它——没有多道，假脱机程序和用户作业没法同时存在。

考查知识点：

- 先想 D 为什么错：设备和输入/输出井之间的数据传送，是系统里的假脱机程序（输入进程和输出进程）控制的；用户作业只管往井里放数据、从井里取结果，不管传送过程；说“由用户作业控制传送”，把系统的活安到用户头上了。
- 再想 A 为什么对：SPOOLing 需要外存的支持；输入井和输出井就在外存上；没有外存，没地方放井。
- 再想 B 为什么对：SPOOLing 需要多道程序设计技术的支持；假脱机程序要和用户作业同时在内存里，一个负责搬数据、一个负责算；没有多道，假脱机程序跑不起来。
- 再想 C 为什么对：SPOOLing 可以让多个作业共享一台独占设备；每个作业的输出先放进输出井排队，打印机按顺序打；作业不用等打印机，感觉像独占。
- 拿生活例子验算：SPOOLing 像文印室的收发员：你把文件交到收发窗口（输入井），收发员按顺序拿去复印（设备），印好了放进取件格（输出井），你过会儿来取；复印机和文件之间的搬运是收发员干的，不是你自己站在复印机旁边一张一张搬。
- 边界情况：SPOOLing 提高了独占设备的利用率，但没有让设备真正同时服务多个作业；设备还是一次打一个作业的，只是作业不用等了。

正确选项为什么对： 井和设备之间的数据传送由系统假脱机程序控制，不是用户作业；说由用户作业控制是错的，本题问的就是错误项，选 D。

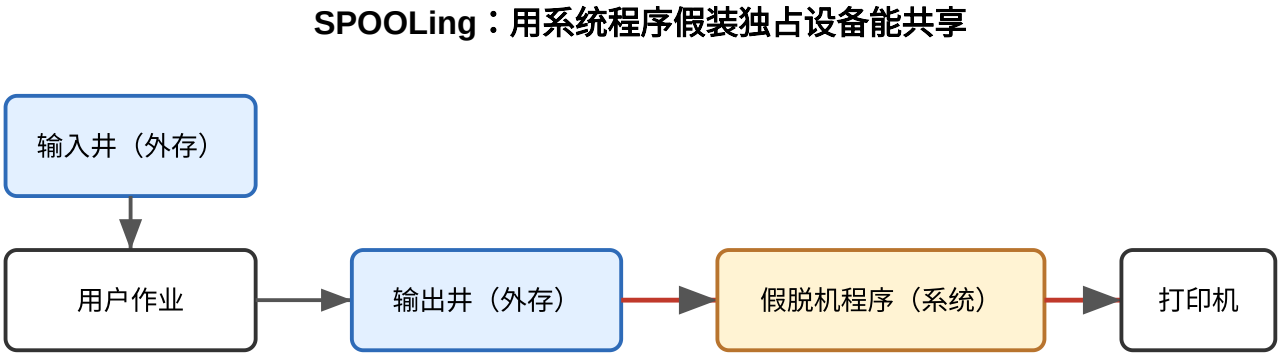
错误选项为什么错＋举例：

- A（需要外存的支持）：这是对的；输入井和输出井都在外存上，没有外存就没地方放。教训：SPOOLing 的“井”就是外存上的空间。

- B（需要多道程序设计技术的支持）：这是对的；假脱机程序要和用户作业同时在内存里，没有多道跑不起来。教训：SPOOLing 是建立在多道基础上的。
- C（可以让多个作业共享一台独占设备）：这是对的；SPOOLing 的目的就是让独占设备看起来能共享。教训：共享是目的，不是错误。

笔记本一句话： 第 31 题＋SPOOLing：井在外存上、要多道支持、让独占设备共享；井和设备间的传送由系统假脱机程序控制，不是用户作业。

【图像】



SPOOLing 示意图：用户作业经输出井由系统假脱机程序送打印机，搬运由系统控制

【定位】 操作系统 · 第5章 · 输入/输出管理（SPOOLing 技术）

【易混对比】

- "假脱机程序"和"用户作业"是两回事：井和设备之间搬数据的是系统里的假脱机程序（输入/输出进程）；用户作业只管往井里放、从井里取——说"由用户作业控制传送"是错的。
- "独占设备"和"共享设备"是两回事：打印机一次只能打一个作业的，是独占的；SPOOLing 让它看起来能被多个作业共享——"共享"是目的，不是错误。
- "需要外存"和"需要多道"是两回事：输入井/输出井在外存上→要外存；假脱机程序和用户作业同时在内存→要多道——两个"需要"都是对的。

井和设备之间的数据传送由系统假脱机程序控制，不是用户作业；D 说由用户作业控制，是错的——本题问的就是错误项，选 D。

【坑位】

问"错误的是"时，先把对的划掉，别顺手选了对的。

最容易踩的坑是选 A 或 B：觉得"需要外存的支持""需要多道程序设计技术的支持"说得太绝对。为什么错：输入井和输出井就在外存上，没有外存没地方放井——A 对；假脱机程序要和用户作业同时在内存里，一个搬数据、一个算，没有多道跑不起来——B 对；C"让多个作业共享一台独占设备"正是 SPOOLing 的目的——也对。正确做法：A、B、C 逐条对定义都成立，剩下 D"由用户作业控制传送"把系统的活安到用户头上，就是错误项。

【类比】

SPOOLing 像文印室的收发员：你把文件交到收发窗口（输入井），收发员按顺序拿去复印（设备），印好了放进取件格（输出井），你过会儿来取。复印机和文件之间的搬运是收发员干的，不是你自己站在复印机旁边一张一张搬——收发员就是系统假脱机程序。

【口诀】 井在外存上，多道来帮忙；独占变共享，搬运系统扛。

【考频】 「SPOOLing技术」在 2009–2022 年解析中出现 1 次 · 低频

第 32 题 · 管程相关叙述中错误的一项（正确答案：A）

题干与选项：

下列关于管程的叙述中，错误的是（）。

- A. 管程只能用于实现进程的互斥
- B. 管程是由编程语言支持的进程同步机制
- C. 任何时候只能有一个进程在管程中执行
- D. 管程中定义的变量只能被管程内的过程访问

入手点： 管程不光能互斥，还能同步；条件变量就是干同步的；说“管程只能用于实现进程的互斥”，把同步那一半漏了，就是错的。

专有名词从零解释：

- 先想“管程”：一种编程语言级别的同步机制；把共享数据和操作这些数据的过程（函数）包在一起，组成一个模块；一次只允许一个进程进入管程。
- 再想“互斥”：几个进程不能同时干同一件事；管程一次只进一个进程，天然就互斥了。
- 再想“同步”：几个进程按一定的顺序协作；比如生产者生产了东西，消费者才能消费；生产者和消费者的先后顺序，就是同步。
- 再想“条件变量”：管程里的等待机制；进程在某个条件不满足时，在条件变量上等着；条件满足了，别的进程叫醒它；条件变量就是管程实现同步的工具。

考查知识点：

- 先想 A 为什么错：管程既能实现互斥（一次只进一个进程），也能实现同步（条件变量让进程按顺序协作）；生产者-消费者问题就是用管程的条件变量解决的；说“只能用于互斥”，漏了同步。
- 再想 B 为什么对：管程是由编程语言支持的进程同步机制；程序员用语言提供的管程结构写代码，不用自己去摆弄底层的信号量。
- 再想 C 为什么对：任何时候只能有一个进程在管程中执行；这是管程的定义——管程的门一次只放一个人进。
- 再想 D 为什么对：管程里定义的变量只能被管程内的过程访问；外面不能直接碰管程的数据，要用只能通过管程的过程（函数）来访问；数据被保护起来了。
- 拿生活例子验算：管程像一个单间办公室，门一次只进一个人（互斥）；办公室里还有个等候规则——没货时消费者在条件变量上等着，有货了生产者叫醒他（同步）；只说“办公室只能防两个人同时进”，漏了“等货叫醒”这个功能。
- 边界情况：信号量也能实现互斥和同步，但信号量是散在代码里的 P、V 操作；管程是把数据和操作包在一起的模块，结构更清晰；两者都能干两件事，区别在组织形式。

正确选项为什么对： 管程既能互斥又能同步（条件变量）；说“只能用于互斥”是错的，本题问的就是错误项，选 A。

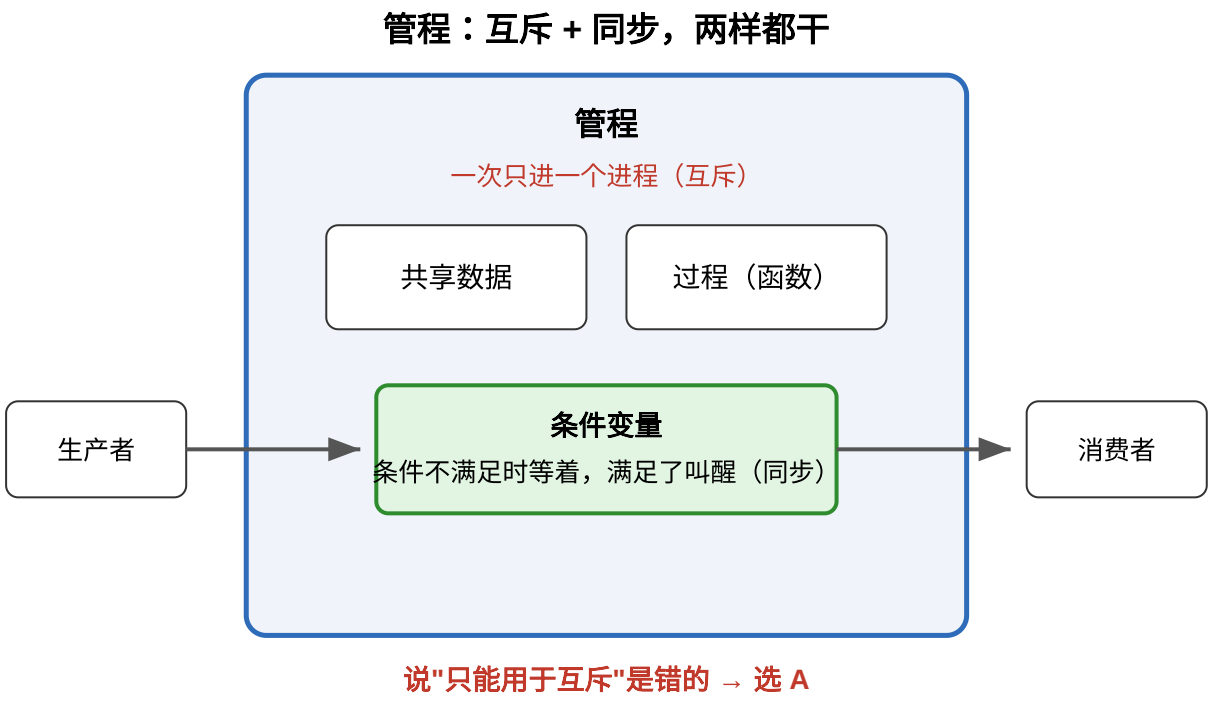
错误选项为什么错＋举例：

- B（管程是由编程语言支持的进程同步机制）：这是对的；管程是语言级别的机制，不是操作系统内核的零散调用。教训：管程和信号量的区别之一就是“语言支持”。
- C（任何时候只能有一个进程在管程中执行）：这是对的；这是管程互斥的定义。教训：一次只进一个，是管程的门规。

- D（管程中定义的变量只能被管程内的过程访问）：这是对的；数据被封装在管程里，外面碰不到。教训：封装是管程保护数据的办法。

笔记本一句话： 第 32 题＋管程：既能互斥（一次进一个），又能同步（条件变量）；说只能互斥是错的。

【图像】



管程示意图：一次只进一个进程，共享数据加过程封装，条件变量实现同步

【定位】 操作系统 · 第2章 · 进程同步（管程）

【易混对比】

- "互斥"和"同步"是两回事：互斥是一次只干一个（管程一次只进一个进程）；同步是按顺序协作（条件变量让生产者/消费者排好先后）——管程两样都干，说"只能互斥"是错的。
- "管程"和"信号量"是两回事：管程是语言级别的模块（共享数据和过程包在一起）；信号量是散在代码里的 P、V 操作——都能实现互斥和同步，区别在组织形式。
- "条件变量"和"锁"是两回事：锁管"一次进一个"（互斥）；条件变量管"条件不满足时等着、满足了叫醒"（同步）——管程里两个都有。

管程既能互斥（一次只进一个进程），又能同步（条件变量）；A 说"只能用于互斥"，漏了同步——本题问的就是错误项，选 A。

【坑位】

看到"只能"先警惕：管程的本事不止互斥，条件变量就是干同步的。

最容易踩的坑是把 B、C、D 当错的：B"管程是由编程语言支持的进程同步机制"——管程是语言级别的机制，对；C"任何时候只能有一个进程在管程中执行"——这是管程互斥的定义，对；D"管程中定义的变量只能被管程内的过程访问"——数据被封装在管程里，对。正确做法：生产者-消费者问题就是用管程的条件变量解决的同步例子，记住"管程=互斥+同步"，A 的"只能"一出来就是错的。

【类比】

管程像一个单间办公室：门一次只进一个人（互斥）；办公室里还有等候规则——没货时消费者在条件变量上等着，有货了生产者叫醒他（同步）。只说"办公室只能防两个人同时进"，漏了"等货叫醒"这个功能，就是 A 的错误。

【口诀】 管程是个单间房，一次只进一个人；条件变量管同步，互斥同步全能干。

【考频】 「管程同步机制」在 2009–2022 年解析中出现 3 次 · 中频

第 33 题 · OSI 参考模型中 R1、Switch、Hub 能实现的最高功能层（正确答案：C）

题干与选项：

题 33～40 均依据题 33～41 图回答。

在 OSI 参考模型中，R1、Switch、Hub 实现的最高功能层分别是（ ）。

- A. 2、2、1
- B. 2、2、2
- C. 3、2、1
- D. 3、2、2

> 注：题 33～40 均依据同一幅网络拓扑图（原卷"题 33～41 图"）作答，插图以原卷为准。

入手点： 先按设备认层：集线器只会原样转发比特，是物理层；交换机按 MAC 地址转发帧，是数据链路层；路由器按 IP 地址转发分组，是网络层；三个设备的最高层排出来就是答案。

专有名词从零解释：

- 先想“OSI 参考模型”：把网络功能从下往上分成 7 层——物理层（第 1 层）、数据链路层（第 2 层）、网络层（第 3 层）、传输层（第 4 层）、会话层、表示层、应用层；下层给上层服务。
- 再想“物理层（第 1 层）”：传比特流；不管比特是什么意思，只管把 0、1 信号送过去；集线器就干这个。
- 再想“数据链路层（第 2 层）”：传帧；帧里有 MAC 地址；交换机看帧的目的 MAC 地址，决定从哪个口转发；同一局域网里的设备靠它互通。
- 再想“网络层（第 3 层）”：传 IP 分组；分组里有 IP 地址；路由器看目的 IP 地址，决定往哪个网络转发；不同网络之间靠它互通。
- 再想“最高功能层”：设备能处理到的最高的那一层；比如路由器能处理到网络层，但它也包含下面的物理层和数据链路层功能；最高层是网络层。

考查知识点：

- 先想 Hub：集线器收到比特流，就原样从所有口广播出去；它不看 MAC 地址、不看 IP 地址；它只在物理层干活，最高层是第 1 层。
- 再想 Switch：交换机看帧的目的 MAC 地址，查表决定从哪个口转发；它处理到了数据链路层；它也包含物理层功能（收发比特），但最高层是第 2 层。
- 再想 R1：路由器看 IP 分组的目的 IP 地址，查路由表决定往哪个网络转发；它处理到了网络层；它也包含下面的物理层和数据链路层功能，但最高层是第 3 层。
- 再想顺序：R1 是 3，Switch 是 2，Hub 是 1；按 R1、Switch、Hub 的顺序写就是 3、2、1。
- 拿生活例子验算：Hub 像个大喇叭，听到什么就原样喊给所有人（物理层）；Switch 像按座位号送信的信使，看信封上的座位号送到那个人（数据链路层）；路由器像按城市地址分拣的邮局，看信封上的城市决定往哪个城市发（网络层）。
- 边界情况：三层交换机能处理到网络层，但本题的 Switch 是普通的 100Base-T 交换机（第 2 层设备），不是三层交换机；别把设备名字和层数记混，要看题目给的设备类型。

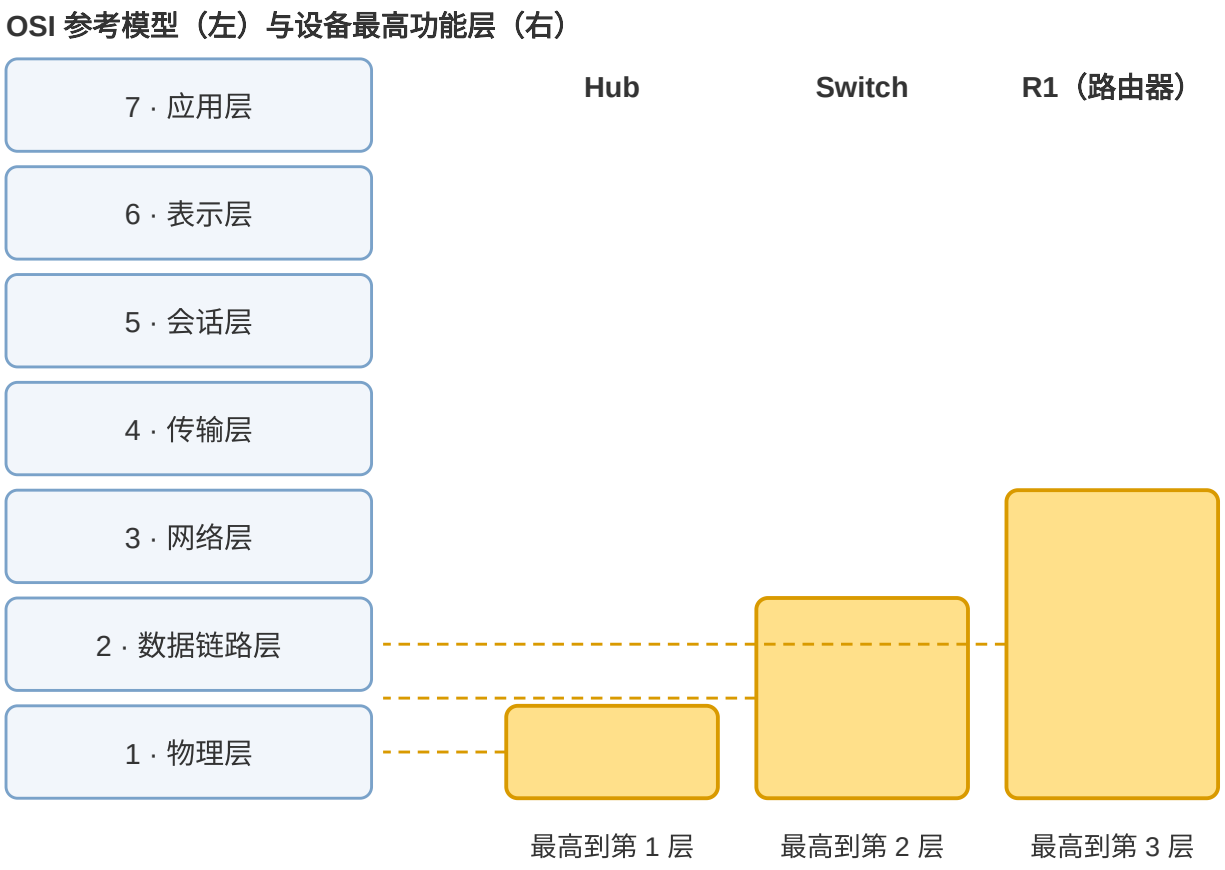
正确选项为什么对： R1 是路由器（第 3 层）、Switch 是交换机（第 2 层）、Hub 是集线器（第 1 层），顺序是 3、2、1，就是 C，选 C。

错误选项为什么错＋举例：

- A（2、2、1）：把 R1 写成第 2 层；路由器是按 IP 地址转发的网络层设备，不是按 MAC 地址转发的；写成 2 层就低了一层。教训：路由器看 IP 地址，是第 3 层。
- B（2、2、2）：把 R1 和 Hub 都写成第 2 层；路由器是第 3 层，集线器是第 1 层；两个都写错了。教训：三个设备各是一层，别写成一样的。
- D（3、2、2）：把 Hub 写成第 2 层；集线器不看 MAC 地址，只广播比特，是第 1 层。教训：集线器没有查表转发的功能，就是物理层。

笔记本一句话： 第 33 题＋设备认层：Hub 物理层 1、Switch 数据链路层 2、路由器网络层 3；R1、Switch、Hub 是 3、2、1。

【图像】



设备同时具备其下各层功能；按 R1、Switch、Hub 顺序即 3、2、1

OSI 七层示意图：Hub 最高到第 1 层物理层，Switch 最高到第 2 层数据链路层，R1 路由器最高到第 3 层网络层

【定位】 计算机网络 · 第1章 · OSI 参考模型与设备功能层

【易混对比】

- "最高功能层"和"只能工作在这一层"是两回事：最高层指设备能处理到的最高一层，它同时包含下面各层的功能；路由器最高在网络层，但收发比特的物理层活儿它也干，别以为路由器只在第 3 层。
- "普通交换机"和"三层交换机"是两回事：普通交换机按 MAC 地址转发，是第 2 层设备；三层交换机还能按 IP 路由，最高到第 3 层；本题的 Switch 是普通交换机，只能按 2 层算。
- "R1、Switch、Hub 的提问顺序"和"1、2、3 的数值顺序"是两回事：题干按 R1、Switch、Hub 的顺序问，答案要写成 3、2、1；写成 1、2、3 就是把顺序搞反了。

按"查什么地址"认层：Hub 只管比特广播是第 1 层，Switch 按 MAC 转发是第 2 层，R1 按 IP 路由是第 3 层；按 R1、Switch、Hub 的顺序就是 3、2、1，选 C。

【坑位】

顺序别写反，题干问的是 R1、Switch、Hub，不是 Hub、Switch、R1。

最容易踩的坑是把 1、2、3 当成答案：1、2、3 是 Hub、Switch、路由器各自的层数排法，而题干的提问顺序是 R1、Switch、Hub，答案要按这个顺序写成 3、2、1。为什么错：答案的顺序必须和题干的提问顺序一致，设备认层只是中间步骤。正确做法：先给每个设备认层（R1=3、Switch=2、Hub=1），再按题干顺序抄出来；抄之前默念一遍题干的顺序。

【套路】

- 第一步，看设备查什么地址：不查地址、只广播比特→物理层；查 MAC 地址→数据链路层；查 IP 地址→网络层。
- 第二步，给设备定最高层：Hub 是 1，Switch 是 2，路由器是 3。
- 第三步，按题干顺序输出：题干问 R1、Switch、Hub，就写 3、2、1；写完回头对一遍题干顺序。

认层看地址，按题干顺序写答案。

【类比】

Hub 像个大喇叭，听到什么就原样喊给所有人，只管比特不管内容（第 1 层）；Switch 像按座位号送信的信使，看信封上的座位号（MAC 地址）送到那个人（第 2 层）；路由器像按城市分拣的邮局，看信封上的城市（IP 地址）决定发往哪个城市（第 3 层）。喇叭只管喊，信使认座位，邮局看城市——查的东西越"高级"，层数越高。

【口诀】 Hub 一层只管喊，Switch 二层认 MAC，路由三层看 IP；题干顺序三二一，认层认址再输出。

【考频】 「OSI 参考模型设备功能层」在 2009–2022 年解析中出现 6 次 · 中频

第 34 题 · R2 与 R3 链路带宽 8kHz 信噪比 30dB 实际速率约为理论最大速率 50% 时的实际数据传输速率（正确答案：C）

题干与选项：

若连接 R2 和 R3 链路的频率带宽为 8kHz，信噪比为 30dB，该链路实际数据传输速率约为理论最大数据传输速率的 50%，则该链路的实际数据传输速率约是（）。

- A. 8kbps
- B. 20kbps
- C. 40kbps
- D. 80kbps

> 注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： 先用香农定理算理论最大速率——带宽乘以 log2（1＋信噪比倍数）；30dB 换算成倍数是 1000 倍；算出来约 80kbps；再乘 50%，就是 40kbps。

专有名词从零解释：

- 先想“带宽”：信道能通过的频率范围；本题是 8kHz，就是 8000Hz；带宽越大，能传的数据越多。
- 再想“信噪比”：信号功率和噪声功率的比；30dB 是用分贝表示的比值；换算成倍数是 10 的（30÷10）次方＝1000 倍。
- 再想“香农定理”：有噪声信道的理论最大数据传输速率＝带宽×log2（1＋信噪比倍数）；这是理论上限，实际速率到不了这么高。
- 再想“理论最大速率和实际速率”：理论值是理想上限；题目说实际约是理论的 50%，就是理论值打五折。

考查知识点：

- 先换算信噪比：30dB 对应的倍数是 10^(30/10)＝10^3＝1000；所以 1＋信噪比＝1001。
- 再算 log2(1001)：2 的 10 次方是 1024，1001 接近 1024，所以 log2(1001) 约等于 10（略小于 10，约 9.97）。
- 再算理论最大速率：8000×9.97 ≈ 79760bps，约 80kbps。
- 再算实际速率：80kbps×50%＝40kbps。
- 拿小例子验算：带宽 1kHz、信噪比 30dB 时，理论速率约 10kbps，实际 50% 就是 5kbps；带宽翻 8 倍到 8kHz，理论速率约 80kbps，实际 40kbps；规律一样——带宽和速率成正比。
- 边界情况：信噪比用 dB 表示时，换算倍数是 10^(dB/10)，不是 10^(dB/20)；后者是电压比的换算，功率比用前者；本题是功率比，用 10^(dB/10)。

正确选项为什么对： 理论最大速率约 80kbps，实际约 50% 就是 40kbps，就是 C，选 C。

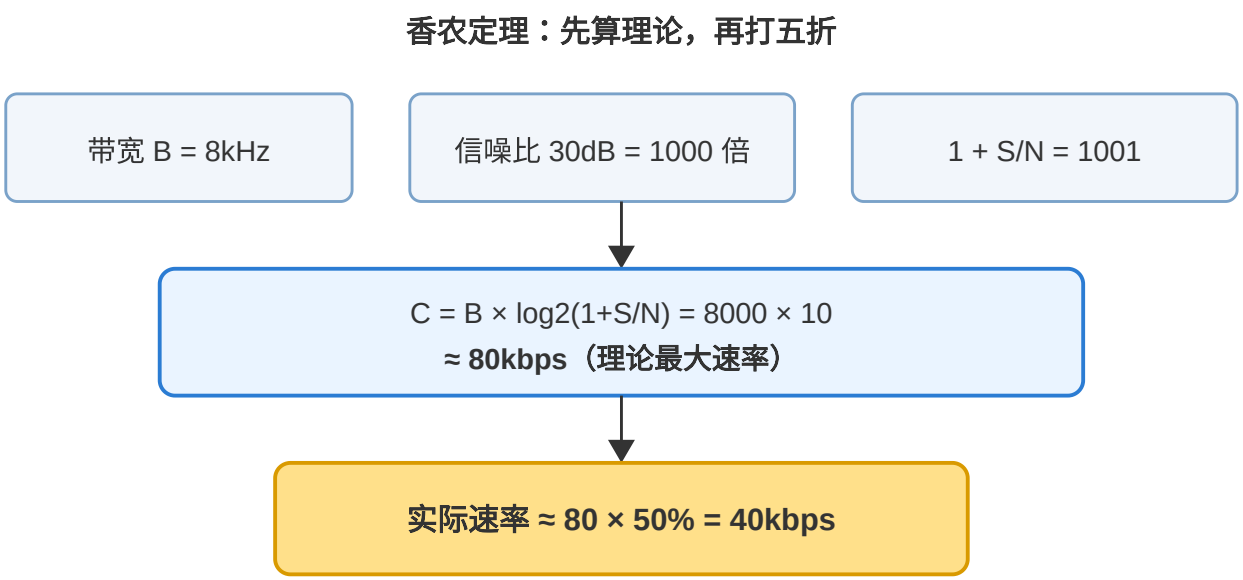
错误选项为什么错＋举例：

- A（8kbps）：8kbps 是带宽的数值（8kHz），不是速率；把带宽直接当速率，漏了乘 log2(1＋信噪比）这一步。教训：带宽是 Hz，速率是 bps，中间隔着一个 log。
- B（20kbps）：20kbps 是把理论速率算成 40kbps 再打五折，或者把 50% 用了两次；理论速率约 80kbps，打一次五折是 40kbps。教训：50% 只乘一次。

- D（80kbps）：80kbps 是理论最大速率，没打五折；题目问的是实际速率，实际约是理论的 50%。教训：问实际速率时，理论值要乘 50%。

笔记本一句话： 第 34 题＋香农定理：速率＝带宽×log2（1＋信噪比）；30dB＝1000 倍，理论约 80kbps，实际 50%＝40kbps。

【图像】



香农定理示意图：带宽 8kHz 与信噪比 30dB 经香农定理得理论最大速率约 80kbps，取 50% 得实际速率约 40kbps

【定位】 计算机网络 · 第2章 · 物理层信道极限容量（香农定理）

【易混对比】

- "带宽"和"数据传输速率"是两回事：带宽单位是 Hz，速率单位是 bps；8kHz 是信道的频率宽度，不是 8kbps；两者中间隔着一个 $log_2(1+S/N)$，别直接画等号。
- "30dB 换成倍数"和"把 30 直接代入"是两回事：dB 是对数单位，功率比换成倍数是 $10^{30/10}=1000$ 倍；直接把 30 塞进公式，数量级全错。
- "理论最大速率"和"实际速率"是两回事：理论值是香农定理给的上限，实际只能到约 50%；问实际速率时，理论值必须打五折。

30dB 换成 1000 倍，log2(1001) 约 10，理论速率约 8000×10=80000bps 即 80kbps；实际约 50% 就是 40kbps，选 C。

【坑位】

50% 只乘一次，别把理论值当答案，也别打两次折。

最容易踩的坑有三处：一是 dB 换算用了 $10^{dB/20}$（那是电压比的换法），功率比要用 $10^{dB/10}$；二是把 8kHz 直接当成 8kbps，漏了乘对数这一步；三是算出 80kbps 就停了，忘了题干问的是"实际速率约是理论最大速率的 50%"。为什么错：三个都是把"中间步骤"当成了"最终答案"。正确做法：先 dB 转倍数（30→1000），再用 $2^{10}=1024\approx1001$ 把对数认成约 10，然后理论值打五折。

【套路】

- 第一步，dB 转倍数：功率比的倍数 $=10^{dB/10}$；30dB→1000 倍，公式里用 $1+1000=1001$。

- 第二步，对数凑整：找最接近的 2 的整次幂，$2^{10}=1024$，所以 $log_2(1001)$ 约 10。
- 第三步，算理论速率：带宽（Hz）× 对数，本题 $8000\times 10=80$kbps。
- 第四步，按题干比例折算：本题 50%，$80\times 0.5=40$kbps；问实际就折，问理论就不折。

dB 转倍数、对数凑整、理论乘带宽、实际按比折。

【类比】

像水管：管子有多粗（带宽）决定理论上每秒能过多少水；水压够不够、杂质多不多（信噪比）决定水流能冲多快；但水龙头平时不会拧到最大，实际出水只有理论上限的一半左右。问"实际出水"时，先把管子宽度×水压冲速算出理论上限，再打折；直接报理论上限就是没看清题干。

【口诀】 三十分贝一千倍，对数凑十带宽乘；理论八十打五折，实际四十记分明。

【考频】 「香农定理信噪比」在 2009–2022 年解析中出现 4 次 · 中频

第 35 题 · H2 向 H4 发数据帧 H4 立即回确认帧时除 H4 外在物理层还能收到该确认帧的主机（正确答案：D）

题干与选项：

若主机 H2 向主机 H4 发送 1 个数据帧，主机 H4 向主机 H2 立即发送一个确认帧，则除 H4 外，从物理层上能够收到该确认帧的主机还有（）。

- A. 仅 H2
- B. 仅 H3
- C. 仅 H1，H2
- D. 仅 H2，H3

> 注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： 确认帧从 H4 出发，先经过 Hub、再经过 Switch、最后到 H2；Hub 是广播的（所有口都收到），Switch 是按 MAC 地址转发的（只送到 H2 那个口）；所以物理层上收到确认帧的，除了 H4，还有 H3（Hub 那边）和 H2（最终目的地）。

专有名词从零解释：

- 先想“数据帧”：数据链路层传的数据单位；帧里有源 MAC 地址和目的 MAC 地址。
- 再想“确认帧”：收到数据帧后回的一个小帧，意思是“我收到了”；本题 H4 收到 H2 的数据帧后，马上回确认帧。
- 再想“Hub（集线器）”：物理层设备；收到比特流就从所有口广播出去；挂在 Hub 上的所有主机，在物理层上都能收到。
- 再想“Switch（交换机）”：数据链路层设备；看帧的目的 MAC 地址，查表后只从对应的口转发；不该收的主机收不到。
- 再想“物理层上收到”：信号实际到达了这台主机的网卡；不管这台主机最后处不处理这个帧，只要信号到了，就算物理层上收到了。

考查知识点：

- 先想确认帧的路：H4 发出确认帧，先到 Hub；Hub 把比特流广播到所有口——一头是 H3，另一头是连向 Switch 的口；所以 H3 在物理层上收到了，Switch 也收到了。
- 再想 Switch 的转发：Switch 收到确认帧后，看目的 MAC 地址是 H2，查表知道 H2 在哪个口，只从那个口转发给 H2；H1 挂在 Switch 的另一个口上，Switch 不会把帧发给 H1；所以 H1 在物理层上收不到。
- 再想 H2：H2 是确认帧的目的地，Switch 把帧转发给 H2，H2 当然收到了。
- 再想 H3：H3 挂在 Hub 上，Hub 广播时 H3 的口也收到了比特流；虽然确认帧的目的地不是 H3，H3 的数据链路层会把它丢掉，但在物理层上，H3 是收到了的。
- 再想 H4 自己：题目问“除 H4 外”，H4 是发送方，不算。
- 拿生活例子验算：H4 在 Hub 这边喊了一嗓子（广播），Hub 这边所有人都听见了（H3）；喊声传到 Switch 那边，Switch 只把话传给 H2 一个人（按地址转发），H1 听不见。
- 边界情况：如果 H4 和 H2 之间没有 Hub、只有 Switch，那么只有 H2 能收到；本题有 Hub 在中间，Hub 那边的主机都会在物理层上收到；题目问的是“物理层上”，不是“数据链路层上处理”，所以 H3 算。

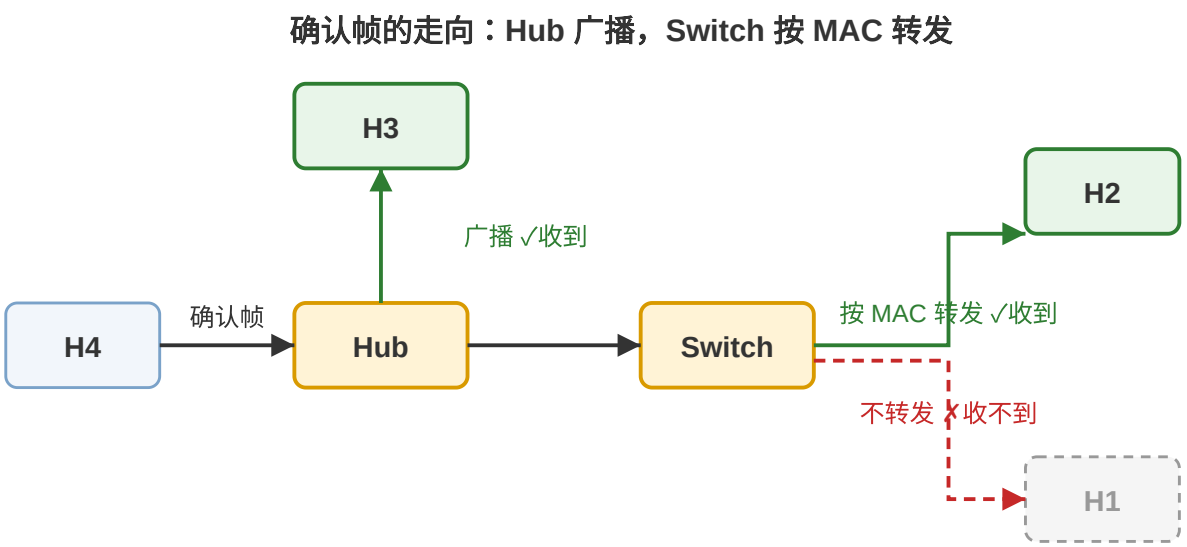
正确选项为什么对： 确认帧经 Hub 广播到 H3、经 Switch 转发到 H2；除 H4 外，物理层上收到的是 H2 和 H3，就是 D，选 D。

错误选项为什么错＋举例：

- A（仅 H2）：只算了最终目的地，漏了 Hub 广播的 H3；Hub 是广播的，挂在 Hub 上的 H3 在物理层上也收到了。教训：看到 Hub 先想广播，挂在 Hub 上的主机都会在物理层上收到。
- B（仅 H3）：只算了 Hub 广播，漏了最终目的地 H2；确认帧是回给 H2 的，Switch 会把它转发给 H2。教训：广播和转发都要算，目的地和广播范围都要列。
- C（仅 H1、H2）：H1 挂在 Switch 的另一个口上，Switch 按 MAC 地址转发，不会把帧发给 H1；H1 收不到；而且漏了 Hub 那边的 H3。教训：Switch 不广播，只按地址转发；Hub 才广播。

笔记本一句话： 第 35 题＋确认帧的物理层范围：Hub 广播到 H3，Switch 按地址转发到 H2；除 H4 外是 H2、H3。

【图像】



除 H4 外，物理层上收到确认帧的是：H2、H3（H1 收不到）

确认帧走向示意图：H4 发出的确认帧经 Hub 广播到 H3，并经 Switch 按 MAC 地址转发到 H2

【定位】 计算机网络 · 第3章 · Hub 与 Switch 的转发范围（局域网设备）

【易混对比】

- "物理层收到"和"数据链路层处理"是两回事：物理层收到只要求信号到了网卡；H3 在物理层上收到了确认帧，虽然它的数据链路层会把这个帧丢掉；题干问的是物理层，别用"处不处理"来判。
- "Hub 的广播"和"Switch 的转发"是两回事：Hub 收到比特就从所有口复制出去，挂在 Hub 上的主机一个都跑不掉；Switch 只看目的 MAC 地址，只送到对应的那个口，别的口收不到。
- "确认帧的目的地"和"确认帧的物理范围"是两回事：目的地只有 H2 一个，但物理范围还包括 Hub 广播覆盖到的 H3；只答目的地会漏掉 H3。

确认帧从 H4 出发，经 Hub 广播到 H3、经 Switch 按目的 MAC 转发到 H2；除 H4 外，在物理层上收到的是 H2 和 H3，选 D。

【坑位】

题干问的是"物理层上"收到，别把链路层"处理"当成收到，也别把 Switch 那边的 H1 算进来。

最容易踩的坑有两处：一是只答目的地 H2，漏了 Hub 广播的 H3——看到 Hub 先想"广播"，挂在 Hub 上的所有主机在物理层上都会收到；二是把 H1 算进来——H1 挂在 Switch 的另一个口上，Switch 按 MAC 地址转发，不会把帧复制给 H1。为什么错：前者忘了 Hub 是广播设备，后者把 Switch 当成了 Hub。正确做法：先画信号的路（H4→Hub→{H3，Switch}→H2），再把 Hub 广播的口和 Switch 转发的口各列一遍。

【套路】

- 第一步，画信号的路：从发送方开始，依次写出经过的设备，本题 H4→Hub→Switch→H2。
- 第二步，按设备类型展开：过 Hub 就把挂在 Hub 上的所有口列出来（广播）；过 Switch 就只列目的 MAC 对应的那个口（按址转发）。
- 第三步，按题干口径收口：问物理层就把广播到的口全算上，问"除 H4 外"就把发送方去掉。

画路、按设备展开（Hub 列全部口、Switch 只列目的口）、按口径收口。

【类比】

H4 在 Hub 这边喊了一嗓子（广播），Hub 这边所有人都听见了（H3）；喊声传到 Switch 那边，Switch 像前台接待，只把话转给预约好的客人（H2），没预约的（H1）听不见。物理层上"听见"就行，不管听完处不处理——H3 虽然听完就扔了，但它确实听见了。

【口诀】 Hub 广播全都能听见，Switch 只送有缘人；H2 目的地 H3 广播到，H1 隔岸听不见。

【考频】 「Hub 广播 Switch 转发」在 2009–2022 年解析中出现 13 次 · 高频

第 36 题 · Hub 再生比特流延迟 1.535μs 信号传播速度 200m/μs 时 H3 与 H4 间理论上可相距的最远距离（正确答案：B）

题干与选项：

若 Hub 再生比特流过程中，会产生 1.535μs 延时，信号传播速度为 200m/μs，不考虑以太网帧的前导码，则 H3 与 H4 之间理论上可以相距的最远距离是（）。

- A. 200m
- B. 205m
- C. 359m
- D. 512m

> 注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： 以太网有个规矩——发送方要在收到冲突信号之前把整帧发完（不然就检测不到冲突）；这个时间就是时槽；时槽要留给“信号跑过去、Hub 再生、再跑回来”；把时槽减去 Hub 的延迟，剩下的时间换算成距离，就是最远距离。

专有名词从零解释：

- 先想“时槽（slot time）”：以太网里检测冲突用的时间单位；100Mbps 以太网的时槽是 512 位，也就是 512÷100Mbps＝5.12μs；发送方发完 512 位之前，必须能听到冲突。
- 再想“冲突检测”：两台主机同时发，信号撞在一起叫冲突；发送方要在发完之前听到冲突，才能知道要重发；如果信号跑得太远，发送方发完了还没听到冲突，冲突就检测不到了。
- 再想“Hub 再生比特流”：Hub 收到比特流后，要重新整形、放大再发出去；这个过程要花时间，本题是 1.535μs；信号每过一个 Hub，就要多花这么多时间。
- 再想“往返”：信号从一头跑到另一头，再跑回来，跑的是两倍的距离；算距离时，时间要除以 2。

考查知识点：

- 先算时槽：100Mbps 以太网，512 位÷100Mbps＝5.12μs；这是留给“信号跑过去、Hub 再生、再跑回来”的总时间。
- 再算 Hub 的延迟：H3 和 H4 之间有两个 Hub（信号从 H3 出发过一个 Hub、从 H4 出发过一个 Hub，往返时各算一次？按题目的算法，往返路上一共算 2 个 Hub 的延迟）；2×1.535μs＝3.07μs。
- 再算留给传播的时间：5.12μs－3.07μs＝2.05μs；这是信号在介质里跑的时间（往返）。
- 再算距离：往返时间 2.05μs，速度 200m/μs，往返路程是 2.05×200＝410m；单程距离是 410÷2＝205m。
- 拿小例子验算：如果没有 Hub 延迟，5.12μs 全用来跑，往返路程是 5.12×200＝1024m，单程 512m；这就是 100Mbps 以太网不加 Hub 时的理论最大距离；加了 Hub，距离就缩短了。
- 边界情况：题目说“不考虑以太网帧的前导码”，意思是前导码的 64 位不算进时槽的 512 位里；如果算上，时间会多一点，距离会大一点；本题不算，按 512 位算。

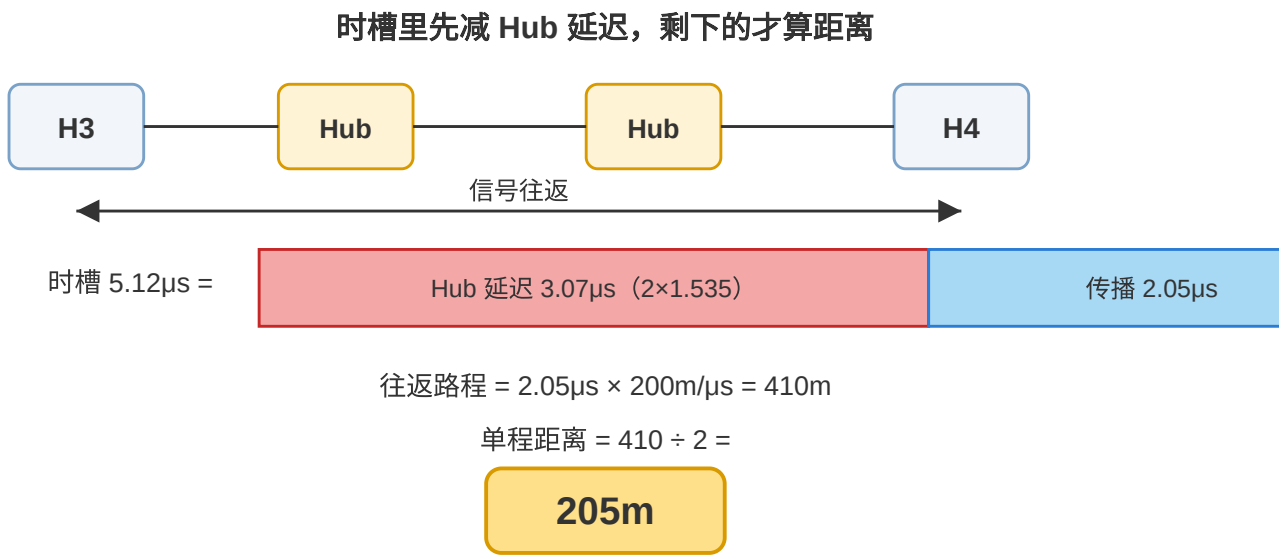
正确选项为什么对： 时槽 5.12μs 减去 2 个 Hub 的延迟 3.07μs，剩 2.05μs；往返路程 410m，单程 205m，就是 B，选 B。

错误选项为什么错＋举例：

- A（200m）：200m 是把 2.05μs 当成单程时间算的（2.05×200÷2＝205，不是 200）；或者把 Hub 延迟算错了。教训：往返时间换算成单程距离要除以 2，数字要算准。
- C（359m）：359m 是把 Hub 延迟只算了一个（5.12－1.535＝3.585μs，3.585×200÷2＝358.5m≈359m）；往返路上有 2 个 Hub 的延迟，不是 1 个。教训：信号往返要过两个 Hub，延迟算两次。
- D（512m）：512m 是没减 Hub 延迟的结果（5.12×200÷2＝512m）；Hub 再生要花时间，不减就多算了。教训：时槽里要先减去设备延迟，剩下的才是传播时间。

笔记本一句话： 第 36 题＋以太网最远距离：时槽 5.12μs－2 个 Hub 延迟 3.07μs＝2.05μs；往返 410m，单程 205m。

【图像】



时槽分配示意图：总时槽 5.12 微秒中两个 Hub 延迟占 3.07 微秒，剩余 2.05 微秒对应往返 410 米、单程 205 米

【定位】 计算机网络 · 第3章 · CSMA/CD 时槽与冲突域最远距离

【易混对比】

- "往返路程"和"单程距离"是两回事：信号从一头跑到另一头再回来，跑的是两倍距离；用往返时间算出往返路程后，必须除以 2 才是 H3 与 H4 之间的距离。
- "2 个 Hub 的延迟"和"1 个 Hub 的延迟"是两回事：信号往返一趟要经过两个 Hub 的再生过程，延迟要算两次（$2 \times 1.535\mu s = 3.07\mu s$）；只减一次会多算出约 359m 的错误答案。
- "时槽的 512 位"和"帧前导码"是两回事：题干说了不考虑前导码，时槽就按 512 位算（$5.12\mu s$）；前导码的 64 位不进这个账。

时槽 5.12μs 减去 2 个 Hub 的延迟 3.07μs，剩 2.05μs 给信号跑；往返路程 410m，单程就是 205m，选 B。

【坑位】

Hub 延迟算两次、往返路程除以 2，这两个数一个都不能省。

最容易踩的坑是三选一的数字陷阱：512m 是没减 Hub 延迟的结果，359m 是只减了一个 Hub 延迟的结果，200m 是把往返时间当成单程算错的结果。为什么错：以太网的规矩是发送方要在发完之前听到冲突，时槽要留给"信号跑过去、Hub 再生、再跑回来"——设备延迟和往返折半都不能省。正确做法：先算时槽（100Mbps 下

512 位 $= 5.12\mu s$），再减 2 个 Hub 的延迟（$3.07\mu s$），剩下的 $2.05\mu s$ 先乘速度得往返 410m，最后除以 2 得 205m。

【套路】

- 第一步，算时槽：100Mbps 以太网时槽 = 512 位 ÷ 100Mbps $= 5.12\mu s$。
- 第二步，减设备延迟：信号往返经过几个 Hub 就减几次，本题 $5.12 - 2 \times 1.535 = 2.05\mu s$。
- 第三步，换算成单程距离：剩余时间 × 传播速度得往返路程，再除以 2 得单程距离，本题 $2.05 \times 200 \div 2 = 205$m。

算时槽、减设备延迟（往返算两次）、往返换单程除以 2。

【类比】

两个人隔着两道门传话，规矩是"喊完一句话之前必须听见对方喊'撞了'"。喊话时间固定（时槽），两道门各耽误一点时间（Hub 延迟），剩下的时间才是声音跑路的时间；声音要跑一趟去、再跑一趟回，所以两个人之间能隔的距离是声音跑路总距离的一半。门越多、耽误越久，两个人就得站得越近。

【口诀】 时槽减去 Hub 延，剩下时间算路程；往返除以二，205 米最远门。

【考频】 「CSMA/CD 时槽最远距离」在 2009–2022 年解析中出现 5 次 · 中频

第 37 题 · R1、R2、R3 用 RIP 交换路由信息且均已收敛后 R3 发现目标网络不可达并通告一次新距离向量后 R2 更新的距离（正确答案：B）

题干与选项：

假设 R1、R2、R3 采用 RIP 协议交换路由信息，且均已收敛。若 R3 检测到网络 201.1.2.0/25 不可达，并向 R2 通告一次新的距离向量，则 R2 更新后，其到达该网络的距离是（）。

- A. 2
- B. 3
- C. 16
- D. 17

注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： RIP 有个毛病叫“坏消息传得慢”；R3 说目标网络不可达（距离 16），但 R2 手里还有 R1 给的旧消息（R1 到该网络的距离是 2）；R2 按 RIP 的规则“旧消息加 1 跳”更新，得到 3；这个 3 是过时的、错的，但这就是 RIP 通告一次后的结果。

专有名词从零解释：

- 先想“RIP 协议”：一种距离向量路由协议；路由器之间定期交换“我到各个网络有多远”；距离用跳数算，经过一个路由器加 1。
- 再想“距离向量”：路由器发给邻居的一张表，写着“我到每个网络的距离”；邻居收到后，把自己的距离更新成“邻居的距离＋1”。
- 再想“收敛”：所有路由器的表都不再变化，稳定了；本题一开始就是收敛状态。
- 再想“16 代表不可达”：RIP 规定距离 16 就是“到不了”；超过 15 跳的网络，RIP 认为不可达；所以 R3 通告的“不可达”就是距离 16。
- 再想“坏消息传得慢”：好消息（距离变小）传得快，坏消息（距离变大、不可达）传得慢；因为路由器会拿邻居的旧消息当新的用，距离一次只加 1，慢慢数到 16 才承认不可达。

考查知识点：

- 先想收敛时的距离：目标网络挂在 R3 下面；R1 到该网络经过 R3，是 2 跳（R1→R3→网络）；R2 到该网络也经过 R3，是 2 跳（R2→R3→网络）；三台路由器都收敛了，表里都是这个数。
- 再想 R3 的通告：R3 发现目标网络不可达，向 R2 通告新的距离向量，里面到该网络的距离是 16（不可达）。
- 再想 R2 的更新规则：R2 收到 R3 的通告（16）后，也会看 R1 给的旧向量——R1 说“我到该网络是 2 跳”；R2 按“邻居距离＋1”的规则，算出经 R1 到该网络是 2＋1＝3 跳；R2 比较“经 R3 的 16”和“经 R1 的 3”，取小的 3，更新自己的表。
- 再想为什么是 3 不是 16：RIP 的规则是“取所有邻居里最小的距离＋1”；R1 的旧消息（2）还没更新，R2 就先用了它；所以通告一次后，R2 的距离是 3，不是 16；这就是坏消息传得慢——R2 要经过多次交换、距离一次一次加 1（4、5、6……），最后数到 16 才承认不可达。
- 拿生活例子验算：三个人传话，丙说“去北京的路断了”，乙听到后想起甲昨天说“去北京只要 2 小时”，乙就改成“经甲去北京要 3 小时”；其实甲的路也是经过丙的，早就断了；但乙只听到了丙的一次通告，还没听到甲的更新，所以先写了 3。

- 边界情况：如果 R3 通告后，R1 也马上通告了（R1 说“我也到不了”），R2 就会直接更新成 16；但本题只问“R3 通告一次后”，R1 还没更新，R2 就是 3。

正确选项为什么对： R2 按“R1 的旧距离 2＋1 跳”更新，得到 3，就是 B，选 B。

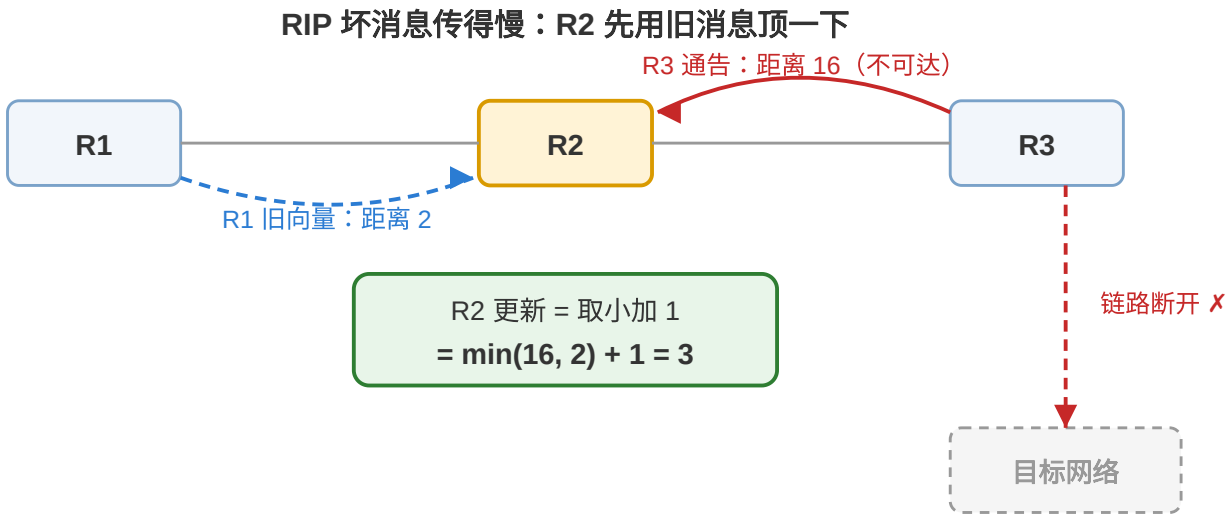
错误选项为什么错＋举例：

- A（2）：2 是 R2 原先的距离（经 R3）；R3 通告不可达后，R2 的表已经变了，不再是 2。教训：通告一次后，表已经更新了，不是原先的数。
- C（16）：16 是 R3 通告的“不可达”；但 R2 没有直接用 16，而是用了 R1 的旧消息算出 3；RIP 取最小的邻居距离＋1，不是直接抄通告的数。教训：坏消息传得慢，R2 先用旧消息顶一下，不会一次就到 16。
- D（17）：17 是在 16 的基础上又加了 1；RIP 的距离上限就是 16，到 16 就停了，不会数到 17；而且 R2 这次更新是 3，不是 16＋1。教训：16 就是不可达的封顶数，不会再加。

> 注：本题图上各路由器到目标网络的原始距离以原卷为准；此处按 R1、R2、R3 两两直连、目标网络挂在 R3 下的标准图推导，标注为“部分核对”。

笔记本一句话： 第 37 题＋RIP 慢收敛：R3 通告 16 后，R2 先用 R1 的旧距离 2＋1＝3；要多次交换才数到 16。

【图像】



一次通告后 R2 先数到 3；要多次交换、一次加 1，才数到 16 承认不可达

收敛时 R1、R2 经 R3 到目标网络的距离均为 2（图上原始距离以原卷为准）

RIP 慢收敛示意图：R3 通告不可达距离 16 后，R2 用 R1 的旧距离 2 加 1 更新为 3

【定位】 计算机网络 · 第4章 · RIP 距离向量与慢收敛

【易混对比】

- "R3 通告的 16"和"R2 更新的 3"是两回事：16 是 R3 说的"不可达"，3 是 R2 按"邻居旧距离加 1"算出来的；R2 不会直接抄通告的数，它只认"最小邻居距离加 1"。
- "好消息"和"坏消息"是两回事：距离变小（好消息）一传就到，距离变大或不可达（坏消息）传得慢——因为邻居的旧消息还能用，距离要一次加 1 慢慢数到 16。
- "16"和"17"是两回事：RIP 里 16 就是不可达的封顶数，不会数到 17；选项里看到 17 直接排除。

R3 通告一次后，R2 手里还有 R1 给的旧距离 2，按"旧距离加 1 跳"更新成 3；坏消息传得慢，R2 这一次只数到 3，选 B。

【坑位】

通告一次不等于一次就数到 16，R2 先用 R1 的旧消息顶一下，更新的是 3。

最容易踩的坑是看到 R3 通告"不可达"就选 16。为什么错：RIP 的更新规则是"取所有邻居里最小的距离加 1"，R1 的旧消息（到目标网络 2 跳）还没更新，R2 就会用它算出 3，而不是抄 R3 的 16。R2 要经过多次交换、距离 4、5、6 一次加 1 慢慢数，最后才数到 16 承认不可达——这就是"坏消息传得慢"。正确做法：先写收敛时的旧距离（R1 和 R2 经 R3 到目标网络都是 2），再看这次通告后 R2 能用的最小邻居距离（R1 的旧 2），加 1 得 3。

【套路】

- 第一步，写收敛时的旧距离：目标网络挂在 R3 下，R1 和 R2 经 R3 到它的距离都是 2。
- 第二步，列通告后可用的邻居距离：R3 的新通告是 16，R1 的旧向量还是 2（它还没更新）。
- 第三步，取小加 1：R2 按"邻居距离取小再加 1"更新，本题取 2 加 1 得 3。

写旧距离、列可用邻居、取小加 1。

【类比】

三个人传话：丙说"去北京的路断了"，乙听到后想起甲昨天说"去北京只要 2 小时"，乙就先改成"经甲去北京要 3 小时"。其实甲的路也是经过丙的，早就断了，但乙只听到了丙的一次通告、还没听到甲的更新，所以先写了 3；乙要听好几次传话、一次加一小时，最后才承认"去不了"——坏消息就是这么传得慢的。

【口诀】 坏消息传得慢，R2 先抄旧邻居；旧二加一得三，数到十六才认输。

【考频】 「RIP 慢收敛」在 2009–2022 年解析中出现 4 次 · 中频

第 38 题 · H3 访问 Web 服务器 S 时 R2 转发出去的封装 HTTP 请求报文的 IP 分组源 IP 与目的 IP（正确答案：D）

题干与选项：

假设连接 R1、R2 和 R3 之间的点对点链路使用 201.1.3.x/30 地址，当 H3 访问 Web 服务器 S 时，R2 转发出去的封装 HTTP 请求报文的 IP 分组的源 IP 地址和目的 IP 地址分别是（）。

- A. 192.168.3.251，130.18.10.1
- B. 192.168.3.251，201.1.3.9
- C. 201.1.3.8，130.18.10.1
- D. 201.1.3.10，130.18.10.1

注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： IP 分组过路由器时，目的 IP 不变、源 IP 可能变；这里 R2 做了地址转换（NAT）：内网地址换成外网地址；所以源 IP 从 H3 的内网地址变成了 R2 外侧接口的地址，目的 IP 还是服务器 S 的地址。

专有名词从零解释：

- 先想“IP 分组”：网络层传的数据单位；分组头里写着源 IP（谁发的）和目的 IP（发给谁）。
- 再想“HTTP 请求报文”：应用层的东西；H3 的浏览器要访问 Web 服务器，就发一个 HTTP 请求；这个请求被包进 IP 分组里传过去。
- 再想“路由器转发”：路由器收到 IP 分组后，看目的 IP、查路由表，从另一个口发出去；转发时，IP 分组的源 IP 和目的 IP一般不变。
- 再想“NAT（网络地址转换）”：内网主机访问外网时，路由器把分组的源 IP 从内网地址换成路由器外侧接口的公网地址；外网服务器回包时，路由器再换回来；这样内网的多台主机可以共用一个公网地址上网。
- 再想“内网地址和公网地址”：192.168.3.251 是内网地址（192.168 开头的是私有地址，外网不认）；201.1.3.10 是 R2 外侧接口的公网地址；130.18.10.1 是 Web 服务器 S 的公网地址。

考查知识点：

- 先想 H3 发出的原始分组：源 IP 是 H3 的地址 192.168.3.251，目的 IP 是服务器 S 的地址 130.18.10.1。
- 再想 R2 的 NAT：分组经过 R2 时，R2 把源 IP 从 192.168.3.251 换成自己外侧接口的地址 201.1.3.10；目的 IP 不变，还是 130.18.10.1。
- 再想为什么目的 IP 不变：NAT 只换源 IP（让外网服务器知道回给谁），不换目的 IP（分组还是要发给 S）；换了目的 IP，分组就送错地方了。
- 再想为什么源 IP 必须换：内网地址 192.168.3.251 在外网是认不出来的；服务器 S 回包时不知道往哪回；换成 R2 的公网地址 201.1.3.10，服务器才能把回包送到 R2，R2 再换回内网地址送给 H3。
- 再想 R2 与 R3 之间的链路地址：这段点对点链路用 201.1.3.x/30 的地址；R2 外侧接口的地址是 201.1.3.10，R3 一侧是 201.1.3.9；所以 NAT 换出来的源 IP 是 201.1.3.10，不是 201.1.3.9。
- 拿生活例子验算：内网主机像公司内部分机（分机号 251），打外线时要通过总机（总机号 201.1.3.10）；外面的人看到的是总机号，回拨也打到总机，总机再转给分机；信封上的收件人（目的 IP）一直没变。
- 边界情况：如果没有 NAT（比如 H3 直接用公网地址），R2 转发时源 IP 和目的 IP 都不变；本题有 NAT，所以源 IP 变了；做题先看图里有没有 NAT。

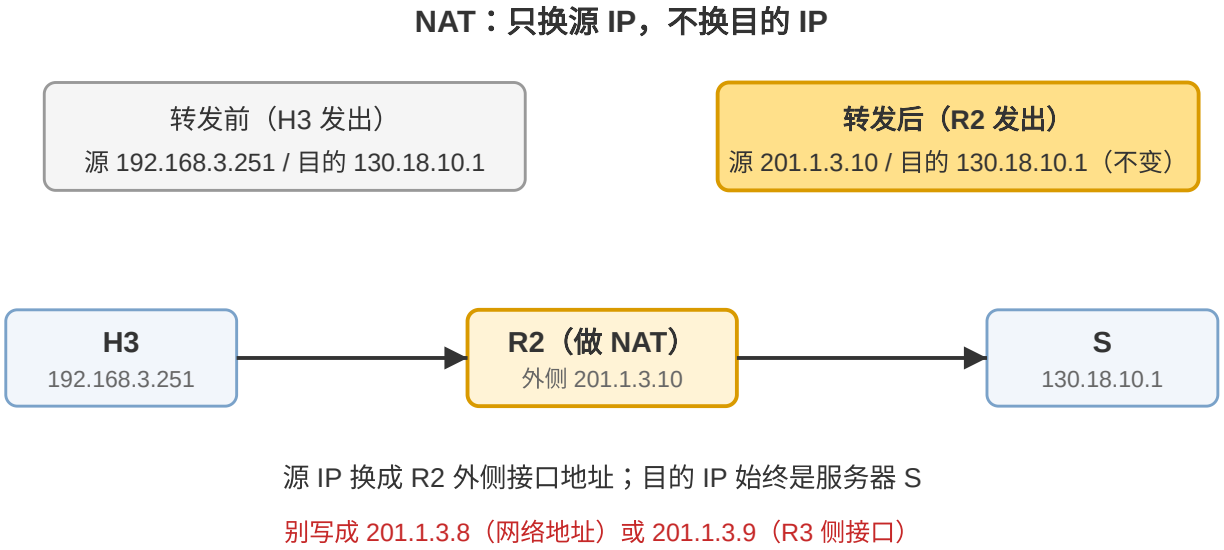
正确选项为什么对： R2 转发出去的分组，源 IP 是 NAT 换出来的 201.1.3.10，目的 IP 还是服务器 S 的 130.18.10.1，就是 D，选 D。

错误选项为什么错＋举例：

- A（192.168.3.251，130.18.10.1）：这是 H3 发出的原始分组，没经过 NAT；R2 转发出去时源 IP 已经换了，不是原来的内网地址。教训：过 NAT 的分组，源 IP 要换成路由器外侧地址。
- B（192.168.3.251，201.1.3.9）：源 IP 没换，还是内网地址；目的 IP 写成了 201.1.3.9（那是 R3 一侧的接口地址，不是服务器 S 的地址）；两个都错了。教训：目的 IP 是最终的服务器，不是中间路由器的接口。
- C（201.1.3.8，130.18.10.1）：源 IP 写成了 201.1.3.8；201.1.3.8 是这段 /30 链路的网络地址，不是 R2 接口的地址；R2 外侧接口的地址是 201.1.3.10。教训：/30 链路里，网络地址和接口地址是两回事，别写成网络地址。

笔记本一句话： 第 38 题＋NAT 转发：源 IP 换成 R2 外侧地址 201.1.3.10，目的 IP 不变（服务器 S 的 130.18.10.1）。

【图像】



NAT 转发示意图：R2 把源 IP 从 192.168.3.251 换成外侧接口地址 201.1.3.10，目的 IP 130.18.10.1 不变

【定位】 计算机网络 · 第4章 · NAT 地址转换

【易混对比】

- "源 IP"和"目的 IP"是两回事：过 NAT 时源 IP 要换成路由器外侧接口地址，目的 IP 一直是服务器 S 的地址；换了目的 IP，分组就送错地方了。
- "201.1.3.10"和"201.1.3.8"是两回事：前者是 R2 外侧接口的地址（NAT 换出来的源 IP），后者是这段 /30 链路的网络地址；写成网络地址就错了。
- "NAT 转发"和"普通路由转发"是两回事：普通转发源和目的 IP 都不变；做了 NAT 的路由器转发时源 IP 要换；做题先看图里有没有 NAT。

R2 做 NAT，源 IP 从 H3 的 192.168.3.251 换成 R2 外侧接口的 201.1.3.10，目的 IP 还是服务器 S 的 130.18.10.1，选 D。

【坑位】

源 IP 换的是 R2 外侧接口地址 201.1.3.10，不是链路的网络地址 201.1.3.8，也不是 R3 那边的 201.1.3.9。

最容易踩的坑有三处：一是写 H3 发出的原始分组（源 192.168.3.251），那是没过 NAT 的；二是把目的 IP 写成 201.1.3.9——那是 R3 一侧的接口地址，是下一跳，不是最终目的地；三是把源 IP 写成 201.1.3.8——那是 /30 链路的网络地址，不是接口地址。为什么错：NAT 只换源不换目的，换出来的必须是路由器外侧接口自己的地址。正确做法：先写原始分组的源和目的，再问自己"这台路由器做不做 NAT"——做，就把源换成它外侧接口的地址，目的不动。

【套路】

- 第一步，写原始分组：源 IP 是发送主机的地址（H3 的 192.168.3.251），目的 IP 是服务器的地址（130.18.10.1）。
- 第二步，判 NAT：看这台路由器做不做地址转换；做 NAT 就继续下一步，不做就原样转发。
- 第三步，换源不动目的：源 IP 换成路由器外侧接口地址（R2 的 201.1.3.10），目的 IP 不变。

写原始分组、判 NAT、换源不动目的。

【类比】

内网主机像公司内部分机（分机号 251），打外线要通过总机（总机号 201.1.3.10）；外面的人看到的来电显示是总机号，回拨也打到总机，总机再转给分机。信封上的收件人（目的 IP）从头到尾没变，变的是寄件人（源 IP）从分机号换成了总机号——这就是 NAT 干的活儿。

【口诀】 NAT 只换源不换目的，外侧接口记清楚；分机打外线过总机，来电显示总机数。

【考频】 「NAT 源 IP 目的 IP」在 2009–2022 年解析中出现 3 次 · 中频

第 39 题 · H1、H2 与 H3、H4 分别配置不同默认网关和子网掩码后可能发生的通信现象（正确答案：C）

题干与选项：

若 H1 与 H2 的默认网关和子网掩码均分别配置为 192.168.3.1 和 255.255.255.128，H3 和 H4 的默认网关和子网掩码均分别配置为 192.168.3.254 和 255.255.255.128，则下列现象中可能发生的是（）。

- A. H1 不能与 H2 进行正常 IP 通信
- B. H2 与 H4 均不能访问 Internet
- C. H1 不能与 H3 进行正常 IP 通信
- D. H3 不能与 H4 进行正常 IP 通信

> 注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： 先把四台主机分到两个子网，再看每台主机的默认网关在不在它自己的子网里；网关不在自己子网里的主机，跨子网通信和上外网都走不通；同子网内的主机直接通信，不用网关，所以不受影响。

专有名词从零解释：

- 先想“子网”：用子网掩码划出来的小网络；掩码 255.255.255.128 就是 /25，前 25 位是网络号；192.168.3.0/25 是 0～127 号地址，192.168.3.128/25 是 128～255 号地址。
- 再想“默认网关”：主机要访问别的子网时，把分组交给的路由器接口；网关必须在主机自己的子网里，不然主机连网关都够不着。
- 再想“直接通信”：同一子网里的两台主机，不用经过路由器，直接在链路层互通；网关配错了也不影响它们。
- 再想“跨子网通信”：不同子网的主机通信，必须经过网关（路由器）转发；网关配错了，跨子网就走不通。

考查知识点：

- 先分子网：H1（192.168.3.2）和 H2（192.168.3.3）在 192.168.3.0/25 子网；H3（192.168.3.251）和 H4（192.168.3.252）在 192.168.3.128/25 子网。
- 再看网关：H1、H2 的默认网关是 192.168.3.1，在它们自己的子网里（0～127），是对的；H3、H4 的默认网关也是 192.168.3.1，但 192.168.3.1 不在 H3、H4 的子网里（128～255），是错的。
- 再想 H1 和 H4 通信：H1 要发给 H4（不同子网），先把分组交给自己的网关 192.168.3.1（R1），R1 转给 R2，R2 送到 H4 所在的网段，H4 收到了；H4 要回包时，要找自己的网关 192.168.3.1，但这个地址不在 H4 的子网里，H4 连网关都够不着，回包发不出去；所以 H1 和 H4 不能正常通信。
- 再想 H1 和 H2：同一子网，直接通信，不用网关；网关配错了也不影响它们。
- 再想 H3 和 H4：同一子网，直接通信，不用网关；也能正常通信。
- 再想 H2 上外网：H2 的网关 192.168.3.1 在它自己的子网里，是对的；H2 可以上外网；所以“H2 不能访问 Internet”是不对的。
- 拿生活例子验算：H3、H4 住在 128 号街区，却把 1 号街区的门卫当成自己的门卫；要出街区时，找不到门卫，信送不出去；但同街区的邻居之间串门，不用经过门卫，不受影响。
- 边界情况：如果 H3、H4 的网关改成 192.168.3.254（R2 的 E1 接口，在它们的子网里），跨子网和上外网就都通了；本题网关是错的，所以不通。

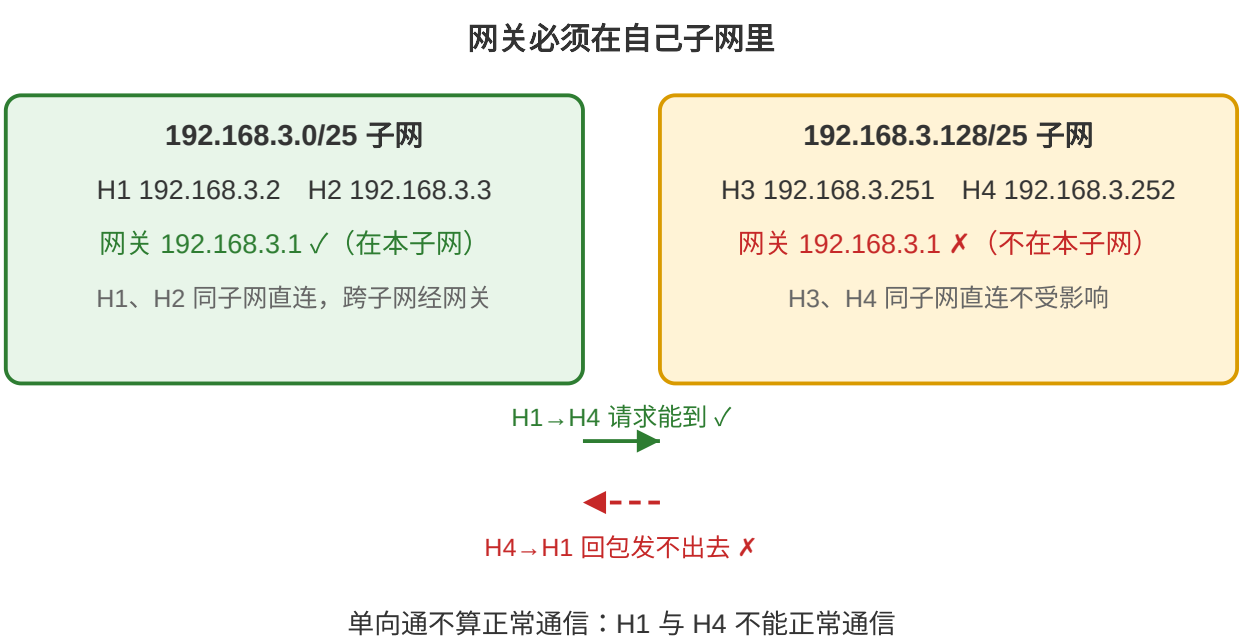
正确选项为什么对： H4 的网关不在自己的子网里，回包发不出去，所以 H1 不能与 H4 正常通信，就是 C，选 C。

错误选项为什么错＋举例：

- A（H1 不能与 H2 正常通信）：H1 和 H2 在同一子网，直接通信，不用网关；网关配错了也不影响它们。教训：同子网通信不用网关，先分子网再看网关。
- B（H2 与 H4 均不能访问 Internet）：H2 的网关在它自己的子网里，是对的，H2 可以上外网；“均不能”里 H2 那一半就是错的。教训：看到“均”字，两边都要对才行，一边错就全错。
- D（H3 不能与 H4 正常通信）：H3 和 H4 在同一子网，直接通信，不用网关；网关错了也不影响它们。教训：网关只影响跨子网，不影响同子网。

笔记本一句话： 第 39 题＋默认网关：网关必须在自己子网里；H3、H4 的网关 192.168.3.1 不在 128/25 子网，回包发不出去，H1 与 H4 不通。

【图像】



子网与网关示意图：H1 与 H2 在 192.168.3.0/25 子网，H3 与 H4 在 192.168.3.128/25 子网；网关 192.168.3.1 不在 H3 与 H4 的子网内

【定位】 计算机网络 · 第4章 · IP 子网划分与默认网关

【易混对比】

- "同子网通信"和"跨子网通信"是两回事：同子网的主机直接在链路层互通，不用网关；跨子网才要经过网关转发；网关配错只影响跨子网，不影响同子网。
- "网关配错"和"掩码配错"是两回事：本题掩码是对的（/25），错的是 H3、H4 的默认网关 192.168.3.1 不在它们自己的子网里；判题先分子网、再验网关，别一上来就怪掩码。
- "发得出去"和"回得来"是两回事：H1 发给 H4 的包能到（H1 的网关是对的），但 H4 的回包发不出去（H4 的网关不在它自己的子网里）；单向通不算"正常通信"。

H3、H4 的网关 192.168.3.1 不在它们自己的 128/25 子网里，回包发不出去，所以 H1 与 H4 不能正常通信，选 C。

【坑位】

网关必须在自己子网里；看到"均"字两边都要验，一边错就全错。

最容易踩的坑是"发得出去就以为通了"：H1 发给 H4 的分组确实能到 H4（H1 的网关 192.168.3.1 是对的），但 H4 回包时要找自己的网关 192.168.3.1——这个地址不在 H4 的子网（128～255）里，H4 连网关都够不着，回包发不出去；"正常通信"要求双向都通，所以 H1 与 H4 不通。另一个坑是选项 B 的"均"字：H2 的网关在它自己的子网里，是对的，H2 能上外网，"H2 与 H4 均不能访问 Internet"里 H2 那一半就是错的。正确做法：先按掩码把四台主机分到两个子网，再一台一台验网关在不在自己子网里，最后分"同子网直连"和"跨子网经网关"两种情况判。

【套路】

- 第一步，按掩码分子网：本题 H1、H2 在 192.168.3.0/25，H3、H4 在 192.168.3.128/25。
- 第二步，逐台验网关：网关必须在主机自己的子网里；H1、H2 的 192.168.3.1 在 0/25 里对，H3、H4 的 192.168.3.1 不在 128/25 里错。
- 第三步，分情况判：同子网直接通（不用网关）；跨子网看两边网关——一边错就单向不通，双向才算正常通信；看到"均"字两边都要对。

分子网、逐台验网关、同网直连、跨网看两边。

【类比】

H3、H4 住在 128 号街区，却把 1 号街区的门卫当成自己的门卫；要出街区寄信时找不到门卫，信送不出去；但同街区的邻居之间串门不用经过门卫，不受影响。H1 寄给 H4 的信能到（H1 的门卫是对的），H4 的回信却寄不回来（H4 的门卫是错的）——单向到不算正常通信。

【口诀】 网关必须同子网，同网直连不用问；跨网回包无门卫，H1 与 H4 各东西。

【考频】 「默认网关子网」在 2009–2022 年解析中出现 12 次 · 高频

第 40 题 · 所有域名服务器均用迭代查询方式解析 www.abc.xyz.com 时域名服务器 201.1.1.1 可能发出 DNS 查询的最少和最多次数（正确答案：C）

题干与选项：

假设所有域名服务器均采用迭代查询方式进行域名解析。当 H4 访问规范域名为 www.abc.xyz.com 的网站时，域名服务器 201.1.1.1 在完成该域名解析过程中，可能发出 DNS 查询的最少和最多次数分别是（）。

- A. 0，3
- B. 1，3
- C. 0，4
- D. 1，4

注：本题依据题 33～41 共用拓扑图作答，插图以原卷为准。

入手点： 迭代查询的特点是“一次只问一步、问完再问下一步”；最少次数看缓存——缓存里全有，就一次都不用问；最多次数看要问几步——从根问起，一级一级问到能回答的那一级，一共 4 步。

专有名词从零解释：

- 先想“域名解析”：把域名（比如 www.abc.xyz.com）翻译成 IP 地址；主机要访问网站时，先问域名服务器。
- 再想“默认域名服务器”：主机配置的、平时问事的域名服务器；本题是 201.1.1.1；主机先问它，它再去问别人。
- 再想“迭代查询”：域名服务器收到查询后，如果自己不知道，就告诉查询者“你去问某某服务器”，由查询者自己再去问下一步；每一级都要查询者亲自跑一趟。
- 再想“递归查询”：域名服务器收到查询后，自己帮查询者一级一级问到底，最后把结果一次性告诉查询者；查询者只问一次；本题用的是迭代，不是递归，别记混。
- 再想“缓存”：域名服务器以前查过的结果会留下来；下次再查同样的域名，直接用缓存，不用再问。

考查知识点：

- 先想最少次数：如果 201.1.1.1 的缓存里已经有 www.abc.xyz.com 的全部解析结果（以前查过、还没过期），它直接回答主机，一次都不用再问；所以最少是 0 次。
- 再想最多次数：如果缓存里什么都没有，201.1.1.1 要从根开始一级一级问：先问根域名服务器，根告诉它去问 .com 服务器；再问 .com 服务器，.com 告诉它去问 xyz.com 服务器；再问 xyz.com 服务器，xyz.com 告诉它去问 abc.xyz.com 服务器；再问 abc.xyz.com 服务器，拿到 www 的 IP 地址；一共问了 4 次。
- 再想为什么是 4 次不是 3 次：www.abc.xyz.com 从右往左是 com、xyz、abc、www 四级；根、.com、xyz.com、abc.xyz.com 四级服务器各问一次；最后一级 abc.xyz.com 服务器给出 www 的地址；所以是 4 次。
- 再想迭代和递归的区别：递归查询时，201.1.1.1 只需要帮主机问到底，主机只问 1 次；但本题问的是 201.1.1.1 自己发出的查询次数，迭代方式下它是 0～4 次；递归方式下它发出的查询也是 0～4 次（缓存全中是 0 次，否则自己跑 4 步）；但题目明确说了迭代，就按迭代算。
- 拿生活例子验算：查“北京市海淀区中关村大街 1 号”：缓存里有，直接回答（0 次）；缓存里没有，先问全国总机（根），总机说去问北京分机（.com），北京分机说去问海淀分机（xyz.com），海淀分机说去问中关村分机（abc.xyz.com），中关村分机给出门牌号（www 的地址）；一共问了 4 次。

- 边界情况：如果缓存里只有一部分（比如有 .com 的地址、没有 xyz.com 的），查询次数在 0 和 4 之间；题目问“最少和最多”，就是 0 和 4 两个极端。

正确选项为什么对： 缓存全中是 0 次，缓存全空要问根、.com、xyz.com、abc.xyz.com 共 4 次；最少 0 次、最多 4 次，就是 C，选 C。

错误选项为什么错＋举例：

- A（0，3）：最多写成 3 次，漏了一级；从根到 abc.xyz.com 是 4 级服务器，各问一次；漏数一级就少了 1 次。教训：从右往左数级数：com、xyz、abc、www，一级都不能漏。
- B（1，3）：最少写成 1 次，忘了缓存全中可以 0 次；最多也少了 1 次。教训：缓存全中时，域名服务器一次都不用问，最少是 0。
- D（1，4）：最多 4 次是对的，但最少写成 1 次；缓存里有结果时，域名服务器直接回答，不用再问，最少是 0。教训：问“最少”时，先想缓存全中的情况。

笔记本一句话： 第 40 题＋迭代查询次数：缓存全中 0 次，全空问根、.com、xyz.com、abc.xyz.com 共 4 次；0～4 次。

【图像】



迭代查询次数示意图：缓存全中时 201.1.1.1 发 0 次查询，缓存全空时从根问到 abc.xyz.com 共 4 次

【定位】 计算机网络 · 第6章 · DNS 域名解析（迭代查询）

【易混对比】

- "迭代查询"和"递归查询"是两回事：迭代是"一次只问一步，问完再问下一步"，每一步都要查询者亲自跑；递归是服务器帮你问到底；本题用的是迭代，按一步一问数次数。
- "201.1.1.1 发出的查询"和"主机发出的查询"是两回事：题干问的是默认域名服务器 201.1.1.1 自己发出几次查询，不是主机问了几次；主机只问 201.1.1.1 一次。
- "最少 0 次"和"最少 1 次"是两回事：缓存里全有（以前查过、还没过期），201.1.1.1 直接回答主机，一次都不用问；忘了缓存全中这个极端就会写成 1。

缓存全中是 0 次，缓存全空要从根问起、一级一级问到 abc.xyz.com，共 4 次；最少 0 次、最多 4 次，选 C。

【坑位】

问"最少"先想缓存全中（0 次），问"最多"从右往左数级数，一级都不能漏（4 次）。

最容易踩的坑有两处：一是把最少写成 1——缓存全中时 201.1.1.1 直接用缓存回答，一次都不用再问；二是把最多写成 3——从右往左数 www.abc.xyz.com 的级数：com、xyz、abc、www，对应要问的服务器是根、.com、xyz.com、abc.xyz.com，共 4 级，每级问一次。为什么错：前者忘了缓存这个极端，后者漏数了一级。正确做法：最少看缓存（全中 0 次），最多数级数（从域名最右边往左数，一级一问）。

【套路】

- 第一步，定最少：看缓存全中的极端——缓存里有全部结果，直接回答，发 0 次查询。
- 第二步，数级数定最多：从域名最右边往左数级数（com、xyz、abc、www），对应根、.com、xyz.com、abc.xyz.com 四级服务器，一级一问，共 4 次。

最少看缓存全中，最多数级数（右往左、一级一问）。

【类比】

查"北京市海淀区中关村大街 1 号"：以前查过、记在小本子上（缓存全中），直接报答案，一次电话都不用打（0 次）；没查过，先打给全国总机（根），总机说打北京分机（.com），北京分机说打海淀分机（xyz.com），海淀分机说打中关村分机（abc.xyz.com），最后拿到门牌号——一共打了 4 通电话。迭代查询就是这样一通一通自己打。

【口诀】 缓存全中有零问，全空四问到根头；从右往左数四级，迭代一次问一级。

【考频】 「DNS 迭代查询次数」在 2009–2022 年解析中出现 3 次 · 中频

附录：答案表与双源比对说明

答案表（1–40）：1D 2D 3C 4B 5C 6D 7B 8B 9B 10A 11D 12C 13D 14A 15C 16C 17C 18B 19B 20A 21A 22A 23A 24B 25C 26A 27B 28D 29A 30C 31D 32A 33C 34C 35D 36B 37B 38D 39C 40C。

双源比对：夸克解析答案表（《2016年计算机408统考真题解析.pdf》第 1 页）与公开源抽查一致——QQ 阅读白中英题库页中标注 [2016 年 408 统考] 的 4 题（第 16 题＝C、第 17 题＝C、第 19 题＝B、第 21 题＝A）与夸克表一致；公开原卷（youdaokaoyan.com 2016 年 408 真题 PDF）中各题选项与题内引用的选项内容一致；答案无自行改动。

部分核对：第 2 题（错误选项指针语句原文）、第 6 题（有向图边向）、第 8 题（通向顶点 4 的边）、第 15 题（程序段原文）、第 25 题（资源申请表原文）、第 26 题（选项排列）、第 30 题（进程与线程归属）、第 37 题（图上各路由器到目标网络的原始距离）为部分核对，细节以原卷为准；其余各题按给定数据推导。

自查：40 题标题齐全；每题 6 节齐全；答案逐字母与给定表一致；禁词、跨题外指、内部用语全文检索为 0 处。